

Reusable Designated Verifier NIZK from Lossy Trapdoor Functions

Riddhi Ghosal *

Ilan Komargodski †

Brent Waters ‡

Abstract

Understanding the minimal assumptions necessary for constructing non-interactive zero-knowledge arguments (NIZKs) for NP and placing it within the hierarchy of cryptographic primitives has been a central goal in cryptography. Unfortunately, there are very few examples of “generic” constructions of NIZKs or any of its natural relaxations.

In this work, we consider the relaxation of NIZKs to the designated-verifier model (DV-NIZK) and present a new framework for constructing (reusable) DV-NIZKs for NP generically from lossy trapdoor functions and PRFs computable by polynomial-size branching programs (a class that includes NC1). Previous “generic” constructions of DV-NIZK for NP from standard primitives relied either on (doubly-enhanced) trapdoor permutations or on a public-key encryption scheme plus a KDM-secure secret key encryption scheme.

Notably, our DV-NIZK framework achieves statistical zero-knowledge. To our knowledge, this is the first DV-NIZK construction from any “generic” standard assumption with statistical zero-knowledge that does not already yield a NIZK.

A key technical component of our construction is an efficient, unconditionally secure secret sharing scheme for non-monotone functions with randomness recovery for all polynomial-size branching programs. As an independent contribution we present an incomparable randomness recoverable (monotone) secret sharing for NC1 in a model with trusted setup that guarantees computational privacy assuming one-way functions. We believe that these primitives will be useful in related contexts in the future.

*UCLA. Email: riddhi@cs.ucla.edu.

†The Hebrew University of Jerusalem. Email: ilank@cs.huji.ac.il.

‡UT Austin, NTT Research. Email: bwaters@cs.utexas.edu.

***This work was primarily conducted while Riddhi Ghosal and Ilan Komargodski were at NTT Research.

Contents

1	Introduction	3
1.1	Our Results	4
1.2	Motivating new constructions of DV-NIZKs	5
2	Technical Overview	6
3	Preliminaries	12
3.1	Randomness Extraction	13
3.2	Length-Parameterized Lossy Trapdoor Functions	13
3.3	Zero-Knowledge Probabilistic Checkable Proof (ZK-PCP)	15
3.4	Weak Function-Hiding Single-Key Attribute-Based Encryption	16
3.5	Attribute-Based Secure Function Evaluation (ABSFE)	17
3.6	Reusable Designated Verifier Non Interactive Zero Knowledge	19
4	Secret Sharing with Randomness Recovery	21
4.1	A Construction for Polynomial-Size Branching Programs	22
5	The ABSFE Construction	28
5.1	Construction	28
6	DV-NIZK from ABSFE	37
7	References	41
A	Computational Monotone Secret Sharing Scheme with Randomness Recovery	46
A.1	Preliminaries	46
A.2	Linear Secret Sharing Scheme with Setup	48
A.3	Construction of LSSS with Randomness Recovery	51
A.4	Construction of Targeted Lossy PRG	59
B	ABSFE from Secret Sharing for Monotone Functions	61

1 Introduction

Interactive proof systems [GMR89, Bab85] enable a verifier with limited resources to decide an intractable language by communicating with a powerful but untrusted prover. Such systems guarantee *soundness*: the prover can only convince the verifier of true statements. An interactive proof is *zero-knowledge* [GMR89] if additionally, the interaction reveals nothing beyond the validity of the statement. It is well known (see [OW93]) that at least one of the above security properties (soundness or zero-knowledge) must be computational, i.e., apply only to computationally-bounded attackers. Furthermore, zero-knowledge proofs must be interactive.

Blum, Feldman, and Micali [BFM88] showed that interaction may be avoided if the prover and the verifier share a common reference string (CRS) chosen by a trusted third party and given to both the prover and the verifier. They call this notion of proofs that consist of only a single message from the prover to the verifier *non-interactive zero-knowledge* (NIZK), and show that it is realizable in the CRS model under computational assumptions.

We know several “generic” constructions of NIZKs from other well-known cryptographic primitives. For instance, NIZKs are implied by (doubly-enhanced) trapdoor permutations [BFM88, SMP87, FLS90], by circular-secure fully homomorphic encryption [CCH⁺19], and by strong KDM encryption [CCRR18]. While we know how to instantiate (doubly-enhanced) trapdoor permutations from factoring [BFM88, SMP87, FLS90, SCO⁺01, Gol11], the other primitives are much less standard and we still do not have plain-model instantiation thereof from standard (and falsifiable) assumptions. NIZKs do exist in the Random Oracle Model [FS86] (and without any additional assumption), but are generically uninstantiable in the plain model [CGH04, GK03].

In contrast, we do have many known constructions from concrete algebraic assumptions, including the Diffie-Hellman assumption over bilinear groups [CHK07, GOS12], (sub-exponential) Decisional Diffie-Hellman [CGJ⁺23], Learning with Errors (LWE) [PS19, Wat24], LPN and MQ [DJJ24], and more. While this is an impressive set of constructions, it is preferable to understand the generic relationship between NIZKs and other cryptographic primitives, allowing us to better position NIZKs in the hierarchy of cryptographic primitives. This modular approach to cryptographic research is crucial to the development of the theoretical foundations of cryptography.

Designated-verifier NIZK. We consider the well-known relaxation of NIZKs to the designated-verifier model (DV-NIZK). Here, a trusted third party generates a CRS together with a secret key, which is given (only) to the verifier and is used to verify proofs. We further consider by default the stronger notion of soundness for DV-NIZKs called *reusable soundness*, where soundness is held even if the scheme is used multiple times and a malicious prover can test whether the verifier accepts or rejects various proofs.¹² Since DV-NIZK is a weaker primitive than NIZK, it is likely easier to construct and also resulting constructions are likely to eventually lead to more practical realizations than regular NIZK.

Our current understanding of DV-NIZKs for NP is quite different from that of NIZKs for NP. Specifically, Lombardi et al. [LQR⁺19] gave a clean and modular construction of a DV-NIZK from generic assumptions: a public-key encryption scheme along with a KDM-secure secret-key encryption scheme. Both components can be instantiated under any of the CDH, LWE, or LPN assumptions. This compiler inherently results with an argument systems with computational soundness

¹There is a seemingly weaker notion for DV-NIZKs called *one-time soundness* wherein soundness is guaranteed only for a single proof of a single statement. DV-NIZKs for NP with one-time soundness can be constructed from any public-key encryption scheme [PSV06].

²Plain NIZKs are “automatically” reusable due to their public verifiability property.

and computational zero-knowledge. Besides this compiler, all other DV-NIZK for NP constructions are tailored for a particular algebraic assumption or rely on ad-hoc intermediate abstractions [CH19, KNY21, QRW19, LPWW20], and we are not aware of any other construction of DV-NIZKs for NP from a generic “well accepted” primitive, except constructions that ultimately result with plain NIZK.

The landscape of statistical designated verifier non-interactive zero-knowledge (DV-NIZK) arguments has also been intriguing. While DV-NIZKs have been realized from various specific assumptions [PS19, Wat24, LPWW20], no known constructions are known from a generic primitive without already achieving statistical NIZK arguments.

This state of affairs raises the following natural questions:

1. Is there a different way of getting DV-NIZKs for NP generically without relying on KDM-secure encryption?
2. Is there any way of getting statistical DV-NIZK argument for NP generically that does not already give us statistical NIZKs?

In this work, we construct reusable statistical DV-NIZK arguments from Lossy Trapdoor Functions (LTDFs) and PRFs in PBP. LTDFs are not known to be implied by KDM-secure encryption, thus, our result provides a new technique to achieve DV-NIZKs from a generic primitive. In doing so, we take a step forward in exploring whether a fundamental qualitative distinction exists between statistical DV-NIZKs and statistical NIZK arguments. Specifically, we present the first compiler that attains statistical zero-knowledge from a generic assumption without being known to imply statistical NIZK arguments.

1.1 Our Results

We present a new framework for constructing (reusable) DV-NIZKs from generic assumptions. Our framework yields reusable DV-NIZKs generically from lossy trapdoor functions and PRFs in PBP. The class PBP stands for all functions computable by polynomial-size branching programs, a class that includes NC1 (and is equal to LOGSPACE) [Cob66, AKG05, Wik24]. Our lossy trapdoor functions need to be length-parameterized, meaning that the output size is independent of the input size; such lossy trapdoor functions (and also the PRF) can be constructed from DDH or LWE [PW11, WZ24, NR04]. Our framework can be instantiated to result with computational soundness and statistical zero-knowledge.

Theorem 1.1 (Informal; see Corollary 6.6). *Assuming the existence of a length-parametrized trapdoor lossy functions (see Definition 3.2) and a PRF in PBP, there exist reusable designated-verifier NIZK for NP with computational soundness and statistical zero-knowledge.*

Technically, we build a form of an Attribute-Based Encryption (ABE) scheme [SW05, GPSW06] that is known to be equivalent to DV-NIZKs (assuming public-key encryption exists) [LQR⁺19]. Specifically, this notion of ABE is a single-key scheme satisfying a certain “function-hiding under decryption queries” property. While the previous work [LQR⁺19] built it from a public-key encryption scheme plus KDM-secure secret-key encryption scheme or Hinting PRG [KW19], we build it from lossy trapdoor functions and a PRF in PBP. As mentioned, our construction also has the added advantage of achieving statistical zero-knowledge, whereas the framework of [LQR⁺19] inherently results with computational soundness and ZK (due to their black-box usage of the single key ABE of [SS10]).

Secret sharing for non-monotone functions with randomness recovery. A key technical component in our construction is a secret sharing scheme with *randomness recovery* for all (even non-monotone) functions in PBP. A secret sharing scheme for non-monotone functions (sometimes referred to as non-monotone secret sharing or conditional disclosure of secrets [GIKM98, BI05, VV15, ABI⁺23]) is a method for sharing a secret among n parties relative to a (not necessarily monotone) function $F: \{0,1\}^n \rightarrow \{0,1\}$. The secret sharing scheme outputs $2n$ shares $(s_1, t_1, s_2, t_2, \dots, s_n, t_n)$ and for any input $(x_1, \dots, x_n) \in \{0,1\}^n$, as above, the collection of shares $\{s_i: x_i = 1\} \cup \{t_i: x_i = 0\}$ determines the secret if $F(x_1, \dots, x_n) = 1$ and does not reveal anything otherwise. The randomness recovery property provides a procedure that recovers all the $2n$ shares from any subset of $< 2n$ shares that are authorized according to F .

Our contribution in this context is an efficient secret sharing scheme with randomness recovery for all PBP; this scheme is *unconditionally secure*. Our scheme is obtained by adapting the well-known st-connectivity secret sharing scheme (see, e.g., [Bei11, BR89]) to handle all functions in PBP.

Theorem 1.2 (See Theorem 4.16). *There exists an efficient secret sharing scheme with randomness recovery for all functions in PBP.*

Secret sharing for non-monotone functions should be contrasted with “standard” secret sharing schemes that can only support monotone functions. Indeed, in a standard secret sharing scheme the n shares $t_1, \dots, t_n$ are essentially fixed to $\perp$, and so having $x_i = 1$ can only increase the amount of information available to recover the secret. Interestingly, Hajiabadi et al. [HKV24a] study the notion of “standard” secret sharing with randomness recovery and obtain three main results. First, every access structure admits a secret sharing scheme with randomness recovery (albeit inefficient). Second, even for very simple access structures, obtaining randomness recovery requires the use of one-way functions. Finally, they note that randomness recovery can be obtained for any NC1 access structure by using a one-time KDM-secure secret-key encryption (which they in turn base on a new assumption they call linear-resistant PRGs). Since our secret sharing scheme in Theorem 1.2 is unconditionally secure, we obtain an intriguing separation between “standard” secret sharing and secret sharing for non-monotone functions, both with randomness recovery.

Lastly, as an independent contribution we give a “standard” secret sharing scheme with randomness recovery for all monotone-NC1 from only one-way functions (in fact, the construction applies to all monotone functions computable by polynomial-size span programs [KW93]). This scheme, however, requires trusted setup (whose randomness the recovery procedure does not recover). We give it for completeness, hoping that it will be useful in related contexts in the future.

Theorem 1.3 (See Theorem A.31). *Assuming that a one-way function exists and trusted setup, there is a (computational) secret sharing scheme with randomness recovery for every monotone function in NC1.*

1.2 Motivating new constructions of DV-NIZKs

Before presenting a technical overview of our construction, we argue why it is crucial to have diverse approaches to constructing DV-NIZKs.

First, a major open problem in our community has been understanding the complexity of NIZKs, i.e., identifying the minimal cryptographic assumptions necessary to build them. Very little is known in this area, and developing new techniques to construct NIZKs is crucial for making progress on this question. New constructions of DV-NIZKs are likely to introduce new methodologies that contribute to the diversity of techniques for addressing this fundamental problem. Moreover, the

minimal assumptions needed to construct a one-time KDM-secure SKE remain unclear, making sole reliance on this primitive severely restrictive in understanding the complexity of DV-NIZKs. Additionally, newer techniques offer crucial flexibility for future work in deriving DV-NIZKs from weaker assumptions.

Second, multiple approaches to solving a problem are essential for gaining a deeper understanding of the primitive. Consider, for instance, the case of CCA-secure PKE, where Peikert and Waters [PW11] demonstrated a construction using Lossy Trapdoor Functions in 2008. This spurred follow-up work deriving CCA-secure PKE from diverse primitives such as one-time KDM-secure SKE for projection functions [KMT19] and hinting PRGs [KW19]. These efforts, though based on incomparable primitives, provided new insights and ultimately led to a construction using the weaker assumption of Injective Trapdoor Functions [HKW20], after a decade of exploration. For DV-NIZKs, an ideal end goal is to establish an equivalence between DV-NIZKs and CCA-secure encryption. A construction from Lossy Trapdoor Functions is a step in this direction, as a construction from LTDFs can potentially be subsumed by one from Trapdoor Functions, the weakest known generic primitive implying CCA-secure encryption.

Finally, we have seen ample evidence in the past that such techniques have proven useful for solving independent problems across different areas. Our construction leverages the novel underlying primitive of secret sharing with randomness recovery (both monotone and non-monotone), which we believe will also find applications in several other problems.

2 Technical Overview

Lombardi et al. [LQR⁺19] proved that *Single-Key Attribute-Based Encryption (ABE) with Weak Function-Hiding property* implies Designated Verifier NIZK argument for NP³. Recall that an ABE scheme allows us to encrypt a message m under some public parameters $\mathbf{pp}$ (generated by a setup procedure along with a master secret key $\mathbf{msk}$) with respect to an attribute x to yield a ciphertext $\mathbf{ct}$. The ciphertext can be decrypted using a secret key $\mathbf{sk}_f$ associated with a particular function f . The decryption algorithm recovers the message m if and only if $f(x) = 1$. (In this work, we will interpret f as a predicate). The semantic security of an ABE requires that if $f(x) = 0$, then the ciphertext $\mathbf{ct}$ must not reveal any information about the message m , even upon given access to the function secret key $\mathbf{sk}_f$. It turns out that a weaker version that requires semantic security to hold only in the presence of a *single* function key suffices to construct DV-NIZK.⁴ In addition, for the construction of DV-NIZK, the ABE needs to satisfy a certain function-hiding property: for *all* functions f , oracle access to the decryption oracle $\text{Dec}(\mathbf{sk}_f, \mathbf{ct})$ must not reveal any information about f other than whether $f(x) = 0$ or not.⁵ We consider schemes where the attribute x is publicly known (it can be given out with the ciphertext). This is formally captured by a simulation based definition which informally states that an oracle call to the decryption function should be efficiently simulated using only the master secret key $\mathbf{msk}$ and the value of $f(x)$.

In this work, our primary focus is constructing the above ABE scheme for an appropriate function f that will be determined by the NP statement of interest.

Challenges of constructing ABE with function-hiding. While we can construct single-key semantically secure ABE directly from Public Key Encryption [SS10], the main challenge arises

³In fact they prove equivalence of these two primitives assuming public-key encryption exists.

⁴In general, an adversary trying to break the semantic security of an ABE scheme is allowed to make oracle queries for arbitrarily many function keys

⁵This is weaker than the usual notion of function-hiding for ABE where one requires that the function key $\mathbf{sk}_f$ does not leak information about f .

when we want to achieve any form of function-hiding. The crux of the problem is that an adversary can behave in a way which forces the real world decryption and the simulator to output different messages, which is bad for function-hiding. The function-hiding property seems to be closely related to Chosen Ciphertext Attack (CCA) security in the following sense: In CCA security, one needs to prove security even when an adversary is given access to the decryption oracle. This is similar to the function-hiding setting where the adversary, given access to the decryption oracle, can try to generate malformed ciphertexts to extract information about the underlying function. This similarity has been observed in prior work, and naturally, several tools and techniques which are useful to achieve CCA security have turned out to be useful to construct DV-NIZK as well. In CCA land, one key technique to deal with this issue is to generate two “correlated” ciphertexts, one of which is decrypted using the function secret key and the other is decrypted using an additional secret key which is only a part of master secret key (not included as the function secret key). The key tool that is used to achieve this property is *Randomness Recovery*. A cryptographic primitive with *Randomness recovery* is an object where the trapdoor process allows one to (partially) recover the randomness used to generate the *hard* instance. For example, the notion of PKE with randomness-recovery has been well studied [LQR⁺19, HKW20]. This is a semantically secure PKE with two additional properties: (1) The decryption process outputs both the message and randomness used during encryption, and (2) there is an efficient and deterministic algorithm that correctly outputs the message upon input the ciphertext and the randomness used for encryption (without having access to the decryption key). Such schemes have been well studied in the context of both CCA security and DV-NIZK in [HKW20, LQR⁺19], and thus randomness recovery is a natural tool to use in our construction.

ABE with randomness recovery. In prior work, it has been the impression that constructing ABE with weak function-hiding can be achieved by constructing an ABE scheme with some form of randomness recovery [GHMO21, LQR⁺19, HKV24b]. In this work, our goal is to construct an ABE scheme which will satisfy randomness recovery which in turn will naturally yield a single-key semantically secure ABE with weak function hiding. In particular, we are going to construct a standard ABE scheme for a function f with the following additional property: (1) For any attribute x such that $f(x) = 1$, the decryption process can extract both the message and randomness used during encryption, and (2) for any attribute x such that $f(x) = 1$, one can use the master secret key (without access to the function f and secret key) to produce outputs consistent with the real world decryption algorithm.

Construction template. Without loss of generality we will assume a function C that takes as input (f, x) and outputs $f(x)$. We will then construct an ABE for C where the key generation algorithm outputs a public key pk , master secret key msk and secret key sk when queried f . The msk is not revealed to the decryptor and encryptions are generated with respect to attribute x and the particular public key. This abstraction is called Attribute Based Secure Function Evaluation (ABSFE) [LQR⁺19] and is generalization to the usual notion of single-key ABE for the function $C(f, \cdot)$. Moreover in ABSFE the function-hiding analogue is referred to as key-hiding where oracle access to the decryption function does not reveal any information about the “key” f . From here onwards, we will refer to f as the key.

Building blocks. The general template of our construction follows. While this template will not achieve all the properties that we want, we gradually augment this primary construction with additional tools in a step-wise manner, eventually obtaining the final construction.

As building blocks, we will use:

1. A family of pairwise independent hash functions $\mathcal{H}$.
2. A secret sharing scheme with access function $C(\cdot, x) : \{0, 1\}^n \rightarrow \{0, 1\}$ that has a randomized share generating algorithm and a deterministic reconstruction algorithm⁶. The share generating algorithm that takes as input a secret s and produces shares $(a_{1,0}, a_{1,1}, \dots, a_{n,0}, a_{n,1})$, and the reconstruction algorithm that takes an input “authorized” sets of shares and outputs the secret and satisfies usual correctness and privacy properties. Specifically, if $C(f, x) = 1$, then $\{a_{i,f_i}\}_{i \in [n]}$ should correctly reconstruct the secret s . On the other hand, if $C(f, x) = 0$, then $\{a_{i,f_i}\}_{i \in [n]}$ should reveal nothing about the secret s in the information theoretical sense. As mentioned in the introduction, such a scheme has been referred to as a secret sharing scheme for non-monotone functions in the literature.

For the time being, let us assume that an efficient secret sharing scheme of this form exists for $C(\cdot, x)$. We will discuss later in this section when this is possible.

3. An (injective) trapdoor function with setup. Such a function has an additional setup algorithm which on input the security parameter outputs the function g along with a trapdoor g^{-1} . The trapdoor satisfy the natural property that $g^{-1}(g(x)) = x$.

Informally, our construction follows the blueprint here:

- The setup algorithm samples a hash function H from a pairwise-independent hash family as public parameter.
- The key generation procedure takes as input the key f . Let f be n bits long. Then it generates a sequence of functions along with its inverse with using the trapdoor function setup. Call them $(g_{i,b}, g_{i,b}^{-1})_{i \in [n], b \in \{0,1\}}$. The function description $g_{i,b}$ are set as public key $\mathbf{pk}$, and the secret key $\mathbf{sk}$ includes the key f along with trapdoor keys $\{g_{i,f_i}^{-1}\}_{i \in [n]}$. The master secret key contains all the other trapdoors.
- To encrypt a message m under attribute x , first sample a random secret s and use the share generating algorithm to produce $(a_{1,0}, a_{1,1}, \dots, a_{n,0}, a_{n,1})$. The ciphertexts are set as $\{\mathbf{ct}_{i,b} \leftarrow g_{i,b}(a_{i,b})\}_{i \in [n], b \in \{0,1\}}$. Finally, use H to generate $\mathbf{ct}_0 = m \oplus H(s)$.
- Decryption works by first computing shares corresponding to the indices for which the secret key has the trapdoor, i.e., for all $g_{i,f_i}^{-1} \in \mathbf{sk}$, compute $a_{i,f_i} = g_{i,f_i}^{-1}(\mathbf{ct}_{i,f_i})$. Then use the secret sharing reconstruction procedure to generate s . Finally output $\mathbf{ct}_0 \oplus H(s)$.

Correctness. Whenever (f, x) is such that $C(f, x) = 1$, correctness is straightforward because (1) the inversion correctness of the trapdoor function ensures that the correct shares $\{a_{i,f_i}\}_{i \in [n]}$ are computed and (2) $C(f, x) = 1$, therefore $\{a_{i,f_i}\}_{i \in [n]}$ are authorized to extract the secret s . Hence, the reconstruction correctness of secret sharing ensures that $\mathbf{ct}_0 \oplus H(s) = m$.

Message-hiding for ABSFE. To show message-hiding, we have to prove that if the key f is not authorized to reveal the secret, i.e., $C(f, x) = 0$, then encryption of m_0 is indistinguishable from encryption of m_1 . The most natural way to proceed with the proof would be to use the randomness extraction property of H and replace $m \oplus H(s)$ in the encryption step to a uniform random value.

⁶The notation $C(\cdot, x)$ means the function C hardcoded with x .

Observe that the view of an adversary includes all the functions: $\{g_{i,b}\}_{i \in [n], b \in \{0,1\}}$, some of the shares: $\{a_{i,f_i}\}_{i \in [n]}$ and trapdoor function evaluations of the other shares: $\{g_{i,1-f_i}(a_{i,1-f_i})\}_{i \in [n]}$, and $m_0 \oplus H(s)$. We can appeal to the privacy of the secret sharing scheme and say that $\{a_{i,f_i}\}_{i \in [n]}$ does not reveal any information about s . However the adversary also sees $\{g_{i,1-f_i}(a_{i,1-f_i})\}_{i \in [n]}$, and because $g_{i,1-f_i}$ is injective, the conditional entropy of s given $\{g_{i,1-f_i}(a_{i,1-f_i})\}_{i \in [n]}$ is 0 as s can be reconstructed efficiently given all the shares. So, there is absolutely no entropy left to use the randomness extractor H and we cannot hope to use any sort of Leftover Hash Lemma for randomness extraction.

Key Ingredient 1: Lossy Trapdoor Function. Trapdoor functions do not seem sufficient to achieve message hiding. To solve this issue, we will replace them with Lossy Trapdoor Functions (LTDF). The notion of Lossy trapdoor function was introduced by Peikert and Waters [PW08] which is a public function f that behaves in one of two modes.

- The first mode gives a (injective) trapdoor function: given a suitable trapdoor for g , i.e., g^{-1} , the input x can be efficiently recovered from $g(x)$. In this mode, the setup outputs the trapdoor along with the function description.
- In the second mode g statistically loses a significant amount of information about its input, i.e., g 's image is significantly smaller than its domain.

The two behaviors are computationally indistinguishable: given just the description of g , no efficient adversary can tell whether g is injective or lossy.

Once we make this change, i.e., $(g_{i,b}, g_{i,b}^{-1})$ are now generated by the setup for lossy trapdoor function, we can indeed make the message hiding proof go through without affecting correctness. In more detail, we will generate the functions in the injective mode in the construction, and therefore correctness remains unaltered. However, to prove message hiding, we will first switch the lossy trapdoor function setup such that $\{g_{i,1-f_i}\}_{i \in [n]}$ are generated in the lossy mode. Now, the view of the adversary again contains $\{g_{i,1-f_i}(a_{i,1-f_i})\}_{i \in [n]}$ but now the lossiness ensure that the residual entropy of s given these evaluation is high enough for the randomness extractor to work. Indeed, we have to pick the parameters carefully to make sure that there is enough information ambiguity in the lossy evaluation to apply the Leftover Hash Lemma. In particular, we need to use a slightly stronger version of lossy trapdoor functions called length-parameterized lossy trapdoor functions. Length-parameterized lossy (trapdoor) functions, introduced in [WZ24], are similar to lossy (trapdoor) functions with a stronger requirement that the setup algorithm takes as input an explicit input length parameter ℓ_{in} and the range of the function in the lossy mode is independent of ℓ_{in} . The original constructions of lossy trapdoor function in [PW08] from DDH and the recent constructions in [WZ24] from LWE satisfy this property.

Remark 2.1. Since we know the indices i for which $f_i = 0$, we can set the lossy trapdoor function keys in the constructions in way that does not require us to switch from injective to lossy mode for the keys $\{g_{i,1-f_i}\}_{i \in [n]}$ in the proof. Specifically, we sample $\{g_{i,1-f_i}\}_{i \in [n]}$ in the lossy mode and $\{g_{i,f_i}\}_{i \in [n]}$ in the injective mode, thereby avoiding the need to rely on the computational setup indistinguishability property of the lossy trapdoor function. In this way, we get the property that message-hiding is statistical (assuming the secret sharing is information theoretically secure).

Key-hiding for ABSFE. Unfortunately, the template above, as is, does not satisfy key-hiding. It turns out that an adversary with access to a decryption oracle can send malformed ciphertexts as queries that will leak non-trivial information about the key f . Concretely, consider the following

scenario : there is an adversary who is aware of some key and attribute pair (f^*, x) such that $C(f^*, x) = 1$ and they also know that the description of f^* is 0 everywhere other than its most significant bit.

Therefore, if the key generation algorithm for the ABSFE generated keys corresponding to f^* and if the encryption is done honestly as prescribed above, then the shares $(a_{1,1}, a_{2,0}, \dots, a_{n,1})$ should be sufficient to recover the secret s and therefore the encrypted message.

However, the adversary chooses to not generate the encryptions honestly but rather does the following tampering: To encrypt a message m under attribute x , the adversary first samples a random secret s and use the share generating algorithm to *honestly* produce $(a_{1,0}, \dots, a_{n,1})$. At this point, the adversary deviates from the encryption procedure. Instead of setting $\{ct_{i,b} \leftarrow g_{i,b}(a_{i,b})\}_{i \in [n], b \in \{0,1\}}$, it first picks arbitrary values $\{a'_{i,1-f_i^*} \neq a_{i,1-f_i^*}\}_{i \in [n]}$. Then it sets $ct_{i,1-f_i^*} = g_{i,1-f_i^*}(a'_{i,1-f_i^*})$ for all $i \in [n]$ and $ct_{i,f_i^*} = g_{i,f_i^*}(a_{i,f_i^*})$ for all $i \in [n]$. Finally, it use H to generate $ct_0 = m \oplus H(s)$.

Such an adversary can learn non-trivial information about the key using the response of the decryption oracle and break the key-hiding property. Consider the two cases:

- If the key generation produced secret keys corresponding to f^* , then the secret key has $\{g_{i,f_i^*}^{-1}\}_{i \in [n]}$, and it can therefore extract $(a_{1,1}, a_{2,0}, \dots, a_{n,1})$. Since the description of f^* has a 1 in its most significant bit only, the decryption oracle can reconstruct some secret s' just by using $(a_{1,1}, a_{2,0}, \dots, a_{n,1})$. And because the adversary did not modify the *honestly generated* $(a_{1,1}, a_{2,0}, \dots, a_{n,1})$, the reconstruction must always produce the correct secret $s' = s$. Thus the decryption oracle responds with $ct_0 \oplus H(s') = ct_0 \oplus H(s) = m$.
- On the other hand, if the key generation produced secret keys corresponding to some $f \neq f^*$ such that $C(f, x) = 1$, then there is no guarantee that the s' reconstructed will match the same secret s used in the encryption. The reconstruction algorithm receives shares which are not generated by an honest share generating process and therefore provides no guarantee of reconstruction correctness. In fact in the worst case where the description of f has 0 in the most significant position and 1 everywhere else, then the reconstruction process is run only on tampered shares which are completely independent of the secret s . Therefore it is very likely that $ct_0 \oplus H(s') \neq ct_0 \oplus H(s) \neq m$.

To conclude, depending on if the decryption oracle returns the correct message m or otherwise, then the adversary can be fairly confident whether the key f^* was used or not.

In order to prevent such malicious behavior, there has to be some way to perform a consistency checks while decrypting to ensure that the adversary cannot take advantage of malformed ciphertexts.

Key Ingredient 2: Secret Sharing with Randomness Recovery. Our problem with proving key-hiding is resolved if we have a secret sharing scheme with an additional randomness recovery property, i.e., the reconstruction algorithm upon input $C(\cdot, x)$ and $\{a_i\}_{i:f_i=1}$ correctly outputs all other shares, i.e., $\{a_i\}_{i:f_i=0}$ along with s . Given such a primitive, we can modify our decryption as follows:

First compute shares corresponding to the indices for which the secret key has the trapdoor, i.e., for all $g_{i,f_i}^{-1} \in \text{sk}$, compute $a_{i,f_i} = g_{i,f_i}^{-1}(ct_{i,f_i})$. Then use the secret sharing reconstruction to generate $s, (a_{1,0}, a_{1,1}, \dots, a_{n,0}, a_{n,1})$. To test for malformed ciphertexts, we run a consistency check. Specifically, check if $g_{i,b}(a_{i,b}) = ct_{i,b}$ for all $i \in [n], b \in \{0,1\}$. If yes, then output $ct_0 \oplus H(s)$. Otherwise output $\perp$.

Now, if our secret sharing has the additional property that the regenerated shares are indeed correctly distributed, the attack described above no longer works. In particular, if an adversary tries to use malformed ciphertexts, then the consistency checks in the decryption algorithm should fail and it will output $\perp$. Thus, such a secret sharing scheme suffices to achieve key-hiding for the ABSFE. We refer readers to Section 5 for formal details.

Constructing the secret sharing scheme As mentioned above, a secret sharing scheme for non-monotone function $F : \{0, 1\}^n \rightarrow \{0, 1\}$ splits a secret into $2n$ shares and satisfies the usual correctness and privacy properties, namely, (1) the collection of shares $\{s_i : x_i = 1\} \cup \{t_i : x_i = 0\}$ determines the secret if $F(x_1, \dots, x_n) = 1$, and (2) if $F(x_1, \dots, x_n) = 0$, then $\{s_i : x_i = 1\} \cup \{t_i : x_i = 0\}$ does not reveal anything about the secret (information theoretically). In addition to these, a secret sharing with randomness recovery has a deterministic regeneration procedure that can generate all $2n$ shares given $\{s_i : x_i = 1\} \cup \{t_i : x_i = 0\}$ along with the secret s if $F(x_1, \dots, x_n) = 1$.

In this work, we construct an information theoretically private secret sharing scheme for any non-monotone functions F that can be computed by a polynomially sized branching program. Such functions are said to belong to the complexity class PBP and can be represented using a Directed Acyclic Graph (DAG) with $n + 2$ nodes⁷ ($v_1, \dots, v_{n+2}$) where v_1 is assigned as the source node, and v_{n+1} and v_{n+2} are assigned as accept and reject sink nodes respectively. All non sink nodes labelled $\{v_1, \dots, v_n\}$ have exactly two outgoing edges labelled (i, b) , for $i \in [n]$ and $b \in \{0, 1\}$. The DAG has the property that the set of edges marked as $\{(i, x_i)\}$ has a path from v_1 to the accept node if and only if $F(x) = 1$ for some input x . At a high level, the secret sharing scheme works as follows:

- **Share Generation.** Given a secret s , assign it to v_{n+1} . Assign 0 to v_1 and v_{n+2} . For all other nodes, assign random values r_i . Let val_i be the value assigned to node v_i . Every edge is now assigned a share $a_{i,b} = val_j - val_i$ if (i, b) is an edge between nodes v_i and v_j . These are the final shares, i.e., $\{a_{i,b}\}_{i \in [n], b \in \{0,1\}}$.
- **Secret Reconstruction.** If one is given access to shares $\{a_{i,x_i}\}$ and $F(x) = 1$, then we know by property of the DAG that the edges corresponding to these shares contain among them a directed path from v_1 to v_{n+1} and such a path can be efficiently found. To reconstruct the secret simply add the shares along this path. Correctness is easy to see by observing that summing shares along a path has a telescopic affect that cancels all intermediate values and results in $val_{n+1} - val_1$ that equals the secret s .
- **Share Regeneration.** Observe that every set of shares $\{a_{i,x_i}\}_{i \in [n]}$ contains share corresponding to exactly one of out two outgoing edges from every non-sink node. Therefore, the set $\{a_{i,x_i}\}_{i \in [n]}$ contains shares along a path from every node v_i to either v_{n+1} or v_{n+2} . If $F(x) = 1$, then using the above bullet point, one can recover the secret s , and thus get access to val_{n+1} . It is known that $val_1 = 0$ and $val_{n+1} = 0$. Now to compute val_i , one can simply add shares along the path from v_i to v_{n+1} or v_{n+2} . Once all $\{val_i\}_{i \in [n+2]}$ are recovered, it is straightforward to compute the shares $\{a_{i,b}\}_{i \in [n], b \in \{0,1\}}$.

Additional technical details. We have hidden some details under the rug in this description. In the above description, we have argued shared regeneration correctness only when the shares the

⁷The number of nodes in the graph will not be $n + 2$ in general. Rather it will be some polynomial in n . However for the sake of simplicity let us assume it is $n + 2$ for now as it does not affect the construction in a significant manner. The technical sections have accurate parameters.

honestly generated using the share generation algorithm. However, an adversary trying to break key-hiding need not generate shares using this procedure. Therefore, for the key-hiding proof to work we have to ensure that there is some way for the share regeneration procedure to detect “invalid” shares. This is captured by the “Regeneration Soundness” property in our formalization.

We refer readers to Section 4 for more details.

In our application of DV-NIZK, the function C is essentially a PRF and (f, x) are the PRF key and evaluation point respectively. Therefore we need to construct the secret sharing with randomness recovery scheme for the access function $\text{PRF}(\cdot, x)$. Since we build such a secret sharing scheme for a functions restricted to the complexity class PBP, we also need to additionally assume the existence of PRF in PBP.

We refer readers to Section 5 and Section 6 for more details.

Remark 2.2. Our construction does not achieve a dual-mode DV-NIZK construction. In more detail, while we can achieve statistical ZK if we set up the functions in the Lossy Mode for key generation, this does not translate to a statistically sound DV-NIZK if the LTDFs functions keys are instead set up in the injective mode. The reasoning is somewhat subtle: if our construction only relied on Lossy Trapdoor functions then this would indeed yield statistical soundness. However we also need a PRF to argue soundness of the underlying PCP that restricts us to computational security.

3 Preliminaries

Notations. We use $\mathbf{e}_i$ to denote a vector with 1 at the i^{th} index and 0 everywhere else. For a matrix M , $M_{i,j}$ denotes the entry at the (i, j) position. $M_{i,\cdot}$ and $M_{\cdot,i}$ denote the i^{th} row and column respectively.

The *statistical distance* between two random variables X and Y having the same (countable) domain $\mathcal{D}$ is

$$\Delta(X, Y) = \frac{1}{2} \sum_{v \in \mathcal{D}} |\Pr[X = v] - \Pr[Y = v]|.$$

We say that two variables are ϵ -close if their statistical distance is at most ϵ . We use the notation $D_1 \approx_\epsilon D_2$ to denote that the statistical distance between two distributions D_1 and D_2 is ϵ .

Negligible Function: A function $f : \mathbb{N} \rightarrow (0, 1)$ is said to be negligible if for every positive constant c , there exists $n_0 \in \mathbb{N}$ such that for all $n > n_0$, $f(n) \leq n^{-c}$.

Two distributions D_1 and D_2 (indexed by security parameter λ) are said to be computationally indistinguishable if for all PPT adversaries $\mathcal{A}$, there exists a negligible function negl such that

$$\left| \Pr_{x \leftarrow D_1} [\mathcal{A}(x) = 1] - \Pr_{x \leftarrow D_2} [\mathcal{A}(x) = 1] \right| \leq \text{negl}(\lambda).$$

Analogously, two distributions D_1 and D_2 are statistically indistinguishable if for all (not restricted to PPT) adversaries $\mathcal{A}$, there exists a negligible function negl such that

$$\left| \Pr_{x \leftarrow D_1} [\mathcal{A}(x) = 1] - \Pr_{x \leftarrow D_2} [\mathcal{A}(x) = 1] \right| \leq \text{negl}(\lambda).$$

Definition 3.1. A family $\mathcal{H}\{H : D \rightarrow R\}$ is said to be pairwise independent, if for any two distinct elements $x_1 \neq x_2 \in D$, and any two $y_1, y_2 \in R$,

$$\Pr_{H \in \mathcal{H}}[H(x_1) = y_1 \text{ and } H(x_2) = y_2] = \frac{1}{|R|^2}.$$

3.1 Randomness Extraction

The *min-entropy* of a random variable X is

$$H_\infty(X) = -\log\left(\max_x \Pr[X = x]\right).$$

In several settings, the variable X is correlated with another variable Y whose value is known to the adversary. The notion of conditional minimum entropy quantifies the amount of information needed to describe the outcome of X given the outcome of another random variable Y . Formally, it is defined as $H_\infty(X|Y = y) = -\log(\max_x \Pr[X = x | Y = y])$. In this work, we use a notion called *average conditional min-entropy* of Dodis, Reyzin, and Smith [DRS04], which captures the remaining unpredictability of X conditioned on the value of Y :

$$\tilde{H}_\infty(X|Y) := -\log\left(\mathbb{E}_{y \leftarrow Y}\left[2^{-H_\infty(X|Y=y)}\right]\right).$$

The average min-entropy corresponds exactly to the optimal probability of guessing X , given knowledge of Y . The following bound on average min-entropy was proved in [DRS04]:

Lemma 3.2. If Y has 2^r possible values and Z is any random variable, then $\tilde{H}_\infty(X|(Y, Z)) \geq H_\infty(X|Z) - r$.

The following lemma is also proved in [DRS04].

Lemma 3.3. Let X, Y be random variables such that $X \in \{0, 1\}^n$ and $\tilde{H}_\infty(X|Y) \geq k$. Let $\mathcal{H}$ be a family of pairwise independent hash functions from $\{0, 1\}^n$ to $\{0, 1\}^\ell$. Then for $h \leftarrow \mathcal{H}$, we have

$$\Delta((Y, h, h(X)), (Y, h, U_\ell)) \leq \epsilon$$

as long as $\ell < k - 2\lg(1/\epsilon)$.

3.2 Length-Parameterized Lossy Trapdoor Functions

The notion of Lossy trapdoor function was introduced by Peikert and Waters [PW08] which is a public function f that behaves in one of two modes. The first mode gives a (injective) trapdoor function: given a suitable trapdoor for f , the input x can be efficiently recovered from $f(x)$. In the second mode, f statistically loses a significant amount of information about its input, i.e., f 's image is significantly smaller than its domain. Finally, the two behaviors are indistinguishable: given just the description of f , no efficient adversary can tell whether f is injective or lossy.

Length-parameterized lossy (trapdoor) functions, introduced in [WZ24], are similar to lossy (trapdoor) functions [PW08] with a stronger requirement that the setup algorithm takes as input an explicit input length parameter ℓ_{in} . As in [PW08], there is both an injective and a lossy mode of setup and these should be computationally indistinguishable. However, the image size of the function in the lossy mode should be a polynomial that depends only on the security parameter and is independent of ℓ_{in} . The trapdoor allows to invert arbitrary outputs (when instantiated in the injective mode).

More precisely, a length-parameterized lossy trapdoor function is a tuple of efficient algorithms $\text{LTDF} = (\text{LTDF.Setup}, \text{LTDF.AltSetup}, \text{LTDF.Eval}, \text{LTDF.Invert})$ with the following syntax.

- $(ev, td) \leftarrow \text{LTDF.Setup}(1^\lambda, 1^{\ell_{\text{in}}})$: On input the security parameter λ and an input length ℓ_{in} (both in unary), the setup procedure outputs an evaluation key ev and a trapdoor key td . We assume that ev contains an implicit description of ℓ_{in} .
- $ev \leftarrow \text{LTDF.AltSetup}(1^\lambda, 1^{\ell_{\text{in}}})$: On input the security parameter λ , and an input length ℓ_{in} (both in unary), the alternate-setup procedure outputs an evaluation key ev . We assume that the key ev contains an implicit description of ℓ_{in} .
- $y \leftarrow \text{LTDF.Eval}(ev, x)$: On input a key ev and an input $x \in \{0, 1\}^{\ell_{\text{in}}}$, the evaluation procedure outputs a value $y \in \{0, 1\}^{\ell_{\text{out}}}$, for some output length ℓ_{out} that is determined during function setup.
- $y \leftarrow \text{LTDF.Invert}(td, y)$: On input a trapdoor key td and an input $y \in \{0, 1\}^{\ell_{\text{out}}}$, the inversion procedure outputs a value $x \in \{0, 1\}^{\ell_{\text{in}}}$.

Below, we formalize the properties that an LTDF as above should satisfy.

- **Efficiency:** For all security parameter $\lambda \in \mathbb{N}$,
 - $\ell_{\text{in}} = \text{poly}(\lambda)$ and $\ell_{\text{out}} = \text{poly}(\lambda)$.
 - LTDF.Setup , LTDF.AltSetup , and LTDF.Eval are probabilistic algorithms that run in time $\text{poly}(\lambda, \ell_{\text{in}})$. LTDF.Invert is a probabilistic algorithm that runs in time $\text{poly}(\lambda, \ell_{\text{out}})$.
- **Correctness of trapdoor inversion:** For all $\lambda \in \mathbb{N}$, $\ell_{\text{in}} = \text{poly}(\lambda)$, every input $x \in \{0, 1\}^{\ell_{\text{in}}}$,

$$\Pr[\text{LTDF.Invert}(td, \text{LTDF.Eval}(ev, x)) = x] = 1,$$

where the probability is over the choice of $(ev, td) \leftarrow \text{LTDF.Setup}(1^\lambda, 1^{\ell_{\text{in}}})$.

Note that this property implies injectivity.

- **Lossiness in alternate mode:** There exists a polynomial $p(\cdot)$ such that for all $\lambda, \ell_{\text{in}} \in \text{poly}(\lambda)$ the following holds. Let $ev \leftarrow \text{LTDF.AltSetup}(1^\lambda, 1^{\ell_{\text{in}}})$ and

$$S_{ev} = \{y \in \{0, 1\}^{\ell_{\text{out}}} \mid \exists x: \text{LTDF.Eval}(ev, x) = y\}$$

It must be that

$$|S_{ev}| \leq 2^{p(\lambda)}, p(\lambda) \leq \ell_{\text{in}}.$$

In other words, the image size is bounded by a polynomial in λ which is independent of ℓ_{in} .

- **Mode indistinguishability:** For all $\lambda, \ell_{\text{in}} = \text{poly}(\lambda)$ and PPT adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that,

$$\left| \Pr[\mathcal{A}(ev) = 1 \mid (ev, td) \leftarrow \text{LTDF.Setup}(1^\lambda, 1^{\ell_{\text{in}}})] - \Pr[\mathcal{A}(ev) = 1 \mid ev \leftarrow \text{LTDF.AltSetup}(1^\lambda, 1^{\ell_{\text{in}}})] \right| \leq \text{negl}(\lambda)$$

Theorem 3.4 (Theorem 3.11 in [WZ24]). *Let λ be the security parameter. Assuming (m, q, σ) -LWE is hard for $\sigma = \omega(\lambda)$, and for all polynomials m, q , there exists an LTDF = (LTDF.Setup, LTDF.AltSetup, LTDF.Eval, LTDF.Invert).*

Theorem 3.5 (Theorem 5.2 in [PW11]). *Assuming hardness of DDH, there exists an LTDF = (LTDF.Setup, LTDF.AltSetup, LTDF.Eval, LTDF.Invert).*

3.3 Zero-Knowledge Probabilistic Checkable Proof (ZK-PCP)

Introduced by [KPT97], a ZK-PCP is like a PCP [AS98] with the additional property that the view of the verifier can be efficiently simulated without access to the witness up to a small statistical distance [IMS12].

Definition 3.6 ([KPT97, IMS12]; definition adapted from [LQR⁺19]). *Let $\mathcal{R} : \{0, 1\}^n \times \{0, 1\}^h \rightarrow \{0, 1\}$ be an NP relation and $\mathcal{L} \in \{0, 1\}^n$ be the associated language. A non-adaptive, ℓ -query zero-knowledge PCP with alphabet Σ for $\mathcal{L}$ is a tuple of algorithms:*

- $\pi \leftarrow \text{zkPCP.Prove}(x, w)$: On input a statement $x \in \{0, 1\}^n$ and a witness $w \in \{0, 1\}^h$, the prove algorithm outputs a proof $\pi \in \Sigma^m$.
- $(\text{st}_x, q_1, \dots, q_\ell) \leftarrow \text{zkPCP.Query}(x)$: On input the statement x , it outputs a verification state st_x and ℓ query indices $q_1, \dots, q_\ell \in [m]$.
- $\{0, 1\} \leftarrow \text{zkPCP.Ver}(\text{st}_x, s_1, \dots, s_\ell)$: On input the verification state st and a set of responses $s_1, \dots, s_\ell \in \Sigma$, output a bit.

A zkPCP satisfies the following properties:

- **Efficiency:** The running time of zkPCP.Prove , zkPCP.Query and zkPCP.Ver should be bounded by $\text{poly}(n)$. In particular $m \in \text{poly}(n)$.
- **Completeness:** For all $x \in \{0, 1\}^n$ and $w \in \{0, 1\}^h$, if $\mathcal{R}(x, w) = 1$, then,

$$\Pr[\text{zkPCP.Ver}(\text{st}_x, \pi_{q_1}, \dots, \pi_{q_\ell}) = 1] = 1,$$

where $\pi \leftarrow \text{zkPCP.Prove}(x, w)$ and $(\text{st}_x, q_1, \dots, q_\ell) \leftarrow \text{zkPCP.Query}(x)$.

- **Soundness:** There exists a negligible function $\text{negl}(\cdot)$ such that for all $n, m \in \mathbb{N}$, $x \notin \mathcal{L}$ and $|x| = n$, all proof strings $\pi \in \Sigma^m$,

$$\Pr[\text{zkPCP.Ver}(\text{st}_x, \pi_{q_1}, \dots, \pi_{q_\ell}) = 1] = \text{negl}(n),$$

where $(\text{st}_x, q_1, \dots, q_\ell) \leftarrow \text{zkPCP.Query}(x)$

- **Semi-malicious Computational Zero-Knowledge:** For all PPT adversaries $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, there exists an efficient simulator Sim and a negligible function $\text{negl}(\cdot)$ such that,

$$\left| \Pr[\mathcal{A}_2(\text{st}_{\mathcal{A}}, \pi_{q_1}, \dots, \pi_{q_\ell}) = 1 \mid \mathcal{R}(x, w) = 1] - \right.$$

$$\left. \Pr[\mathcal{A}_2(\text{st}_{\mathcal{A}}, \tilde{\pi}_{q_1}, \dots, \tilde{\pi}_{q_\ell}) = 1 \mid \mathcal{R}(x, w) = 1] \right| \leq \text{negl}(\lambda),$$

where $(\text{st}_{\mathcal{A}}, x, w, r) \leftarrow \mathcal{A}_1(1^n)$, $(q_1, \dots, q_\ell) \leftarrow \text{zkPCP.Query}(x; r)$, $\pi \leftarrow \text{zkPCP.Prove}(x, w)$, $(\tilde{\pi}_{q_1}, \dots, \tilde{\pi}_{q_\ell}) \leftarrow \text{Sim}(x, q_1, \dots, q_\ell)$.

- **Semi-malicious Statistical Zero-Knowledge:** For all adversaries $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, there exists an efficient simulator Sim and a negligible function $\text{negl}(\cdot)$ such that,

$$\left| \Pr[\mathcal{A}_2(\text{st}_{\mathcal{A}}, \pi_{q_1}, \dots, \pi_{q_\ell}) = 1 \mid \mathcal{R}(x, w) = 1] - \right.$$

$$\left. \Pr[\mathcal{A}_2(\text{st}_{\mathcal{A}}, \tilde{\pi}_{q_1}, \dots, \tilde{\pi}_{q_\ell}) = 1 \mid \mathcal{R}(x, w) = 1] \right| \leq \text{negl}(\lambda),$$

where $(\text{st}_{\mathcal{A}}, x, w, r) \leftarrow \mathcal{A}_1(1^n)$, $(q_1, \dots, q_\ell) \leftarrow \text{zkPCP.Query}(x; r)$, $\pi \leftarrow \text{zkPCP.Prove}(x, w)$, $(\tilde{\pi}_{q_1}, \dots, \tilde{\pi}_{q_\ell}) \leftarrow \text{Sim}(x, q_1, \dots, q_\ell)$.

Remark 3.7 (Malicious ZK). A stronger notion of malicious zero-knowledge where $(q_1, \dots, q_\ell)$ are maliciously generated can also be defined. We refer readers to [LQR⁺19] for the formal definition. For our application to DV-NIZK, the weaker semi-malicious zero-knowledge suffices.

Instantiating ZK-PCP with statistical zero knowledge As noted in [IMS12], the non-interactive zero knowledge construction of [GMW87] relies on a semi-malicious zero-knowledge PCP. To elaborate, in order to prove that a graph is 3-colorable, the prover assigns a color to each node. This coloring serves as the PCP string which the verifier can query. The (semi-malicious) verifier can now query a random edge (pair of nodes) from this string, and checks if the corresponding colors are distinct. This construction involves no computational assumptions, and indeed satisfies statistical zero-knowledge against semi-malicious verifiers. While the base construction only achieves $1 - 1/\text{poly}(n)$ soundness, this can be amplified by parallel repetition, i.e., generating multiple independent copies of the PCP. Note that semi-malicious zero-knowledge is preserved by parallel repetition.

Theorem 3.8 (Semi-malicious zkPCP [GMW87]). *There exists an ℓ -query PCP with statistical soundness and semi-malicious statistical zero-knowledge for all NP, alphabet $\Sigma = \{0, 1, 2\}$, and $\ell = \text{poly}(n)$.*

Additional instantiations of zkPCP exist; see, for example, [Blu86, IKOS07].

3.4 Weak Function-Hiding Single-Key Attribute-Based Encryption

We define an Attribute-Based encryption (ABE) scheme [SW05, GPSW06] with a certain function-hiding property. In this work, it suffices to construct an ABE scheme that satisfies semantic security with a single function key. Such ABE schemes (without the function hiding property) have been well studied and are known to exist assuming Public Key Encryption [SS10, GVW12].

Definition 3.9 (Adapted from [LQR⁺19]). *An Attribute-Based Encryption (ABE) scheme over a message space $\mathcal{M}$, attribute space $\mathcal{K}$, and function family $\mathcal{F} = \{f : \mathcal{X} \rightarrow \{0, 1\}\}$ is a tuple $\text{ABE} = (\text{ABE.Setup}, \text{ABE.Keygen}, \text{ABE.Enc}, \text{ABE.Dec})$ such that:*

- $(\text{pp}, \text{msk}) \leftarrow \text{ABE.Setup}(1^\lambda)$: *The setup algorithm on input the security parameter λ outputs the public parameters pp and master secret key msk .*
- $\text{sk} \leftarrow \text{ABE.Keygen}(\text{pp}, \text{msk}, f)$: *The generation algorithm on input the public parameters, master secret key, function $f \in \mathcal{F}$, outputs a function secret key sk .*
- $\text{ct} \leftarrow \text{ABE.Enc}(\text{pp}, x, m)$: *The encryption algorithm takes as input the public parameters, attribute x and message $m \in \mathcal{M}$, and outputs a ciphertext ct .*
- $(x, m) \leftarrow \text{ABE.Dec}(\text{pp}, \text{sk}, \text{ct})$: *The decryption algorithm takes as input the public parameters, secret key sk (which maybe the master secret key) and ciphertext ct , and outputs an attribute $x \in \mathcal{X}$ and message $m \in \mathcal{M} \cup \perp$.*

A Weak Function-Hiding ABE scheme with the above algorithms satisfies the following properties:

- **(Perfect) Correctness:** For all messages $m \in \mathcal{M}$, attributes $x \in \mathcal{X}$, predicates $f \in \mathcal{F}$,
 - $\Pr[\text{ABE.Dec}(\text{pp}, \text{msk}, \text{ABE.Enc}(\text{pp}, x, m)) = m \mid (\text{pp}, \text{msk}) \leftarrow \text{ABE.Setup}(1^\lambda)] = 1.$

- If $f(x) = 1$, then

$$\Pr [\text{ABE.Dec}(\text{pp}, \text{ABE.Keygen}(\text{pp}, \text{msk}, f), \text{ABE.Enc}(\text{pp}, x, m)) = m \mid (\text{pp}, \text{msk}) \leftarrow \text{ABE.Setup}(1^\lambda)] = 1.$$

- **Single-Key Semantic Security:** For all $\lambda \in \mathbb{N}$, and all *admissible* PPT adversaries $(\mathcal{A}_1, \mathcal{A}_2)$, there exists a negligible function $\text{negl}(\cdot)$ such that

$$\Pr \left[b' \leftarrow \mathcal{A}_2^{\mathcal{O}(\text{pp}, b, \cdot, \cdot)}(\text{pp}, \text{sk}, \tau) \wedge b' = b \mid (f, \tau) \leftarrow \mathcal{A}_1(1^\lambda), (\text{pp}, \text{msk}) \leftarrow \text{ABE.Setup}(1^\lambda), \right. \\ \left. \text{sk} \leftarrow \text{ABE.Keygen}(\text{pp}, \text{msk}, f), b \xleftarrow{\$} \{0, 1\} \right] = \text{negl}(\lambda),$$

where

- $\mathcal{O}(\text{pp}, b, x, m_0, m_1)$ is an oracle which upon queried an attribute x , messages $m_0, m_1 \in \mathcal{M}$ returns a ciphertext $\text{ct} \leftarrow \text{ABE.Enc}(\text{pp}, x, m_b)$.
- $\mathcal{A}$ is *admissible* if it makes one ciphertext query (x, m_0, m_1) to $\mathcal{O}$, and $f(x) = 0$.
- **Weak Function-Hiding:**⁸ For all $\lambda \in \mathbb{N}$, and $p = \text{poly}(\lambda)$, there exists a PPT simulator Sim such that for all functions $f \in \mathcal{F}$, and for all PPT adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that

$$\left| \Pr \left[\mathcal{A}^{\mathcal{O}_1(\text{pp}, \text{sk}, \cdot)}(\text{pp}) \mid (\text{pp}, \text{msk}) \leftarrow \text{ABE.Setup}(1^\lambda), \text{sk} \leftarrow \text{ABE.Keygen}(\text{pp}, \text{msk}, f) \right] - \right. \\ \left. \Pr \left[\mathcal{A}^{\mathcal{O}_2(\text{pp}, \text{msk}, \cdot)}(\text{pp}) \mid (\text{pp}, \text{msk}) \leftarrow \text{ABE.Setup}(1^\lambda) \right] \right| \leq \text{negl}(\lambda).$$

Here,

- $\mathcal{O}_1(\text{pp}, \text{sk}, \text{ct})$ is the *real* decryption oracle which upon queried a ciphertext $\text{ct} \in \{0, 1\}^p$, outputs $\text{ABE.Dec}(\text{pp}, \text{sk}, \text{ct})$.
- $\mathcal{O}_2(\text{pp}, \text{msk}, \text{ct})$ is the *ideal* decryption oracle which upon queried a ciphertext $\text{ct} \in \{0, 1\}^p$, outputs $\text{Sim}^{f(\cdot)}(\text{pp}, \text{msk}, \text{ct})$. Moreover, Sim is restricted to query $f(\cdot)$ at most once per invocation.

3.5 Attribute-Based Secure Function Evaluation (ABSFE)

An ABSFE is a generalization of single-key ABE satisfying the following two notions of security: (1) Weak message hiding (analogous to single-key semantic security of ABE), and (2) Strong key hiding (analogous to weak function-hiding property of ABE).

Definition 3.10 (Adapted from [LQR⁺19]). *An Attribute-Based Secure Function Evaluation (ABSFE) scheme for a function $F : \mathcal{X} \times \mathcal{Y} \rightarrow \{0, 1\}$ with message space $\mathcal{M}$ is a tuple $\text{ABSFE} = (\text{ABSFE.Setup}, \text{ABSFE.Keygen}, \text{ABSFE.Enc}, \text{ABSFE.Dec})$ such that:*

⁸This is weaker than the standard notion of function-hiding ABE. Informally, a function-hiding ABE requires that the function secret key hides the function. On the other hand, here, we want that oracle access to the decryption function does not leak information about the function

- $\text{pp} \leftarrow \text{ABSFE.Setup}(1^\lambda)$: The setup algorithm on input the security parameter λ outputs the public parameters pp .
- $(\text{pk}, \text{sk}) \leftarrow \text{ABSFE.Keygen}(\text{pp}, y)$: The generation algorithm on input the public parameters, and a value $y \in \mathcal{Y}$, outputs a public key pk and secret key sk .
- $\text{ct} \leftarrow \text{ABSFE.Enc}(\text{pp}, \text{pk}, x, m)$: The encryption algorithm takes as input the public parameters, public key, a value $x \in \mathcal{X}$, and message $m \in \mathcal{M}$, and outputs a ciphertext ct .
- $m \leftarrow \text{ABSFE.Dec}(\text{pp}, \text{sk}, x, \text{ct})$: The decryption algorithm takes as input the public parameters, secret key sk , an attribute x and ciphertext ct , and outputs a message $m \in \mathcal{M} \cup \perp$.

An ABSFE scheme with the above algorithms satisfies the following properties:

- **(Perfect) Correctness:** For all messages $m \in \mathcal{M}$, $x \in \mathcal{X}$, $y \in \mathcal{Y}$, where $F(x, y) = 1$,

$$\Pr \left[\text{ABSFE.Dec}(\text{pp}, \text{sk}, x, \text{ABSFE.Enc}(\text{pp}, \text{pk}, x, m)) = m \mid \text{pp} \leftarrow \text{ABSFE.Setup}(1^\lambda), \right. \\ \left. (\text{pk}, \text{sk}) \leftarrow \text{ABSFE.Keygen}(\text{pp}, y) \right] = 1$$

- **Weak Computational Message Hiding:** This is essentially single-key semantic security for ABE: namely, a ciphertext with attribute $x \in \mathcal{X}$ encrypted under a public key for $y \in \mathcal{Y}$ where $F(x, y) = 0$ should hide the underlying message.

Definition 3.11. Let ABSFE be an ABSFE scheme for F . Then, for all $\lambda \in \mathbb{N}$, and all admissible PPT adversaries $(\mathcal{A}_1, \mathcal{A}_2)$, there exists a negligible function $\text{negl}(\cdot)$ such that

$$\Pr \left[b' \leftarrow \mathcal{A}_2^{\mathcal{O}(\text{pp}, \text{pk}, b, \cdot, \cdot, \cdot)}(\text{pp}, \text{pk}, \text{sk}, \tau) \wedge b' = b \mid (y, \tau) \leftarrow \mathcal{A}_1(1^\lambda), \text{pp} \leftarrow \text{ABSFE.Setup}(1^\lambda), \right. \\ \left. (\text{pk}, \text{sk}) \leftarrow \text{ABSFE.Keygen}(\text{pp}, y), b \xleftarrow{\$} \{0, 1\} \right] = \text{negl}(\lambda),$$

where

- $\mathcal{O}(\text{pp}, \text{pk}, b, x, m_0, m_1)$ is an oracle which upon queried an attribute $x \in \mathcal{X}$, messages $m_0, m_1 \in \mathcal{M}$ returns a ciphertext $\text{ct} \leftarrow \text{ABSFE.Enc}(\text{pp}, \text{pk}, x, m_b)$.
- $\mathcal{A}$ is admissible if it makes one ciphertext query (x, m_0, m_1) to $\mathcal{O}$, and $F(x, y) = 0$.

- **Weak Statistical Message Hiding:** We introduce the statistical version of weak message hiding which is essentially single-key *statistical* semantic security for ABE: namely, a ciphertext with attribute $x \in \mathcal{X}$ encrypted under a public key for $y \in \mathcal{Y}$ where $F(x, y) = 0$ should statistically hide the underlying message.

Definition 3.12. Let ABSFE be an ABSFE scheme for F . Then, for all $\lambda \in \mathbb{N}$, and all admissible adversaries $(\mathcal{A}_1, \mathcal{A}_2)$, there exists a negligible function $\text{negl}(\cdot)$ such that

$$\Pr \left[b' \leftarrow \mathcal{A}_2^{\mathcal{O}(\text{pp}, \text{pk}, b, \cdot, \cdot, \cdot)}(\text{pp}, \text{pk}, \text{sk}, \tau) \wedge b' = b \mid (y, \tau) \leftarrow \mathcal{A}_1(1^\lambda), \text{pp} \leftarrow \text{ABSFE.Setup}(1^\lambda), \right. \\ \left. (\text{pk}, \text{sk}) \leftarrow \text{ABSFE.Keygen}(\text{pp}, y), b \xleftarrow{\$} \{0, 1\} \right] = \text{negl}(\lambda),$$

where

- $\mathcal{O}(\text{pp}, \text{pk}, b, x, m_0, m_1)$ is an oracle which upon queried an attribute $x \in \mathcal{X}$, messages $m_0, m_1 \in \mathcal{M}$ returns a ciphertext $\text{ct} \leftarrow \text{ABSFE.Enc}(\text{pp}, \text{pk}, x, m_b)$.
- $\mathcal{A}$ is admissible if it makes one ciphertext query (x, m_0, m_1) to $\mathcal{O}$, and $F(x, y) = 0$.
- **Strong Key Hiding:** The strong key hiding notion requires that y remains hidden even if the adversary has access to a decryption oracle (with the associated secret key sk). Strong key-hiding is reminiscent of the weak function-hiding property we defined for ABE. ABE schemes which satisfy weak function-hiding imply ABSFE schemes that satisfy strong key-hiding [LQR⁺19].

Definition 3.13. For all $\lambda \in \mathbb{N}$, and $p = \text{poly}(\lambda)$, there exists a PPT simulator $\text{Sim} := (\text{Sim}_1, \text{Sim}_2)$ such that for all keys $y \in \mathcal{Y}$, and for all PPT adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that

$$\left| \Pr \left[\mathcal{A}^{\mathcal{O}_1(\text{pp}, \text{sk}, \cdot, \cdot)}(\text{pp}, \text{pk}) = 1 \mid \text{pp} \leftarrow \text{ABSFE.Setup}(1^\lambda), (\text{pk}, \text{sk}) \leftarrow \text{ABSFE.Keygen}(\text{pp}, y) \right] - \Pr \left[\mathcal{A}^{\mathcal{O}_2(\text{st}, \cdot, \cdot)}(\text{pp}, \text{pk}) = 1 \mid (\text{st}, \text{pp}, \text{pk}) \leftarrow \text{Sim}_1(1^\lambda) \right] \right| \leq \text{negl}(\lambda).$$

Here,

- $\mathcal{O}_1(\text{pp}, \text{sk}, x, \text{ct})$ is the real decryption oracle which upon queried an attribute $x \in \mathcal{X}$ and a ciphertext $\text{ct} \in \{0, 1\}^p$, outputs $\text{ABSFE.Dec}(\text{pp}, \text{sk}, x, \text{ct})$.
- $\mathcal{O}_2(\text{st}, x, \text{ct})$ is the ideal decryption oracle which upon queried an attribute x and a ciphertext $\text{ct} \in \{0, 1\}^p$, outputs $\text{Sim}_2^{F(\cdot, y)}(\text{st}, x, \text{ct})$. Moreover, Sim_2 is restricted to query $F(\cdot, y)$ at most once per invocation.

Remark 3.14. By a standard hybrid argument, any ABSFE scheme that satisfies weak message hiding against an adversary that makes a single challenge query (x, m_0, m_1) is also secure against an adversary that makes polynomially many challenge queries.

Remark 3.15. One can also define a “strong” notion of message-hiding which says semantic security holds even in the setting where the public-key is maliciously chosen. Informally, in this case, we require that there exists an efficient algorithm that can extract an attribute y from any (possibly malformed) public key pk , and ciphertexts encrypted to any attribute x where $F(x, y) = 0$ still hide the underlying message. The weaker notion suffices for our application, thus we refer readers to [LQR⁺19] for a detailed definition.

3.6 Reusable Designated Verifier Non Interactive Zero Knowledge

Let L be an NP language associated with an NP relation $\mathcal{R}$. A reusable designated-verifier non-interactive zero-knowledge (DV-NIZK) argument for L consists of a tuple of three efficient algorithms $\text{DV-NIZK} = (\text{DV-NIZK.Setup}, \text{DV-NIZK.Keygen}, \text{DV-NIZK.Prove}, \text{DV-NIZK.Ver})$ with the following properties:

- $\text{crs} \leftarrow \text{DV-NIZK.Setup}(1^\lambda)$: On input the security parameter λ , the setup algorithm outputs a common reference string crs .

- $(\text{pk}, \text{sk}) \leftarrow \text{DV-NIZK.Keygen}(\text{crs})$: On input a common reference string crs , the key-generation algorithm outputs a public key pk and a secret verification key sk which is sent only to the verifier.
- $\pi \leftarrow \text{DV-NIZK.Prove}(\text{crs}, \text{pk}, x, w)$: On input the common reference string crs , a public key pk , a statement x , and a witness w , the prove algorithm outputs a proof π .
- $\{0, 1\} \leftarrow \text{DV-NIZK.Ver}(\text{crs}, \text{sk}, x, \pi)$: On input the common reference string crs , a secret key sk , a statement x , and a proof π , the verification algorithm outputs a bit $b \in \{0, 1\}$.

Moreover, DV-NIZK should satisfy the following properties:

- **Completeness:** For all $(x, w) \in \mathcal{R}$, we have that

$$\Pr \left[\text{DV-NIZK.Ver}(\text{crs}, \text{sk}, x, \pi) = 1 \mid \pi \leftarrow \text{DV-NIZK.Prove}(\text{crs}, \text{pk}, x, w), \right. \\ \left. \text{crs} \leftarrow \text{DV-NIZK.Setup}(1^\lambda), (\text{pk}, \text{sk}) \leftarrow \text{DV-NIZK.Keygen}(\text{crs}) \right] = 1.$$

- **Soundness:** We consider two variants of soundness:

- *Non-adaptive soundness:* For all $x \notin L$, all PPT adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that

$$\Pr \left[\pi \leftarrow \mathcal{A}^{\text{DV-NIZK.Ver}(\text{crs}, \text{sk}, \cdot)}(1^\lambda, \text{crs}, \text{pk}, x) : \text{DV-NIZK.Ver}(\text{crs}, \text{sk}, x, \pi) = 1 \right] = \text{negl}(\lambda),$$

where $\text{crs} \leftarrow \text{DV-NIZK.Setup}(1^\lambda)$ and $(\text{pk}, \text{sk}) \leftarrow \text{DV-NIZK.Keygen}(\text{crs})$.

- *Adaptive soundness:* For all PPT adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that

$$\Pr \left[(x, \pi) \leftarrow \mathcal{A}^{\text{DV-NIZK.Ver}(\text{crs}, \text{sk}, \cdot)}(1^\lambda, \text{crs}) : x \notin L \wedge \text{DV-NIZK.Ver}(\text{crs}, \text{sk}, x, \pi) = 1 \right] = \text{negl}(\lambda),$$

where $(\text{crs}, \text{pk}, \text{sk}) \leftarrow \text{DV-NIZK.Setup}(1^\lambda)$ and $(\text{pk}, \text{sk}) \leftarrow \text{DV-NIZK.Keygen}(\text{crs})$.

- **Computational Zero-knowledge:** For all PPT adversaries $\mathcal{A}$, there exists a PPT simulator $S = (S_1, S_2)$ and negligible function $\text{negl}(\cdot)$ such that

$$\left| \Pr \left[\mathcal{A}^{\mathcal{O}_0(\text{crs}, \text{pk}, \cdot)}(\text{crs}, \text{pk}, \text{sk}) = 1 \right] - \Pr \left[\mathcal{A}^{\mathcal{O}_1(\text{st}_S, \text{sk}, \cdot)}(\text{crs}, \text{pk}, \text{sk}) = 1 \right] \right| = \text{negl}(\lambda),$$

where $\text{crs} \leftarrow \text{DV-NIZK.Setup}(1^\lambda)$, $(\text{pk}, \text{sk}) \leftarrow \text{DV-NIZK.Keygen}(\text{crs})$, and $(\text{st}_S, \text{crs}, \text{pk}, \text{sk}) \leftarrow S_1(1^\lambda)$, the oracle $\mathcal{O}_0(\text{crs}, \text{pk}, w)$ outputs $\text{DV-NIZK.Prove}(\text{crs}, \text{pk}, x, w)$ if $(x, w) \in \mathcal{R}$ and $\perp$ otherwise, and the oracle $\mathcal{O}_1(\text{st}_S, x)$ outputs $S_2(\text{st}_S, x)$ if $(x, w) \in \mathcal{R}$ and $\perp$ otherwise.

- **Statistical Zero-knowledge:** For all adversaries $\mathcal{A}$, there exists a PPT simulator $S = (S_1, S_2)$ and negligible function $\text{negl}(\cdot)$ such that

$$\left| \Pr \left[\mathcal{A}^{\mathcal{O}_0(\text{crs}, \text{pk}, \cdot)}(\text{crs}, \text{pk}, \text{sk}) = 1 \right] - \Pr \left[\mathcal{A}^{\mathcal{O}_1(\text{st}_S, \text{sk}, \cdot)}(\text{crs}, \text{pk}, \text{sk}) = 1 \right] \right| = \text{negl}(\lambda),$$

where $\text{crs} \leftarrow \text{DV-NIZK.Setup}(1^\lambda)$, $(\text{pk}, \text{sk}) \leftarrow \text{DV-NIZK.Keygen}(\text{crs})$, and $(\text{st}_S, \text{crs}, \text{pk}, \text{sk}) \leftarrow S_1(1^\lambda)$, the oracle $\mathcal{O}_0(\text{crs}, \text{pk}, w)$ outputs $\text{DV-NIZK.Prove}(\text{crs}, \text{pk}, x, w)$ if $(x, w) \in \mathcal{R}$ and $\perp$ otherwise, and the oracle $\mathcal{O}_1(\text{st}_S, x)$ outputs $S_2(\text{st}_S, x)$ if $(x, w) \in \mathcal{R}$ and $\perp$ otherwise.

4 Secret Sharing with Randomness Recovery

A key ingredient in our ABSFE construction is a secret sharing scheme for non-monotone functions with a randomness recovery property.

A secret sharing scheme for non-monotone functions is a generalization of the standard secret sharing which applies to arbitrary (not necessarily monotone functions). In such a scheme, the shares consist of 2ℓ values $(s_1, t_1, s_2, t_2, \dots, s_\ell, t_\ell)$ and for any input $(x_1, \dots, x_\ell) \in \{0, 1\}^\ell$, the collection of shares $\{s_i: x_i = 1\} \cup \{t_i: x_i = 0\}$ determines the secret if $F(x_1, \dots, x_\ell) = 1$ and does not reveal anything otherwise. The randomness recovery property requires a procedure that recovers all the 2ℓ shares from ℓ shares in the case where F is satisfied.

Analogously to standard secret sharing, we refer to x as an authorized set of users if $F(x) = 1$ and unauthorized otherwise. Further, when x is authorized, the corresponding shares, i.e., $\{s_i: x_i = 1\} \cup \{t_i: x_i = 0\}$ are referred to as the *authorized shares*.

Remark 4.1. A secret sharing scheme for non-monotone functions captures the case where parties are partitioned into pairs, and we care about coalitions where exactly one party from each pair is present. This notion was studied in the past in several works, e.g., [BI05, VV15, ABI⁺23]. The additional randomness recovery property that we require is new to our work.

Definition 4.2 (Secret sharing with randomness recovery (RR-SS)). *A secret sharing scheme with randomness recovery for a function family $\mathcal{F} = \{F: \{0, 1\}^\ell \rightarrow \{0, 1\}\}_{\ell \in \mathbb{N}}$ with secret space $\mathcal{P}$ and share space $\mathcal{Q}$ is defined as a tuple $\text{SS-RR} = (\text{SS-RR.GenShares}, \text{SS-RR.Reconstruct}, \text{SS-RR.Regenerate})$ such that*

- $(a_{1,0}, a_{1,1}, \dots, a_{\ell,0}, a_{\ell,1}) \leftarrow \text{SS-RR.GenShares}(F, s)$: A randomized share generation algorithm that takes as input the function $F \in \mathcal{F}$, the secret $s \in \mathcal{P}$, and it outputs shares $(a_{1,0}, a_{1,1}, \dots, a_{\ell,0}, a_{\ell,1}) \in \mathcal{Q}^{2\ell}$.
- $s \leftarrow \text{SS-RR.Reconstruct}(F, (a_{1,0}, a_{1,1}, \dots, a_{\ell,0}, a_{\ell,1}))$: A deterministic algorithm that takes as input the function $F \in \mathcal{F}$, 2ℓ shares $(a_{1,0}, a_{1,1}, \dots, a_{\ell,0}, a_{\ell,1})$, and outputs a secret $s \in \mathcal{P}$.
- $\beta \leftarrow \text{SS-RR.isValid}(F, (a_{1,0}, a_{1,1}, \dots, a_{\ell,0}, a_{\ell,1}))$: A deterministic algorithm that takes as input the function $F \in \mathcal{F}$, 2ℓ shares $(a_{1,0}, a_{1,1}, \dots, a_{\ell,0}, a_{\ell,1})$, and outputs $\beta = \{1, \perp\}$.
- $(b_{1,0}, b_{1,1}, \dots, b_{\ell,0}, b_{\ell,1}) \leftarrow \text{SS-RR.Regenerate}(F, x, (a_{1,x_1}, \dots, a_{\ell,x_\ell}))$: A deterministic algorithm that on input a function $F \in \mathcal{F}$, input x , and shares $(a_{1,x_1}, \dots, a_{\ell,x_\ell})$, and outputs shares $(b_{1,0}, b_{1,1}, \dots, b_{\ell,0}, b_{\ell,1}) \in \mathcal{Q}^{2\ell} \cup \{\perp\}$.

The share size of a scheme as above is defined to be $\lceil \log |\mathcal{Q}| \rceil$.

Definition 4.3 (Efficiency). *For any function $F \in \mathcal{F}$ let $|F|$ denote the length of the description of F , then,*

- SS-RR.GenShares runs in time $\text{poly}(|F|, \lceil \log |\mathcal{P}| \rceil)$.
- SS-RR.Reconstruct runs in time $\text{poly}(|F|, \ell, \lceil \log |\mathcal{Q}| \rceil)$.
- SS-RR.isValid runs in time $\text{poly}(|F|, \ell, \lceil \log |\mathcal{Q}| \rceil)$.
- SS-RR.Regenerate runs in time $\text{poly}(|F|, \ell, \lceil \log |\mathcal{Q}| \rceil)$.

The correctness of a secret sharing scheme for a family class $\mathcal{F}$ as above is captured by the following property.

Definition 4.4 (Correctness). For any function $F \in \mathcal{F}$, and any secret $s \in \mathcal{P}$,

- *Reconstruction correctness:*

$$\Pr [\text{SS-RR.Reconstruct}(F, (a_{1,0}, a_{1,1}, \dots, a_{\ell,0}, a_{\ell,1})) = s] = 1,$$

where the probability is over the randomness used for sampling $(a_{1,0}, a_{1,1}, \dots, a_{\ell,0}, a_{\ell,1}) \leftarrow \text{SS-RR.GenShares}(F, s)$.

- *Regeneration correctness:* for all $x \in \{0, 1\}^\ell$ such that $F(x) = 1$,

$$\Pr [\text{SS-RR.Regenerate}(F, (a_{1,x_1}, \dots, a_{\ell,x_\ell})) = (a_{1,0}, a_{1,1}, \dots, a_{\ell,0}, a_{\ell,1})] = 1,$$

where the probability is over the randomness used for sampling $(a_{1,0}, a_{1,1}, \dots, a_{\ell,0}, a_{\ell,1}) \leftarrow \text{SS-RR.GenShares}(F, s)$.

In other words, if $(a_{1,0}, a_{1,1}, \dots, a_{\ell,0}, a_{\ell,1})$ was generated honestly by the share generation algorithm, then every set of authorized shares must regenerate exactly the set of shares $(a_{1,0}, a_{1,1}, \dots, a_{\ell,0}, a_{\ell,1})$.

Definition 4.5 (Perfect privacy). For any access function F and any input x such that $F(x) = 0$, for any two secret $s_1, s_2 \in \mathcal{P}$, the distributions $(a_{i,x_i})_{i \in \ell}$ and $(a'_{i,x_i})_{i \in \ell}$ are identical, where $(a_{1,0}, a_{1,1}, \dots, a_{\ell,0}, a_{\ell,1}) \leftarrow \text{SS-RR.GenShares}(F, s_1)$ and $(a'_{1,0}, a'_{1,1}, \dots, a'_{\ell,0}, a'_{\ell,1}) \leftarrow \text{SS-RR.GenShares}(F, s_2)$.

Lastly, we formalize the soundness property of the **Regenerate** procedure. In short, it says that every (potentially maliciously generated) set of shares are either *invalid*, or correspond to some legal sharing of some secret (meaning they are valid).

Definition 4.6 (Soundness). For any function $F \in \mathcal{F}$, for all set of (maliciously generated) shares $(a_{1,0}, a_{1,1}, \dots, a_{\ell,0}, a_{\ell,1}) \in \mathcal{Q}^{2\ell}$, for any $x \in \{0, 1\}^\ell$ such that $F(x) = 1$,

- *Regeneration Soundness:*

$$\exists s \in \mathcal{P}, \exists r: \text{SS-RR.Regenerate}(F, x, (a_{1,x_1}, \dots, a_{\ell,x_\ell})) = \text{SS-RR.GenShares}(F, s; r).$$

- *Validation Soundness:* if $\nexists s \in \mathcal{P}, r$ such that $\text{SS-RR.GenShares}(F, s; r) = (a_{1,0}, a_{1,1}, \dots, a_{\ell,0}, a_{\ell,1})$, then

$$\text{SS-RR.isValid}(F, (a_{1,0}, a_{1,1}, \dots, a_{\ell,0}, a_{\ell,1})) = \perp,$$

and if $\exists s \in \mathcal{P}, r$ such that $\text{SS-RR.GenShares}(F, s; r) = (a_{1,0}, a_{1,1}, \dots, a_{\ell,0}, a_{\ell,1})$, then

$$\text{SS-RR.isValid}(F, (a_{1,0}, a_{1,1}, \dots, a_{\ell,0}, a_{\ell,1})) = 1.$$

4.1 A Construction for Polynomial-Size Branching Programs

We construct such a secret sharing scheme for all functions that can be computed by polynomial-sized branching programs (PBP).

Definition 4.7 (Branching Programs of size ℓ). A size ℓ branching program is a Directed Acyclic Graph (DAG) G with the following properties:

- The graph G has $\ell + 2$ nodes.

- There exists a designated source node, call it v_1 .
- There are 2 sink nodes denoted by v_{acc} and v_{rej} with no outgoing edges. All non sink nodes labelled $\{v_1, \dots, v_\ell\}$ have exactly two outgoing edges labelled (i, b) , for $i \in [\ell]$ and $b \in \{0, 1\}$. Without loss of generality, assume that $v_{\ell+1} = v_{\text{acc}}$ and $v_{\ell+2} = v_{\text{rej}}$
- Every directed path in G must either terminate at v_{acc} or v_{rej} .

Definition 4.8 (Functions solvable by Polynomial Branching Programs). A function $F : \{0, 1\}^n \rightarrow \{0, 1\}$ is solvable by a size ℓ branching program G if there is an efficiently computable mapping $\mu : \{0, 1\}^n \rightarrow \{0, 1\}^\ell$ such that for all inputs $x \in \{0, 1\}^n$ and $y = \mu(x)$:

- the set of edges $\{(i, y_i)\}_{i \in [\ell]}$ in G forms a directed path from v_1 to v_{acc} if $F(x) = 1$, and
- the set of edges $\{(i, y_i)\}_{i \in [\ell]}$ in G forms a directed path from v_1 to v_{rej} if $F(x) = 0$.

G is said to be polynomial sized if $\ell \in \text{poly}(n)$.

Definition 4.9 (Complexity Class (PBP) [AKG05]). PBP is the class of decision problems which are solvable by a polynomial sized branching program.

We now present the construction of a secret sharing scheme for PBP. We assume every function $F \in \text{PBP}$ is represented as a polynomial sized branching program.

Intuition. To share a secret s , we assign s to the accept sink node. The source and reject sink node are assigned the value 0. Every other node is assigned uniform random value. The share corresponding to each edge (hence the parties) is the difference between the values of the nodes at both ends. Clearly, if one has a path from the source node to the accept sink node, then adding the shares along the path will yield the correct secret s . Intuitively, regeneration correctness follows from the observation that (1) we always know the share corresponding to at least one outgoing edge from every node, and (2) for every node directly connected to a sink with an edge, once we know the secret, and the share corresponding to one outgoing edge from it, then we can simply add these two values to get the random value assigned to the node. This step can be repeated iteratively to extract all the shares. In our construction we work with $\mathcal{P} = \mathcal{Q} = \{0, 1\}^p$. We proceed with the full details.

- **SS-RR.GenShares(F, s) :**
 1. Assign values to every node $\{val_i\}_{i \in [\ell+2]}$ to $\{v_i\}_{i \in [\ell+2]}$:
 - if $i = 1$ or $i = \ell + 2$, then $val_i = 0$
 - if $i = \ell + 1$, then $val_i = s$
 - else $val_i \xleftarrow{\$} \{0, 1\}^p$
 2. Define the shares $\{a_{i,b}\}_{i \in [\ell], b \in \{0,1\}}$ as follows: if edge labelled (i, b) is directed from node v_i to v_j , then set $a_{i,b} = val_j - val_i$
 3. output $\{a_{i,b}\}_{i \in [\ell], b \in \{0,1\}}$
- **SS-RR.Reconstruct($F, (a_{1,0}, a_{1,1}, \dots, a_{\ell,0}, a_{\ell,1})$) :**
 1. Perform Depth First Search to find a directed path from v_1 to v_{acc} in F . Denote the path by E .

2. If no path exists from v_1 to v_{acc} , then output $\perp$ and terminate.
 3. Output $\sum_{(j,b) \in E} a_{j,b}$.
- **SS-RR.isValid**($F, (a_{1,0}, a_{1,1} \dots, a_{\ell,0}, a_{\ell,1})$) :
 1. Add the edge (j, b) to set E (initialized as an empty set) for all $j \in [\ell], b \in \{0, 1\}$
 2. Construct a DAG with $\ell + 2$ nodes and edge set E .
 3. If no path exists from v_1 to v_{acc} , then output $\perp$ and terminate.
 4. Perform a Depth First Search to find a path E' from v_1 to v_{acc} .
 5. $val_{\ell+1} = \sum_{(j,b) \in E'} a_{j,b}$
 6. $val_{\ell+2} = val_1 = 0$
 7. Let n_{v_i} be the minimum length of a directed path from v_i to either v_{acc} or v_{rej} .
 8. Do the following iteratively for all $k = 1$ to ℓ
 - (a) For all nodes v_i such that $n_{v_i} = k$
 - i. Let v_j be any node which has the directed edge (i, b) from v_i such that $n_{v_j} = k-1$ and $a_{i,b} \in \{y_i\}_{i \in T}$
 - ii. $val_i = val_j - a_{i,b}$
 9. For all i, j , if edge labelled (i, b) is directed from node v_i to v_j , then set $a'_{i,b} = val_j - val_i$
 10. If $\exists (j, b) \in E$ such that $a_{j,b} \neq a'_{j,b}$, then output $\perp$
 11. Else output 1
 - **SS-RR.Regenerate**($F, x, (a_{1,x_1}, \dots, a_{\ell,x_\ell})$) :
 1. Add the edge (j, a_{j,x_j}) to set E (initialized as an empty set).
 2. Construct a DAG with $\ell + 2$ nodes and edge set E .
 3. If no path exists from v_1 to v_{acc} , then output $\perp$ and terminate.
 4. Perform a Depth First Search to find a path E' from v_1 to v_{acc} .
 5. $val_{\ell+1} = \sum_{(j,b) \in E'} a_{j,b}$
 6. $val_{\ell+2} = val_1 = 0$
 7. Let n_{v_i} be the minimum length of a directed path from v_i to either v_{acc} or v_{rej} .
 8. Do the following iteratively for all $k = 1$ to ℓ
 - (a) For all nodes v_i such that $n_{v_i} = k$
 - i. Let v_j be any node which has the directed edge (i, b) from v_i such that $n_{v_j} = k-1$ and $a_{i,b} \in \{y_i\}_{i \in T}$
 - ii. $val_i = val_j - a_{i,b}$
 9. For all i, j , if edge labelled (i, b) is directed from node v_i to v_j , then set $a'_{i,b} = val_j - val_i$
 10. Output $(a'_{1,0}, a'_{1,1}, \dots, a'_{\ell,0}, a'_{\ell,1})$

Efficiency:

Lemma 4.10. *For any function $F \in \mathcal{F}$, secret/share space $\{0, 1\}^p$, SS-RR.GenShares, SS-RR.Reconstruct, SS-RR.isValid and SS-RR.Regenerate all run in time $\text{poly}(|F|, \ell, p)$.*

The proof follows by observing that Depth First Search can be done in polynomial time in the size of function F , and all other operations in the algorithms run in polynomial time in the length of their respective inputs.

Reconstruction correctness:

Lemma 4.11. *For any function $F \in \text{PBP}$ and secret $s \in \mathcal{P}$,*

$$\Pr [\text{SS-RR.Reconstruct}(F, (a_{1,0}, a_{1,1}, \dots, a_{\ell,0}, a_{\ell,1})) = s] = 1,$$

where the probability is over the randomness used for sampling $(a_{1,0}, a_{1,1}, \dots, a_{\ell,0}, a_{\ell,1}) \leftarrow \text{SS-RR.GenShares}(F, s)$. Here SS-RR.Reconstruct and SS-RR.GenShares are as presented in the above construction.

Proof. By the correctness of Depth First Search, E contains v_1 and v_{acc} , and a sequence of edges $((j, b))_{j \in \ell}$ for some $b \in \{0, 1\}$ which starts from v_1 and ends at v_{acc} . Let $\{a_{j,b}\}_{(j,b) \in E}$ be the shares corresponding to these edges in the path.

Let us assume that the length of path E is t for some $t \leq \ell$. Let $u_1, \dots, u_t$ denote the nodes in E in the correct order, i.e., $u_1 = v_1, u_t = v_{\text{acc}}$ and there is an edge in E from u_i to u_{i+1} . Let val'_i be the value assigned to node u_i by the algorithm GenShares . Therefore as per construction of GenShares ,

$$\sum_{(j,b) \in E} a_{j,b} = \sum_{i \in [t]} (val'_{i+1} - val'_i) = val'_t - val'_1 = val_{\text{acc}} - val_1 = s.$$

Thus, we prove a stronger statement that for all randomness r ,

$$\text{SS-RR.Reconstruct}(F, (a_{1,0}, a_{1,1}, \dots, a_{\ell,0}, a_{\ell,1})) = s,$$

where r is the internal randomness used by SS-RR.GenShares . □

Regeneration correctness:

Lemma 4.12. *For all functions $F \in \mathcal{F}$ and $x \in \{0, 1\}^\ell$ such that $F(x) = 1$,*

$$\Pr[\text{SS-RR.Regenerate}(F, (a_{1,x_1}, \dots, a_{\ell,x_\ell})) = (a_{1,0}, a_{1,1}, \dots, a_{\ell,0}, a_{\ell,1})] = 1,$$

Here SS-RR.Regenerate and SS-RR.GenShares are as presented in the above construction.

Proof. By construction, we know that for every directed path from v_1 to v_{acc} is such that adding the shares corresponding to the edges along this path in the correct order yields the same secret s . Therefore, Step 4 in SS-RR.Regenerate in our construction always sets $val_{\ell+1} = val_{\text{acc}} = s$ which is indeed the correct value.

Recall that by our construction, for every $x \in \{0, 1\}^\ell$, the set of shares $\{a_{i,x_i}\}_{i \in [\ell]}$ always includes shares corresponding to at least one out of the two edges outgoing from every node.

For any node v_i such that $n_{v_i} = 1$, we compute the value of val_i by inspecting F and either computing $val_i = val_{\ell+1} - a_{i,x_i}$ or $val_i = val_{\ell+2} - a_{i,x_i}$ depending on whether (i, x_i) joins v_i to v_{acc} or v_{rej} respectively. This assigns values to every node which has an edge to either v_{acc} or v_{rej} . Indeed, these are identical to the values assigned in the $\text{GenShares}(F, s)$ algorithm.

We can continue this process iteratively for all nodes with $n_{v_i} = 2$ and so on till we get val_i , for all $i \in [\ell + 2]$. At this step, we have recovered all the randomness used by GenShares . Finally Step 8 of Regenerate simply repeats the steps performed to generate the honest shares. Since we correctly recovered the same randomness which was used in GenShares , it must be that $\forall (j, b) \in E$ such that $a_{j,b} \neq a'_{j,b}$. Therefore we get $a_{i,b}$ for all $i \in [\ell]$ and all $b \in \{0, 1\}$ using the val_i s. □

Perfect privacy:

Lemma 4.13. *For any function F and any input x such that $F(x) = 0$, for any two secret $s_1, s_2 \in \mathcal{P}$, the distributions $(a_{i,x_i})_{i \in \ell}$ and $(a'_{i,x_i})_{i \in \ell}$ are identical, where $(a_{1,0}, a_{1,1}, \dots, a_{\ell,0}, a_{\ell,1}) \leftarrow \text{SS-RR.GenShares}(F, s_1)$ and $(a'_{1,0}, a'_{1,1}, \dots, a'_{\ell,0}, a'_{\ell,1}) \leftarrow \text{SS-RR.GenShares}(F, s_2)$.*

Proof. Intuitively, the set of unauthorized shares does not have any path in the graph F which between v_1 and v_{acc} . Thus, there is at least one *cut* missing in this subgraph, and hence the set of unauthorized shares can have no information about the shared secret. This proof follows the blueprint of the proof of perfect privacy of the undirected st-connectivity secret sharing scheme of [Bei11]. We present the formal proof below.

To see that this scheme is private consider an unauthorized set, that is, a set of edges T not containing a path from v_1 to v_{acc} . Define the set of vertices V_1 such that path in the graph made of nodes in F and edges set T from v_i to v_{acc} . By definition, $v_{\text{acc}} \in V_1$ and $v_1 \notin V_1$. Furthermore, for every $(v_i, v_j) \in T$ either both vertices v_i, v_j are in V_1 or both of them are not in V_1 . It is also crucial to note that the graph formed by edges in T cannot contain any *forks* because the set of unauthorized shares contains shares corresponding to exactly one outgoing edge for every node, and therefore crucially $v_{\text{rej}} \notin V_1$.

Let $\{a_{i,b}\}_{(i,b) \in T}$ be a set of shares generated for the edges in T with the secret $s = s_1$. We next show that the number of randomness r that generate $\{a_{i,b}\}_{(i,b) \in T}$ given the secret $s = s_1$ is equal to the number of vectors of random bits that generate these shares given the secret $s = s_2$.

Let $(val_1, \dots, val_{\ell+2})$ be the vector of randomness that generates $\{a_{i,b}\}_{(i,b) \in T}$ with the secret s_1 . Recall that $val_1 = 0, val_{\ell+1} = val_{\text{acc}} = s_1$ and $val_{\ell+2} = val_{\text{rej}} = 0$. Now consider the following: set $val'_i = val_i + (s_2 - s_1)$ if $v_i \in V_1$ else set $val'_i = val_i$. Note that $val'_1 = 0, val'_{\text{acc}} = s_2$ and $val'_{\text{rej}} = 0$. Therefore the vector $(val'_1, \dots, val'_{\ell+2})$ generates shares for secret s_2 . If we can show that $(val'_1, \dots, val'_{\ell+2})$ also generate the shares $\{a_{i,b}\}_{(i,b) \in T}$, then we will be done. This is because we can conclude that the number of vectors of randomness that yield s_1 must be the same as the number of vectors of randomness that yield s_2 for all s_1, s_2 . Adding the fact that the vector of randomness is chosen uniformly random, we get the conclusion that the distribution of unauthorized shares is invariant of the choice of secret. Thus, it suffices to prove that $(val'_1, \dots, val'_{\ell+2})$ also generate the shares $\{a_{i,b}\}_{(i,b) \in T}$. Here we consider two cases:

- For node pairs (v_i, v_j) such that both $v_i, v_j \in V_1$ and the edges joining v_i, v_j is in T , the share corresponding to this edges is $val'_j - val'_i = val_j + (s_2 - s_1) - val_i + (s_2 - s_1) = val_j - val_i$. Thus the share remains unchanged.
- For node pairs (v_i, v_j) such that both $v_i, v_j \notin V_1$ and the edges joining v_i, v_j is in T , the share corresponding to this edges is $val'_j - val'_i = val_j - val_i$. Thus the share again remains unchanged.

This concludes the proof. □

Regeneration Soundness

Lemma 4.14. *For any function $F \in \mathcal{F}$, for all set of (maliciously generated) shares $(a_{1,0}, a_{1,1}, \dots, a_{\ell,0}, a_{\ell,1}) \in \mathcal{Q}^{2\ell}$, for any $x \in \{0, 1\}^\ell$ such that $F(x) = 1$,*

$$\exists s \in \mathcal{P}, \exists r: \text{SS-RR.Regenerate}(F, x, (a_{1,x_1}, \dots, a_{\ell,x_\ell})) = \text{SS-RR.GenShares}(F, s; r)$$

Proof. The proof of this lemma follows by observing the construction of `SS-RR.Regenerate`. First observe that since $F(x) = 1$, the shares $(a_{1,x_1}, \dots, a_{\ell,x_\ell})$ must include a path from v_1 to v_{acc} that will be detected by DFS. So, `SS-RR.Regenerate` will not output $\perp$.

Clearly, upon input any set of shares, the first step is to compute val_{acc} that will correspond to some value s . The steps of regeneration after this involves recomputing the values of every node assuming $v_{\text{acc}} = s$, $v_1 = v_{\text{rej}} = 0$. Clearly the construction ensures that the shares on the edges which have a path from v_1 to v_{acc} all add up to s . Therefore, it must be that the resulting shares $(a'_{1,0}, a'_{1,1}, \dots, a'_{\ell,0}, a'_{\ell,1})$ is a valid sharing of s with randomness $r := (val_2, \dots, val_\ell)$. $\square$

Validation soundness

Lemma 4.15. *For any function $F \in \mathcal{F}$, for all set of (maliciously generated) shares $(a_{1,0}, a_{1,1}, \dots, a_{\ell,0}, a_{\ell,1}) \in \mathcal{Q}^{2\ell}$, for any $x \in \{0, 1\}^\ell$ such that $F(x) = 1$, if $\nexists s \in \mathcal{P}, r$ such that $\text{SS-RR.GenShares}(F, s; r) = (a_{1,0}, a_{1,1}, \dots, a_{\ell,0}, a_{\ell,1})$, then*

$$\text{SS-RR.isValid}(F, (a_{1,0}, a_{1,1}, \dots, a_{\ell,0}, a_{\ell,1})) = \perp,$$

and if $\exists s \in \mathcal{P}, r$ such that $\text{SS-RR.GenShares}(F, s; r) = (a_{1,0}, a_{1,1}, \dots, a_{\ell,0}, a_{\ell,1})$, then

$$\text{SS-RR.isValid}(F, (a_{1,0}, a_{1,1}, \dots, a_{\ell,0}, a_{\ell,1})) = 1.$$

Proof. Using arguments from the previous proof, we know that $(a'_{1,0}, a'_{1,1}, \dots, a'_{\ell,0}, a'_{\ell,1})$ must be such that $\exists s \in \mathcal{P}, r$ such that $\text{SS-RR.GenShares}(F, s; r) = (a'_{1,0}, a'_{1,1}, \dots, a'_{\ell,0}, a'_{\ell,1})$, thus if $\nexists s \in \mathcal{P}, r$ such that $\text{SS-RR.GenShares}(F, s; r) = (a_{1,0}, a_{1,1}, \dots, a_{\ell,0}, a_{\ell,1})$, then it is obvious that Step 10 of `SS-RR.isValid` returns $\perp$.

On the other hand, if $\exists s \in \mathcal{P}, r$ such that $\text{SS-RR.GenShares}(F, s; r) = (a_{1,0}, a_{1,1}, \dots, a_{\ell,0}, a_{\ell,1})$, then using arguments from reconstruction correctness and the fact that the regeneration process is deterministic, we can say that

$$\text{SS-RR.Regenerate}(F, (a_{1,0}, a_{1,1}, \dots, a_{\ell,0}, a_{\ell,1})) \neq \perp.$$

$\square$

Thus, we get the following theorem:

Theorem 4.16 (Efficient secret sharing for non-monotone functions with randomness recovery for functions in (PBP)). *There exists an efficient secret sharing scheme with randomness recovery satisfying perfect reconstruction correctness, perfect regeneration correctness, perfect privacy and regeneration soundness for any access function $F \in \text{PBP}$.*

A computational monotone secret sharing scheme with randomness recovery. Additionally, we also construct a randomness-recoverable monotone secret sharing scheme with computational privacy in Appendix A.2. The construction relies on a primitive called Targeted Lossy Pseudorandom Generators (see Definition A.2) which can be constructed from one-way function. We believe such a scheme can be of independent interest, but postpone the details to the appendix.

Remark 4.17 (Comparison with [HKV24b]). Hajiabadi et al. introduce the notion of randomness recoverable secret sharing schemes both in the information theoretic and computational regimes. Our DAG based information theoretic construction is incomparable to their work as our scheme

works for a *non-monotone access structure*, and therefore it is not really a secret sharing scheme in the usual sense. On the other hand, our computational LSSS is a strictly weaker object because it needs a Setup algorithm to generate the secret along with the shares. On the positive side, we can show that such an LSSS with setup suffices to construct ABSFE which in turn implies DV-NIZK for NP, and we can construct them directly from One-Way functions. Note that the construction in [HKV24b] relies on either a one-time KDM secure SKE or hinting PRGs [KW19] which are not known to be implied by One-Way functions.

5 The ABSFE Construction

In this section we lay out the details of our ABSFE construction for some function $f : \{0,1\}^\ell \rightarrow \{0,1\}$. Our construction uses the following building blocks in a black-box manner:

- A length parameterized lossy trapdoor function family $\text{LTDF} = (\text{LTDF.Setup}, \text{LTDF.AltSetup}, \text{LTDF.Eval})$ with input length δ and lossiness parameter γ_1 . See Section 3.2 and Theorem 3.4.
- A non-monotone secret sharing scheme with randomness recovery $\text{SS-RR} = (\text{SS-RR.GenShares}, \text{SS-RR.Reconstruct}, \text{SS-RR.Regenerate})$ for f with long secret and shares in $\{0,1\}^\delta$. See Section 4 and Theorem 4.16.
- A pairwise independent hash function family $\mathcal{H} = \{H : \{0,1\}^\delta \rightarrow \{0,1\}^\gamma\}$ where $\gamma < \delta - 2\ell\gamma_1 - 2\log(\frac{1}{\text{negl}(\lambda)})$. Here λ is the security parameter for the scheme. See Definition 3.1.

We summarize the various parameters used in our construction in the table below:

Parameter	Notation
Security parameter	λ
Length of ABSFE key and	ℓ
Length of plaintext message and output length of hash function	γ
Input length to the hash function and LTDF	δ
Lossiness parameter of LTDF	γ_1

5.1 Construction

As mentioned, we rely on the following primitives in a black box manner: a length-parametrized lossy trapdoor function family, a pairwise independent hash function family, and a non-monotone secret sharing scheme with randomness recovery.

Notation: For any bit $b \in \{0,1\}$, $\bar{b}$ denotes $1 - b \in \{0,1\}$. For any function f with two inputs, say (x,k) , f_x is the function which is hardcoded with x which upon input k satisfies the property that for all (x,k) , $f(x,k) = f_x(k)$.

- $\text{ABSFE.Setup}(1^\lambda)$:
 1. Sample $H \xleftarrow{\$} \mathcal{H}$
 2. Output $\text{pp} := H$
- $\text{ABSFE.Keygen}(\text{pp}, k \in \{0,1\}^\ell)$:
 1. $\forall i \in ([\ell], (ev_{i,k_i}, td_{i,k_i}) \leftarrow \text{LTDF.Setup}(1^\lambda, 1^\delta)$

2. $\forall i \in ([\ell], ev_{i, \overline{k_i}} \leftarrow \text{LTDF.AltSetup}(1^\lambda, 1^\delta)$
 3. $\text{pk} \leftarrow \{ev_{i,b}\}_{i \in [\ell], b \in \{0,1\}}$
 4. $\text{sk} := (k, \{td_{i,k_i}\}_{i \in \ell})$
 5. $\text{msk} := \text{sk}$
 6. Output (pk, sk)
- $\text{ABSFE.Enc}(\text{pp}, \text{pk}, x \in \{0,1\}^\lambda, m \in \{0,1\}^\gamma)$:
 1. $s \xleftarrow{\$} \{0,1\}^\delta$
 2. $(a_{i,0}, a_{i,1}, \dots, a_{\ell,0}, a_{\ell,1}) \leftarrow \text{SS-RR.GenShares}(f_x, s)$
 3. $\forall j \in \ell, b \in \{0,1\}, \text{ct}_{j,b} \leftarrow \text{LTDF.Eval}(ev_{j,b}, a_{j,b})$
 4. $\text{ct}_0 \leftarrow m \oplus H(s)$
 5. Output $\text{ct} := (\text{ct}_0, \{\text{ct}_{j,b}\}_{j \in [\ell], b \in \{0,1\}})$
 - $\text{ABSFE.Dec}(\text{pp}, \text{sk}, x, \text{ct})$:
 1. If $f(x, k) = 0$, then output $\perp$
 2. for all $j \in [\ell]$, let $a'_{j,k_j} \leftarrow \text{LTDF.Invert}(td_{j,k_j}, \text{ct}_{j,k_j})$
 3. $(\tilde{a}_{1,0}, \tilde{a}_{1,1}, \dots, \tilde{a}_{\ell,0}, \tilde{a}_{\ell,1}) \leftarrow \text{SS-RR.Regenerate}(f_x, k, \{a'_{j,k_j}\}_{j \in [\ell]})$
 4. $s' \leftarrow \text{SS-RR.Reconstruct}(f_x, \{\tilde{a}_{j,b}\}_{j \in [\ell], b \in \{0,1\}})$
 5. If $\exists j \in [\ell], b \in \{0,1\}$, such that $\text{LTDF.Eval}(ev_{j,b}, \tilde{a}_{j,b}) \neq \text{ct}_{j,b}$, then output $\perp$
 6. Else, output $\text{ct}_0 \oplus H(s')$

Correctness. We prove correctness, namely, that decrypting an ABE ciphertext with an appropriate secret key results with the correct underlying message.

Lemma 5.1. *For all $\lambda \in \mathbb{N}$, $\gamma, n \in \text{poly}(\lambda)$, $m \in \{0,1\}^\gamma$, $x \in \{0,1\}^n$ and $k \in \{0,1\}^\lambda$, assuming perfect regeneration and reconstruction correctness of SS-RR, and inversion correctness of LTDF in the injective mode, we have*

$$\Pr \left[\text{ABSFE.Dec}(\text{pp}, \text{sk}, x, \text{ABSFE.Enc}(\text{pp}, \text{pk}, x, m)) = m \mid \text{pp} \leftarrow \text{ABSFE.Setup}(1^\lambda); \right. \\ \left. (\text{pk}, \text{sk}) \leftarrow \text{ABSFE.Keygen}(\text{pp}, k) \right] = 1$$

Proof. This follows immediately from the correctness of LTDF inversion and correctness of SS-RR reconstruction and regeneration algorithms. We further elaborate on this. Note that we are only interested in the case where $f(x, k) = 1$. Note that this means k must be an *authorized* input for the function f_x . Now,

1. for all $j \in [\ell]$, ct_{j,k_j} was computed honestly using LTDF.Eval , therefore the correctness of LTDF inversion in the injective mode implies that a'_{j,k_j} must indeed be the correct and unique pre-image of ct_{j,k_j} with respect to the evaluation key ev_{j,k_j} .
2. Since, k is an authorized input for f_k , perfect regeneration correctness of SS-RR implies that for all $j \in [\ell], b \in \{0,1\}$, $\tilde{a}_{j,b} = a'_{j,b}$ where $\{a'_{j,b}\}_{j \in [\ell], b \in \{0,1\}} \leftarrow \text{SS-RR.GenShares}(f_x, s)$.
3. The above step also guarantees that Step 5 of the decryption procedure does not output $\perp$.
4. Perfect reconstruction correctness of SS-RR also guarantees that $s' = s$, therefore $\text{ct}_0 \oplus H(s') = \text{ct}_0 \oplus H(s) = m$.

This completes the proof. $\square$

Strong key hiding. We prove key hiding, namely, we show that oracle access to the function key generation function does not reveal any information about the PRF key k . Formally, this is captured by showing the existence of a simulator which gets the master secret key and has oracle access to $F(\cdot, k)$, and produces outputs which are computationally indistinguishable from that generated by the real world decryption procedure.

Lemma 5.2. *Assume the existence of length parameterized lossy trapdoor functions (see Definition 3.2). For all $\lambda \in \mathbb{N}$, there exists a PPT simulator $\text{Sim} := (\text{Sim}_1, \text{Sim}_2)$ such that for all keys $k \in \{0, 1\}^\ell$ and for all PPT adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that*

$$\left| \Pr \left[\mathcal{A}^{\mathcal{O}_1(\text{pp}, \text{sk}, \cdot)}(\text{pp}, \text{pk}) = 1 \mid \text{pp} \leftarrow \text{ABSFE.Setup}(1^\lambda), (\text{pk}, \text{sk}) \leftarrow \text{ABSFE.Keygen}(\text{pp}, k) \right] - \Pr \left[\mathcal{A}^{\mathcal{O}_2(\text{st}, \cdot)}(\text{pp}, \text{pk}) = 1 \mid (\text{st}, \text{pp}, \text{pk}) \leftarrow \text{Sim}_1(1^\lambda) \right] \right| \leq \text{negl}(\lambda),$$

where $\mathcal{O}_1$ is the real decryption oracle, i.e., $\text{ABSFE.Dec}(\text{pp}, \text{sk}, x, \text{ct})$, and $\mathcal{O}_2$ is the ideal decryption oracle which uses the simulator $\text{Sim}_2^{f(\cdot, k)}(\text{st}, x, \text{ct})$.

Proof. It suffices to construct a simulator such that the two oracles $\mathcal{O}_1$ and $\mathcal{O}_2$ are computationally indistinguishable. Clearly, the existence of a distinguisher for this can be transformed into an adversary that can win the key-hiding game with non-negligible probability.

First, let us describe the simulators $\text{Sim}_1(1^\lambda)$ and $\text{Sim}_2^{F(\cdot, k)}(\text{st}, x, \text{ct})$:

- $\text{Sim}_1(1^\lambda)$:

1. $H \xleftarrow{\$} \mathcal{H}$
2. $\forall (i, b) \in ([\ell] \times \{0, 1\}), (ev_{i,b}, td_{i,b}) \leftarrow \text{LTDF.Setup}(1^\lambda, 1^\delta)$
3. $\text{pk} \leftarrow \{ev_{i,b}\}_{i \in [\ell], b \in \{0, 1\}}$
4. $\text{st} \leftarrow (\text{pk}, \{td_{i,b}\}_{i \in [\ell], b \in \{0, 1\}})$
5. Output $(\text{st}, H, \text{pk})$

- $\text{Sim}_2^{F(\cdot, k)}(\text{st}, x, \text{ct})$

1. Parse ct as $(\text{ct}_0, \{\text{ct}_{j,b}\}_{j \in [\ell], b \in \{0, 1\}})$
2. Query x to oracle $F(\cdot, k)$. If it returns 0, then output $\perp$, else continue
3. $\forall j \in [\ell], b \in \{0, 1\}, a'_{j,b} \leftarrow \text{LTDF.Invert}(td_{j,b}, \text{ct}_{j,b})$
4. $\beta \leftarrow \text{SS-RR.isValid}(f_x, \{a'_{j,b}\}_{j \in [\ell], b \in \{0, 1\}})$
5. If $\beta = \perp$, then output $\perp$
6. Else, output $\text{ct}_0 \oplus H(s')$

We will now go through a sequence of hybrids starting from the real world decryption oracle. Note that in all the hybrids here, the challenger always has access to the master secret key.

- **Hybrid 0:** This is the real world key generation and decryption oracle which responds to ciphertext queries by using the secret key sk . We write out the entire hybrid for ease of exposition. The setup algorithm remains unchanged, hence we skip writing it out.

- **ABSFE.Keygen**(pp, $k \in \{0, 1\}^\ell$):
 1. $\forall i \in [\ell], (ev_{i,k_i}, td_{i,k_i}) \leftarrow \text{LTDF.Setup}(1^\lambda, 1^\delta)$
 2. $\forall i \in [\ell], ev_{i,\overline{k_i}} \leftarrow \text{LTDF.AltSetup}(1^\lambda, 1^\delta)$
 3. $\text{pk} \leftarrow \{ev_{i,b}\}_{i \in [\ell], b \in \{0,1\}}$
 4. $\text{sk} := (k, \{td_{i,k_i}\}_{i \in \ell})$
 5. $\text{msk} := \text{sk}$
 6. Output (pk, sk)
- **ABSFE.Dec**(pp, sk, x , ct):
 1. If $f_x(k) = 0$, then output $\perp$
 2. for all $j \in [\ell]$, let $a'_{j,k_j} \leftarrow \text{LTDF.Invert}(td_{j,k_j}, \text{ct}_{j,k_j})$
 3. $(\tilde{a}_{1,0}, \tilde{a}_{1,1}, \dots, \tilde{a}_{\ell,0}, \tilde{a}_{\ell,1}) \leftarrow \text{SS-RR.Regenerate}(f_x, k, \{a'_{j,k_j}\}_{j \in [\ell]})$
 4. $s' \leftarrow \text{SS-RR.Reconstruct}(f_x, \{\tilde{a}_{j,b}\}_{j \in [\ell], b \in \{0,1\}})$
 5. If $\exists j \in [\ell], b \in \{0, 1\}$, such that $\text{LTDF.Eval}(ev_{j,b}, \tilde{a}_{j,b}) \neq \text{ct}_{j,b}$, then output $\perp$
 6. Else, output $\text{ct}_0 \oplus H(s')$
- **Hybrid 1:** We change the key generation algorithm such that for all $i \in [\ell]$ and $b \in \{0, 1\}$, the keys for LTDF are generated in the injective mode. The additional trapdoor keys thus generated are added to the master secret key.

- **ABSFE.Keygen**(pp, $k \in \{0, 1\}^\ell$):
 1. $\forall i \in ([\ell], b \in \{0, 1\}), (ev_{i,b}, td_{i,b}) \leftarrow \text{LTDF.Setup}(1^\lambda, 1^\delta)$
 2. $\text{pk} \leftarrow \{ev_{i,b}\}_{i \in [\ell], b \in \{0,1\}}$
 3. $\text{sk} := (k, \{td_{i,k_i}\}_{i \in \ell})$
 4. $\text{msk} := \{td_{i,b}\}_{i \in [\ell], b \in \{0,1\}}$
 5. Output (pk, sk)
- **ABSFE.Dec**(pp, sk, x , ct):
 1. If $f_x(k) = 0$, then output $\perp$
 2. for all $j \in [\ell]$, let $a'_{j,k_j} \leftarrow \text{LTDF.Invert}(td_{j,k_j}, \text{ct}_{j,k_j})$
 3. $(\tilde{a}_{1,0}, \tilde{a}_{1,1}, \dots, \tilde{a}_{\ell,0}, \tilde{a}_{\ell,1}) \leftarrow \text{SS-RR.Regenerate}(f_x, k, \{a'_{j,k_j}\}_{j \in [\ell]})$
 4. $s' \leftarrow \text{SS-RR.Reconstruct}(f_x, \{\tilde{a}_{j,b}\}_{j \in [\ell], b \in \{0,1\}})$
 5. If $\exists j \in [\ell], b \in \{0, 1\}$, such that $\text{LTDF.Eval}(ev_{j,b}, \tilde{a}_{j,b}) \neq \text{ct}_{j,b}$, then output $\perp$
 6. Else, output $\text{ct}_0 \oplus H(s')$
- **Hybrid 2:** In addition to the steps in Hybrid 1, we use msk to invert $\text{ct}_{j,b}$ for all $j \in [\ell], b \in \{0, 1\}$. We also add an extra step which checks if these pre-images match $\tilde{a}_{j,b}$, for all $j \in [\ell], b \in \{0, 1\}$. If not, then we output $\perp$. From here, onwards, the key generation algorithm remains unchanged, hence, we do not write it explicitly for the sake of space.

1. If $f_x(k) = 0$, then output $\perp$
2. for all $j \in [\ell], b \in \{0, 1\}$, let $a'_{j,b} \leftarrow \text{LTDF.Invert}(td_{j,b}, \text{ct}_{j,b})$
3. $(\tilde{a}_{1,0}, \tilde{a}_{1,1}, \dots, \tilde{a}_{\ell,0}, \tilde{a}_{\ell,1}) \leftarrow \text{SS-RR.Regenerate}(f_x, k, \{a'_{j,k_j}\}_{j \in [\ell]})$
4. $s' \leftarrow \text{SS-RR.Reconstruct}(f_x, \{\tilde{a}_{j,b}\}_{j \in [\ell], b \in \{0,1\}})$

5. If $\exists j \in [\ell], b \in \{0, 1\}$, such that $\text{LTDF.Eval}(ev_{j,b}, \tilde{a}_{j,b}) \neq \text{ct}_{j,b}$, then output $\perp$
6. If $\exists j \in [\ell], b \in \{0, 1\}$ such that $a'_{j,b} \neq \tilde{a}_{j,b}$, then output $\perp$
7. Else, output $\text{ct}_0 \oplus H(s')$

- **Hybrid 3:** In this hybrid, we regenerate the shares using all the shares which were inverted using the master secret key. The reason for doing this step is to check whether $\{a'_{j,b}\}_{j \in [\ell], b \in \{0,1\}}$ forms a valid shares for some secret.

1. If $f_x(k) = 0$, then output $\perp$
2. for all $j \in [\ell], b \in \{0, 1\}$, let $a'_{j,b} \leftarrow \text{LTDF.Invert}(td_{j,b}, \text{ct}_{j,b})$
3. $(\tilde{a}_{1,0}, \tilde{a}_{1,1}, \dots, \tilde{a}_{\ell,0}, \tilde{a}_{\ell,1}) \leftarrow \text{SS-RR.Regenerate}(f_x, k, \{a'_{j,k_j}\}_{j \in [\ell]})$
4. $\beta \leftarrow \text{SS-RR.isValid}(f_x, \{a'_{j,b}\}_{j \in [\ell], b \in \{0,1\}})$
5. if $\beta = \perp$, then output $\perp$
6. $s' \leftarrow \text{SS-RR.Reconstruct}(f_x, \{\tilde{a}_{j,b}\}_{j \in [\ell], b \in \{0,1\}})$
7. If $\exists j \in [\ell], b \in \{0, 1\}$, such that $\text{LTDF.Eval}(ev_{j,b}, \tilde{a}_{j,b}) \neq \text{ct}_{j,b}$, then output $\perp$
8. If $\exists j \in [\ell], b \in \{0, 1\}$ such that $a'_{j,b} \neq \tilde{a}_{j,b}$, then output $\perp$
9. Else, output $\text{ct}_0 \oplus H(s')$

- **Hybrid 4:** Here we reconstruct the secret using different the newly regenerated shares.

1. If $f_x(k) = 0$, then output $\perp$
2. for all $j \in [\ell], b \in \{0, 1\}$, let $a'_{j,b} \leftarrow \text{LTDF.Invert}(td_{j,b}, \text{ct}_{j,b})$
3. $(\tilde{a}_{1,0}, \tilde{a}_{1,1}, \dots, \tilde{a}_{\ell,0}, \tilde{a}_{\ell,1}) \leftarrow \text{SS-RR.Regenerate}(f_x, k, \{a'_{j,k_j}\}_{j \in [\ell]})$
4. $\beta \leftarrow \text{SS-RR.isValid}(f_x, \{a'_{j,b}\}_{j \in [\ell], b \in \{0,1\}})$
5. if $\beta = \perp$, then output $\perp$
6. $s' \leftarrow \text{SS-RR.Reconstruct}(f_x, \{\tilde{a}_{j,b}\}_{j \in [\ell], b \in \{0,1\}})$
7. $s^* \leftarrow \text{SS-RR.Reconstruct}(f_x, \{a'_{j,b}\}_{j \in [\ell], b \in \{0,1\}})$
8. If $\exists j \in [\ell], b \in \{0, 1\}$, such that $\text{LTDF.Eval}(ev_{j,b}, \tilde{a}_{j,b}) \neq \text{ct}_{j,b}$, then output $\perp$
9. If $\exists j \in [\ell], b \in \{0, 1\}$ such that $a'_{j,b} \neq \tilde{a}_{j,b}$, then output $\perp$
10. Else, output $\text{ct}_0 \oplus H(s^*)$

- **Hybrid 5:** We will remove some extra steps.

1. If $f_x(k) = 0$, then output $\perp$
2. for all $j \in [\ell], b \in \{0, 1\}$, let $a'_{j,b} \leftarrow \text{LTDF.Invert}(td_{j,b}, \text{ct}_{j,b})$
3. $(\tilde{a}_{1,0}, \tilde{a}_{1,1}, \dots, \tilde{a}_{\ell,0}, \tilde{a}_{\ell,1}) \leftarrow \text{SS-RR.Regenerate}(f_x, k, \{a'_{j,k_j}\}_{j \in [\ell]})$
4. $\beta \leftarrow \text{SS-RR.isValid}(f_x, \{a'_{j,b}\}_{j \in [\ell], b \in \{0,1\}})$
5. if $\beta = \perp$, then output $\perp$
6. $s' \leftarrow \text{SS-RR.Reconstruct}(f_x, \{\tilde{a}_{j,b}\}_{j \in [\ell], b \in \{0,1\}})$
7. $s^* \leftarrow \text{SS-RR.Reconstruct}(f_x, \{a'_{j,b}\}_{j \in [\ell], b \in \{0,1\}})$
8. If $\exists j \in [\ell], b \in \{0, 1\}$, such that $\text{LTDF.Eval}(ev_{j,b}, \tilde{a}_{j,b}) \neq \text{ct}_{j,b}$, then output $\perp$

9. ~~If $\exists j \in [\ell], b \in \{0, 1\}$ such that $a'_{j,b} \neq \tilde{a}_{j,b}$, then output $\perp$~~
10. Else, output $\text{ct}_0 \oplus H(s^*)$

• **Hybrid 6:** This is identical to the Simulator

1. Query x to oracle $F(\cdot, k)$. If it returns 0, then output $\perp$, else continue
2. for all $j \in [\ell], b \in \{0, 1\}$, let $a'_{j,b} \leftarrow \text{LTDF.Invert}(td_{j,b}, \text{ct}_{j,b})$
3. $\beta \leftarrow \text{SS-RR.isValid}(f_x, \{a'_{j,b}\}_{j \in [\ell], b \in \{0, 1\}})$
4. if $\beta = \perp$, then output $\perp$
5. Else, output $\text{ct}_0 \oplus H(s^*)$

Lemma 5.3. *Assuming setup indistinguishability of LTDF, for all $\lambda \in \mathbb{N}$, and all PPT distinguishers $\mathcal{D}$, there exists a negligible function such that*

$$|\Pr[\mathcal{D}^{\text{Hybrid } 0}(1^\lambda) = 1] - \Pr[\mathcal{D}^{\text{Hybrid } 1}(1^\lambda) = 1]| \leq \text{negl}(\lambda).$$

Proof. The indistinguishability of these two hybrids follow from a straightforward reduction to the setup indistinguishability of LTDF via a sequence of intermediate hybrids. In more detail, we define an a sequence of intermediate hybrids indexed by $j \in [\ell]$, where Hybrid $0, j$ defines is defined as follows: $\forall i \leq j, (ev_{i,k_i}, td_{i,k_i}) \leftarrow \text{LTDF.Setup}(1^\lambda, 1^\delta)$ and $ev_{i,\bar{k}_i} \leftarrow \text{LTDF.AltSetup}(1^\lambda, 1^\delta)$. And $\forall i > j, (ev_{i,b}, td_{i,b}) \leftarrow \text{LTDF.Setup}(1^\lambda, 1^\delta)$, for all $b \in \{0, 1\}$. It therefore suffices to show that for all $j \in [\ell]$ Hybrid $0, j$ is computationally indistinguishable from Hybrid $0, j+1$. We prove that if there exists an adversary $\mathcal{D}$ that can distinguish between these hybrids, then there exists a distinguisher $\mathcal{A}$ that upon receiving an evaluation key ev^* can tell whether ev^* was generated by LTDF.Setup or LTDF.AltSetup. The reduction proceeds as follows: Upon receiving ev^* , $\mathcal{A}$ simulated the ABSFE algorithms. In particular, it runs ABSFE.Setup and ABSFE.Keygen with the only change that for index $i = j$ it sends ev^* instead of $ev_{i,\bar{k}_i}$. It then sends the output of setup and key generation to $\mathcal{D}$. Note that if ev^* was generated by LTDF.Setup, then $\mathcal{D}$ is interacting with Hybrid $0, j+1$, whereas if ev^* was generated by LTDF.AltSetup, then $\mathcal{D}$ is interacting with Hybrid $0, j$. Finally, upon receiving decryption queries from $\mathcal{D}$, $\mathcal{A}$ simulates ABSFE.Dec. Note that this is doable because $\mathcal{A}$ never has to run LTDF.Invert on any index of the form $(i, \bar{k}_i)$. Clearly, the distinguishing advantage of $\mathcal{A}$ is same as that of $\mathcal{D}$. $\square$

Lemma 5.4. *Assuming correctness of LTDF inversion in the injective mode, for all $\lambda \in \mathbb{N}$, and all PPT distinguishers $\mathcal{D}$,*

$$\Pr[\mathcal{D}^{\text{Hybrid } 1}(1^\lambda) = 1] = \Pr[\mathcal{D}^{\text{Hybrid } 2}(1^\lambda) = 1].$$

In fact we prove that Hybrid 1 and 2 are identical to each other.

Proof. The only difference between Hybrid 1 and Hybrid 2 is the additional check in Step 6 in Hybrid 2. The only case in which this additional check can change the output behavior is if $\nexists (j \in [\ell], b \in \{0, 1\})$ such that $\text{LTDF.Eval}(ev_{j,b}, \tilde{a}_{j,b}) \neq \text{ct}_{j,b}$ but $\exists j \in [\ell], b \in \{0, 1\}$ such that $a'_{j,b} \neq \tilde{a}_{j,b}$.

However, if it is the case that $\forall j \in [\ell], b \in \{0, 1\}, \text{LTDF.Eval}(ev_{j,b}, \tilde{a}_{j,b}) = \text{ct}_{j,b}$, then the injectivity of LTDF ensures that $\forall j \in [\ell], b \in \{0, 1\}, \text{LTDF.Invert}(td_{j,b}, \text{ct}_{j,b}) = \tilde{a}_{j,b}$. Thus, it must be that $\forall j \in [\ell], b \in \{0, 1\}, a'_{j,b} \neq \tilde{a}_{j,b}$.

Therefore, both the hybrids are identical, and therefore computationally indistinguishable. $\square$

Lemma 5.5. *Assuming regeneration soundness and validation soundness of SS-RR Hybrids 2 and 3 are identical to each other.*

Proof. Hybrids 2 and 3 are not identical if Step 5 in Hybrid 3 outputs $\perp$ but Hybrid 2 does not output $\perp$. Now, Step 5 in Hybrid 3 outputs $\perp$ iff $(a'_{1,0}, a'_{1,1}, \dots, a'_{\ell,0}, a'_{\ell,1})$ is not a valid set of shares generated by $\text{SS-RR.GenShares}(f, s)$ for any $s \in \{0, 1\}^\delta$. This is because of the soundness of SS-RR.isValid . In this case, Step 6 in Hybrid 2 will also output $\perp$ because $\{\tilde{a}_{j,b}\}_{j \in [\ell], b \in \{0,1\}}$ is an output of SS-RR.Regenerate and we have regeneration soundness. Thus it cannot match $\{a'_{j,b}\}_{j \in [\ell], b \in \{0,1\}}$ because there does exist any $s \in \{0, 1\}^\delta$ such that $(a'_{1,0}, a'_{1,1}, \dots, a'_{\ell,0}, a'_{\ell,1}) \leftarrow \text{SS-RR.GenShares}(f, s)$.

Thus, Hybrid 2 and Hybrid 3 are identical assuming regeneration and validation soundness of SS-RR. □

Lemma 5.6. *Hybrid 3 is identical to Hybrid 4.*

Proof. These two hybrids are identical if $s' = s^*$ or both do not output $\perp$. Since, SS-RR.Reconstruct is a deterministic algorithm, this is true if $\{\tilde{a}_{j,b}\}_{j \in [\ell], b \in \{0,1\}} = \{a'_{j,b}\}_{j \in [\ell], b \in \{0,1\}}$.

If there exists $j \in [\ell], b \in \{0, 1\}$ such that $a'_{j,b} \neq \tilde{a}_{j,b}$ then both hybrids will anyway output $\perp$.

Therefore, if both hybrids do not output $\perp$, then it must be that $\{\tilde{a}_{j,b}\}_{j \in [\ell], b \in \{0,1\}} = \{a^*_{j,b}\}_{j \in [\ell], b \in \{0,1\}}$ and the result follows. □

Lemma 5.7. *Assuming perfect regeneration correctness and validation soundness of SS-RR, Hybrid 4 is identical to Hybrid 5.*

Proof. The hybrids differ only if Steps 8 and 9 in Hybrid 4 outputs $\perp$ but Hybrid 5 does not output $\perp$. Let us split the analysis into two cases:

1. $(a'_{1,0}, a'_{1,1}, \dots, a'_{\ell,0}, a'_{\ell,1})$ is not a valid set of shares generated by $\text{SS-RR.GenShares}(f, s)$ for any $s \in \{0, 1\}^\delta$: In this case, validation soundness ensure that $\beta = \perp$. Thus both hybrids output $\perp$.
2. $(a'_{1,0}, a'_{1,1}, \dots, a'_{\ell,0}, a'_{\ell,1})$ is a valid set of shares generated by $\text{SS-RR.GenShares}(f, s)$ for some $s \in \{0, 1\}^\delta$: In this case, first validation soundness again ensures that $\beta = 1$ in both hybrids. Moreover, regeneration correctness again ensure that $(a'_{1,0}, a'_{1,1}, \dots, a'_{\ell,0}, a'_{\ell,1}) = (\tilde{a}_{1,0}, \tilde{a}_{1,1}, \dots, \tilde{a}_{\ell,0}, \tilde{a}_{\ell,1})$. Thus, Steps 8 and 9 in Hybrid 4 do not output $\perp$.

This concludes the proof. □

Lemma 5.8. *Hybrid 5 is identical to Hybrid 6.*

Proof. The only difference in the two hybrids is the way it is checked whether $f_x(k) = 1$ or not. Rather than computing f_x locally, in hybrid 5, the simulator forwards the query to the $F(\cdot, k)$ oracle. This does not affect the output of the hybrids. □

□

Weak Statistical Message Hiding. We prove weak statistical message hiding. In particular, we show that the encryption of a message has a distribution which is statistically close to uniform random, even when an unbounded adversary is given access to function secret key for a PRF key k , as long as, $F(x, k) = 0$.

Lemma 5.9. *Assume the existence of length parametrized lossy trapdoor functions and a SS-RR perfect privacy. Fix $\lambda, \delta \in \mathbb{N}$, a PRF key $k \in \{0, 1\}^\lambda$. Then, for all admissible adversaries $\mathcal{A} := (\mathcal{A}_1, \mathcal{A}_2)$, there exists a negligible function $\text{negl}(\cdot)$ such that*

$$\Pr \left[b' \leftarrow \mathcal{A}_2^{\mathcal{O}(\text{pp}, \text{pk}, b, \cdot, \cdot)}(\text{pp}, \text{pk}, \text{sk}, \tau) \wedge b' = b \mid (k, \tau) \leftarrow \mathcal{A}_1(1^\lambda), \text{pp} \leftarrow \text{ABSFE.Setup}(1^\lambda), \right. \\ \left. (\text{pk}, \text{sk}) \leftarrow \text{ABSFE.Keygen}(\text{pp}, k), b \xleftarrow{\$} \{0, 1\} \right] = \text{negl}(\lambda),$$

where

- $\mathcal{O}(\text{pp}, \text{pk}, b, (x, m_0, m_1))$ is an oracle which upon queried an attribute x , messages $m_0, m_1 \in \mathcal{M}$ returns a ciphertext $\text{ct} \leftarrow \text{ABSFE.Enc}(\text{pp}, \text{pk}, x, m_b)$.
- $\mathcal{A}$ is admissible if it makes one ciphertext query (x, m_0, m_1) to $\mathcal{O}$, and $F(x, k) = 0$.

Proof. We show via a sequence of hybrids that there does not exist any PPT distinguisher $\mathcal{A}$ that can distinguish between $\text{ABSFE.Enc}(\text{pp}, \text{pk}, x, m_0)$ and $\text{ABSFE.Enc}(\text{pp}, \text{pk}, x, m_1)$ even when it is allowed to make a single-key-generation query for k as long as for all encryption queries of the form $(x, \cdot, \cdot)$ satisfy $F(x, k) = 0$. Since $\mathcal{A}$ is admissible, it receives a secret key sk corresponding to k which is associated with an *unauthorized* set of shares.

1. **Hybrid 0:** This is the real world encryption of a message m

- $\text{ABSFE.Setup}(1^\lambda)$:
 - (a) $H \xleftarrow{\$} \mathcal{H}$
 - (b) output $\text{pp} := \mathcal{H}$
- $\text{ABSFE.Keygen}(\text{pp}, k \in \{0, 1\}^\ell)$:
 - (a) $\forall i \in [\ell], (ev_{i, k_i}, td_{i, k_i}) \leftarrow \text{LTDF.Setup}(1^\lambda, 1^\delta)$
 - (b) $\forall i \in [\ell], ev_{i, \overline{k_i}} \leftarrow \text{LTDF.AltSetup}(1^\lambda, 1^\delta)$
 - (c) $\text{pk} \leftarrow \{ev_{i, b}\}_{i \in [\ell], b \in \{0, 1\}}$
 - (d) $\text{sk} := (k, \{td_{i, k_i}\}_{i \in [\ell]})$
 - (e) $\text{msk} := \text{sk}$
 - (f) Output (pk, sk)
- $\text{ABSFE.Enc}(\text{pp}, \text{pk}, x \in \{0, 1\}^\lambda, m \in \{0, 1\}^\gamma)$:
 - (a) $s \xleftarrow{\$} \{0, 1\}^\delta$
 - (b) $(a_{i, 0}, a_{i, 1}, \dots, a_{\ell, 0}, a_{\ell, 1}) \leftarrow \text{SS-RR.GenShares}(f_x, s)$
 - (c) $\forall j \in \ell, b \in \{0, 1\}, \text{ct}_{j, b} \leftarrow \text{LTDF.Eval}(ev_{j, b}, a_{j, b})$
 - (d) $\text{ct}_0 \leftarrow m \oplus H(s)$
 - (e) Output $\text{ct} := (\text{ct}_0, \{\text{ct}_{j, b}\}_{j \in [\ell], b \in \{0, 1\}})$

2. **Hybrid 1:** We exploit the lossiness of LTDF to make the ciphertext independent of the message m .

- **ABSFE.Setup**(1^λ):
 - (a) $H \leftarrow \mathcal{H}$
 - (b) output $\text{pp} := H$
- **ABSFE.Keygen**($\text{pp}, k \in \{0, 1\}^\ell$):
 - (a) $\forall i \in [\ell], (ev_{i,k_i}, td_{i,k_i}) \leftarrow \text{LTDF.Setup}(1^\lambda, 1^\delta)$
 - (b) $\forall i \in [\ell], ev_{i,\overline{k_i}} \leftarrow \text{LTDF.AltSetup}(1^\lambda, 1^\delta)$
 - (c) $\text{pk} \leftarrow \{ev_{i,b}\}_{i \in [\ell], b \in \{0,1\}}$
 - (d) $\text{msk} \leftarrow \{td_{i,b}\}_{i \in [\ell], b \in \{0,1\}}$
 - (e) $\text{sk} := (k, \{td_{i,k_i}\}_{i \in [\ell]})$
 - (f) Output (pk, sk)
- **ABSFE.Enc**($\text{pp}, \text{pk}, x \in \{0, 1\}^\lambda, m \in \{0, 1\}^\gamma$):
 - (a) $s \xleftarrow{\$} \{0, 1\}^\delta$
 - (b) $(a_{i,0}, a_{i,1}, \dots, a_{\ell,0}, a_{\ell,1}) \leftarrow \text{SS-RR.GenShares}(f_x, s)$
 - (c) $\forall j \in \ell, b \in \{0, 1\}, \text{ct}_{j,b} \leftarrow \text{LTDF.Eval}(ev_{j,b}, a_{j,b})$
 - (d) $r \xleftarrow{\$} \{0, 1\}^\gamma$
 - (e) $\text{ct}_0 \leftarrow r$
 - (f) Output $\text{ct} := (\text{ct}_0, \{\text{ct}_{j,b}\}_{j \in [\ell], b \in \{0,1\}})$

Lemma 5.10. Fix $\lambda \in \mathbb{N}$ and $\delta, \ell, \gamma, \gamma_1 \in \text{poly}(\lambda)$. Assume γ_1 -lossiness of length parameterized LTDF in the lossy mode, for all distinguishers $\mathcal{D}$, there exists a negligible function $\text{negl}(\cdot)$ such that

$$\left| \Pr[\mathcal{D}(\text{pp}, \text{pk}, \text{sk}, \text{ct}) = 1 : (\text{pp}, \text{pk}, \text{sk}, \text{ct}) \leftarrow \text{Hybrid } 0] - \Pr[\mathcal{D}(\text{pp}, \text{pk}, \text{sk}, \text{ct}) = 1 : (\text{pp}, \text{pk}, \text{sk}, \text{ct}) \leftarrow \text{Hybrid } 1] \right| \leq \text{negl}(\lambda),$$

whenever $\gamma < \delta - 2\ell\gamma_1 - 2\log\left(\frac{1}{\text{negl}(\lambda)}\right)$.

Proof. First, observe that for a fixed key k , any adversary can view

$$m \oplus H(s), \{\text{LTDF.Eval}(ev_{i,b}, a_{i,b})\}_{i \in [\ell], b \in \{0,1\}}, \{ev_{i,b}\}_{i \in [\ell], b \in \{0,1\}}, \{td_{i,k_i}\}_{i \in [\ell]}.$$

This is equivalent to viewing

$$m \oplus H(s), \{\text{LTDF.Eval}(ev_{i,\overline{k_i}}, a_{i,\overline{k_i}})\}_{i \in [\ell]}, \{a_{i,k_i}\}_{i \in [\ell]}, \{ev_{i,b}\}_{i \in [\ell], b \in \{0,1\}}.$$

The perfect privacy of SS-RR implies that whenever k is the characteristic string for an unauthorized set of parties, $H_\infty(s \mid a_{i,k_i})_{i \in [\ell]} = H_\infty(s) = \delta$. By Lemma 3.2 and since the size of the range of the length parameterized LTD function LTDF is at most 2^{γ_1} , we get that

$$\begin{aligned} \tilde{H}_\infty(s \mid \{\text{LTDF.Eval}(ev_{i,\overline{k_i}}, a_{i,\overline{k_i}})\}_{i \in [\ell]}, \{a_{i,k_i}\}_{i \in [\ell]}) &\geq H_\infty(s \mid a_{i,k_i})_{i \in [\ell]} - \log(|2^{2\ell\gamma_1}|) \\ &= \delta - 2\ell\gamma_1. \end{aligned}$$

Since $\gamma < \delta - 2\ell\gamma_1 - 2\log\left(\frac{1}{\text{negl}(\lambda)}\right)$ and using Lemma 3.3, we get that

$$\begin{aligned} \Delta\left(\left(\{\text{LTDF.Eval}(ev_{i,\overline{k_i}}, a_{i,\overline{k_i}})\}_{i \in [\ell]}, \{a_{i,k_i}\}_{i \in [\ell]}, r \xleftarrow{\$} \{0, 1\}^\gamma\right), \right. \\ \left. \left(\{\text{LTDF.Eval}(ev_{i,\overline{k_i}}, a_{i,\overline{k_i}})\}_{i \in [\ell]}, \{a_{i,k_i}\}_{i \in [\ell]}, H(s)\right)\right) \leq \text{negl}(\lambda). \end{aligned}$$

Thus, the two hybrids are statistically indistinguishable. $\square$

At this stage, the ciphertext is independent of the message. Thus, statistical message hiding follows. $\square$

The above construction along with Theorem 4.16 immediately yield the following corollary:

Corollary 5.11. *Assuming length parameterized lossy trapdoor functions, there exists ABSFE with statistical message-hiding and computational function-hiding for all functions f computable by polynomial time branching programs (PBP).*

Remark 5.12 (Statistical soundness). By setting the `ABSFE.Keygen` in a way that all the lossy trapdoor function keys are generated in the injective mode, we get an ABSFE scheme with statistical key-hiding and computational weak message-hiding. However, this does not translate to a DV-NIZK with statistical soundness. This is because of a technicality where an intermediate hybrid in the soundness proof relies on the security of a pseudorandom function. We refer reader to [LQR⁺19] for further details regarding the proof of soundness.

Remark 5.13 (On the relevance of length parameterized LTDF). Length parameterized LTDFs are a variant of LTDF where the range of the function is independent of its input length. Typically, LTDFs have parameters such that if the input length is ℓ_{in} , then the range has size $2^{c\ell_{\text{in}}}$ for constant $c < 1$. However, in our application we need the input length $\delta > 2\ell_{\gamma_1}$. Thus the input domain must be larger than the size of the range by an arbitrary polynomial. Such a requirement is satisfied by a length parameterized LTDF.

6 DV-NIZK from ABSFE

We briefly recall the main theorem of [LQR⁺19] which shows that DV-NIZKs can be constructed in a black-box manner from ABSFE. We present their main theorem and show how it can be extended to get DV-NIZK with statistical zero-knowledge if the underlying ABSFE satisfies statistical message-hiding.

Designated Verifier NIZKs from ABSFE [LQR⁺19]: Let $\mathcal{L} \subseteq \{0, 1\}^n$ be an NP language associated with NP relation $\mathcal{R} \subseteq \{0, 1\}^n \times \{0, 1\}^h$. A DV-NIZK for $\mathcal{L}$ relies on the following building blocks:

- A semi-malicious zero-knowledge PCP $\text{zkPCP} = (\text{zkPCP.Prove}, \text{zkPCP.Query}, \text{zkPCP.Ver})$ for $\mathcal{L}$. Let m be the length of the PCP and ρ be a bound on the number of random bits needed for `zkPCP.Query`.
- A pseudorandom function $\text{PRF} : \mathcal{K} \times \{0, 1\}^n \rightarrow \{0, 1\}^\rho$.
- An attribute-based secure function evaluation scheme $\text{ABSFE} = (\text{ABSFE.Setup}, \text{ABSFE.Keygen}, \text{ABSFE.Enc}, \text{ABSFE.Dec})$ with weak message hiding and strong key hiding for function F which is defined below.

Function for ABSFE: Let $\mathcal{F} = \{f : \{0, 1\}^\lambda \times \{0, 1\}^\ell \rightarrow \{0, 1\}\}$ be the function family where

$$f((x, t), k) = \begin{cases} 1 & \text{if } t \in [Q] \\ 0 & \text{otherwise.} \end{cases}$$

Here, $(\text{st}_x, Q) \leftarrow \text{zkPCP.Query}(x; \text{PRF}(k, x))$.

See [LQR⁺19, Section 4] for detailed construction and proofs. We present their main theorem below.

Theorem 6.1 (See [LQR⁺19]). *If there exists attribute-based secure function evaluation (ABSFE) with weak message hiding and strong key hiding, semi-malicious zkPCP for NP, and a pseudorandom function, then there exists reusable non-adaptively sound DV-NIZK argument for NP.*

Observe that in the theorem statement of [LQR⁺19], the ABSFE and the zero-knowledge PCP satisfy computational weak message hiding and computational zero-knowledge respectively. The final conclusion also yields a DV-NIZK argument with computational ZK. However, the result can be easily extended to achieve DV-NIZK with statistical zero-knowledge if the aforementioned properties are made statistical. In particular, we prove the following theorem:

Theorem 6.2. *Let $\lambda \in \mathbb{N}$ be the security parameter. For all $n, \rho = \text{poly}(\lambda)$, if there exists*

- *a semi-malicious statistical zero-knowledge PCP $\text{zkPCP} = (\text{zkPCP.Prove}, \text{zkPCP.Query}, \text{zkPCP.Ver})$ for NP. Let ρ be a bound on the number of random bits needed for zkPCP.Query ,*
- *a pseudorandom function $\text{PRF} : \{0, 1\}^\lambda \times \{0, 1\}^n \rightarrow \{0, 1\}^\rho$, and*
- *an attribute-based secure function evaluation scheme $\text{ABSFE} = (\text{ABSFE.Setup}, \text{ABSFE.Keygen}, \text{ABSFE.Enc}, \text{ABSFE.Dec})$ with weak statistical message hiding and strong key hiding for function F defined above,*

then there exists reusable non-adaptively sound DV-NIZK argument with statistical zero-knowledge for NP.

To prove this theorem, let us first recall the construction of DV-NIZK from ABSFE from [LQR⁺19] using these building blocks.

- $\text{DV-NIZK.Setup}(1^\lambda)$: Output $\text{pp} \leftarrow \text{ABSFE.Setup}(1^\lambda)$.
- $\text{DV-NIZK.Keygen}(\text{pp})$: Sample $k \xleftarrow{\$} \{0, 1\}^\lambda$, and $(\text{pk}', \text{sk}') \leftarrow \text{ABSFE.Keygen}(\text{pp}, k)$. Output the public key $\text{pk} = \text{pk}'$, and the secret verification key $\text{sk} = (k, \text{sk}')$.
- $\text{DV-NIZK.Prove}(\text{pp}, \text{pk}, x, w)$: Construct a PCP $\pi^{(\text{PCP})} \leftarrow \text{zkPCP.Prove}(x, w)$. Then, for each $i \in [m]$, compute ciphertexts $\text{ct}_i \leftarrow \text{ABSFE.Enc}(\text{pp}, \text{pk}, (x, i), \pi_i^{(\text{PCP})})$, and finally, output the proof $\pi = (\text{ct}_1, \dots, \text{ct}_m)$.
- $\text{DV-NIZK.Ver}(\text{pp}, \text{sk}, x, \pi)$: On input the verification key $\text{sk} = (k, \text{sk}')$, a statement $x \in \{0, 1\}^n$ and a proof $\pi = (\text{ct}_1, \dots, \text{ct}_m)$, compute $(\text{st}_x, q_1, \dots, q_\ell) \leftarrow \text{zkPCP.Query}(x; \text{PRF}(k, x))$. For each $j \in [\ell]$, compute $s_j \leftarrow \text{ABSFE.Dec}(\text{pp}, \text{sk}, (x, q_j), \text{ct}_{q_j})$, and finally, output $\text{zkPCP.Ver}(\text{st}_x, s_1, \dots, s_\ell)$.

We only prove the statistical zero-knowledge and skip the proof for completeness and soundness as they are identical to that presented in [LQR⁺19]. Formally we show the following lemma:

Lemma 6.3 (Statistical Zero-knowledge). *If ABSFE satisfies weak statistical message-hiding and zkPCP satisfies semi-malicious statistical zero-knowledge, then DV-NIZK from the construction above satisfies statistical zero-knowledge.*

Proof. First we define the simulators:

- S_1 : Sample $\text{pp} \leftarrow \text{ABSFE.Setup}(1^\lambda)$, $k \xleftarrow{\$} \{0,1\}^\lambda$, and $(\text{pk}', \text{sk}') \leftarrow \text{ABSFE.Keygen}(\text{pp}, k)$. Output pp , the public key $\text{pk} = \text{pk}'$, and the secret verification key $\text{sk} = (k, \text{sk}')$. Let $\text{st}_S = (k, \text{sk}')$
- S_2 : Using st_S compute $(\text{st}_x, q_1, \dots, q_\ell) \leftarrow \text{zkPCP.Query}(x; \text{PRF}(k, x))$ and $(\tilde{\pi}_1^{(PCP)}, \dots, \tilde{\pi}_\ell^{(PCP)}) \leftarrow S^{(PCP)}(x, q_1, \dots, q_\ell)$. For all $i \in [m]$, the simulator takes $z_i \leftarrow \tilde{\pi}_j^{(PCP)}$ if $i = q_j$ for some $j \in [\ell]$, else $z_i \leftarrow 0$. Finally, for every $i \in [m]$, it computes $\text{ct}_i \leftarrow \text{ABSFE.Enc}(\text{pp}, \text{pk}, (x, i), z_i)$ and output the simulated proof $(\text{ct}_1, \dots, \text{ct}_m)$.

We can complete the proof via a sequence of hybrids:

- Hybrid 0: This is the real world where the pp and proof $\pi := (\text{ct}_1, \dots, \text{ct}_m)$ are correctly generated by the proving algorithm.
- Hybrid 1: On every (x, w) such that $\mathcal{R}(x, w) = 1$, the challenger computes $\pi^{(PCP)} \leftarrow \text{zkPCP.Prove}(x, w)$. It then computes $(\text{st}_x, q_1, \dots, q_\ell) \leftarrow \text{zkPCP.Query}(x; \text{PRF}(k, x))$. Note that this is possible because the challenger knows the key k . Finally, for all $i \in [m]$, let $z_i \leftarrow \pi_j^{(PCP)}$ if $i = q_j$ for some $j \in [\ell]$, else $z_i \leftarrow 0$. Compute $\text{ct}_i \leftarrow \text{ABSFE.Enc}(\text{pp}, \text{pk}, (x, i), z_i)$ and output the simulated proof $(\text{ct}_1, \dots, \text{ct}_m)$.

Observe that Hybrid 1 is statistically indistinguishable from Hybrid 0 if ABSFE satisfies statistical weak message-hiding. Note that for all indices which are generated by the zkPCP.Query algorithm, the ciphertexts in Hybrid 0 is identical to Hybrid 1. Formally we have the following lemma.

Lemma 6.4. *Assuming statistical message-hiding of ABSFE, the output of Hybrid 0 and Hybrid 1 are statistically indistinguishable.*

Proof. The statistical indistinguishability of Hybrid 0 and Hybrid 1 follow from the statistical ciphertext indistinguishability (message-hiding) of ABSFE. Namely, the only difference between Hybrid 0 and Hybrid 1 is that in Hybrid 1, instead of computing the ciphertexts as $\text{ct}_i \leftarrow \text{ABSFE.Enc}(\text{pp}, \text{pk}, (x, i), \pi_i^{(PCP)})$ when $F((x, i), k) = 0$, the challenger instead computes $\text{ct}_i \leftarrow \text{ABSFE.Enc}(\text{pp}, \text{pk}, (x, i), 0)$. In the formal proof, the statistical message-hiding adversary receives pp , pk and the PRF key k . It then sends $\text{pp}, \text{pk}, \text{sk}$ to the zero-knowledge adversary. Afterwards, the ciphertext indistinguishability adversary uses its knowledge of k to issue challenge queries on all $i \in [m]$ such that $F((x, i), k) = 0$. In particular, we assume that the adversary is able to make multiple challenge queries, all under the same pk (see Remark 3.14). If the zero-knowledge adversary can distinguish between these two worlds, then the message-hiding adversary can also distinguish between ciphertext of $\pi_i^{(PCP)}$ and that of 0. $\square$

- Hybrid 2: This is same as the previous hybrid except the bits of the PCP are first simulated by $S^{(PCP)}$, and then follow all steps of Hybrid 1.

This hybrid is statistically close to the previous hybrid if the zkPCP satisfies statistical zero-knowledge.

Lemma 6.5. *Assuming statistical zero-knowledge of zkPCP, the output of Hybrid 1 and Hybrid 2 are statistically indistinguishable.*

Proof. The statistical indistinguishability of Hybrid 1 and Hybrid 2 follows from the statistical zero-knowledge of zkPCP. Namely, the only difference between Hybrid 1 and Hybrid 2 is that in Hybrid 2, instead of computing the ciphertexts as $\pi_i^{(PCP)}$ using the zero-knowledge proving algorithm when $F((x, i), k) = 0$, the challenger instead computes $\bar{\pi}_i^{(PCP)}$ using the zero-knowledge simulator. In the formal reduction, the statistical zkPCP adversary receives either $\pi^{(PCP)}$ or $\bar{\pi}^{(PCP)}$ along with x . It then samples k , runs `ABSFE.Setup`, `ABSFE.Keygen` to generate the DV-NIZK keys. It then simulates the DV-NIZK prover where for all $i \in [m]$ such that $F((x, i), k) = 1$, it uses the PCP proof that it receives. If it received $\pi^{(PCP)}$, then the DV-NIZK zero-knowledge adversary is interacting with Hybrid 1, else it is interacting with Hybrid 2. If the DV-NIZK zero-knowledge adversary can distinguish between these two worlds, then the zkPCP adversary can also distinguish $\pi^{(PCP)}$ from $\bar{\pi}^{(PCP)}$ with the same advantage. Therefore, statistical zero-knowledge of zkPCP implies that Hybrids 1 and 2 are statistically indistinguishable. $\square$

$\square$

Corollary 6.6 (DV-NIZK for NP from Length Parameterized Lossy Trapdoor Functions and PRFs computable in PBP). *If there exists length parameterized lossy trapdoor functions and PRFs computable in PBP, then there exists DV-NIZK for NP with statistical zero-knowledge and non-adaptive computational soundness.*

Adaptive soundness using complexity leveraging. As noted in Remark 2.6 in [LQR⁺19], one can generically boost any non-adaptively sound DV-NIZK to an adaptively sound one using standard Complexity Leveraging at the cost of suffering a super-polynomial security loss in the reduction. [BB04].

Corollary 6.7 (Adaptively sound DV-NIZK for NP from Length Parameterized Lossy Trapdoor Functions and PRFs computable in PBP). *If there exists sub-exponentially secure length parameterized lossy trapdoor functions and sub-exponentially secure PRFs computable in PBP, then there exists DV-NIZK for NP with statistical zero-knowledge and adaptive computational soundness.*

Acknowledgements

We are very grateful to Hoeteck Wee for letting us use his observation that the st-connectivity secret sharing scheme has a randomness recovery property.

7 References

- [ABI⁺23] Benny Applebaum, Amos Beimel, Yuval Ishai, Eyal Kushilevitz, Tianren Liu, and Vinod Vaikuntanathan. Succinct computational secret sharing. In *Proceedings of the 55th Annual ACM Symposium on Theory of Computing*, pages 1553–1566, 2023.
- [AKG05] Scott Aaronson, Greg Kuperberg, and Christopher Granade. The complexity zoo, 2005.
- [AS98] Sanjeev Arora and Shmuel Safra. Probabilistic checking of proofs: A new characterization of np. *Journal of the ACM (JACM)*, 45(1):70–122, 1998.
- [Bab85] László Babai. Trading group theory for randomness. In *Proceedings of the 17th Annual ACM Symposium on Theory of Computing, STOC*, pages 421–429, 1985.
- [BB04] Dan Boneh and Xavier Boyen. Efficient selective-id secure identity-based encryption without random oracles. In *Advances in Cryptology-EUROCRYPT 2004: International Conference on the Theory and Applications of Cryptographic Techniques, Interlaken, Switzerland, May 2-6, 2004. Proceedings 23*, pages 223–238. Springer, 2004.
- [Bei96] Amos Beimel. Secure schemes for secret sharing and key distribution. *PhD thesis, Israel Institute of Technology, Technion*, 1996.
- [Bei11] Amos Beimel. Secret-sharing schemes: A survey. In *International conference on coding and cryptography*, pages 11–46. Springer, 2011.
- [BFM88] Manuel Blum, Paul Feldman, and Silvio Micali. Non-interactive zero-knowledge and its applications (extended abstract). In *STOC*, pages 103–112. ACM, 1988.
- [BHR12] Mihir Bellare, Viet Tung Hoang, and Phillip Rogaway. Foundations of garbled circuits. In Ting Yu, George Danezis, and Virgil D. Gligor, editors, *ACM CCS 2012: 19th Conference on Computer and Communications Security*, pages 784–796, Raleigh, NC, USA, October 16–18, 2012. ACM Press.
- [BI05] Amos Beimel and Yuval Ishai. On the power of nonlinear secret-sharing. *SIAM Journal on Discrete Mathematics*, 19(1):258–280, 2005.
- [Blu86] Manuel Blum. How to prove a theorem so no one else can claim it. In *Proceedings of the International Congress of Mathematicians*, volume 1, page 2. Citeseer, 1986.
- [BR89] Josh Benaloh and Steven Rudich. Private communication, 1989. Private communication.
- [CCH⁺19] Ran Canetti, Yilei Chen, Justin Holmgren, Alex Lombardi, Guy N. Rothblum, Ron D. Rothblum, and Daniel Wichs. Fiat-shamir: from practice to theory. In *STOC*, pages 1082–1090. ACM, 2019.
- [CCRR18] Ran Canetti, Yilei Chen, Leonid Reyzin, and Ron D. Rothblum. Fiat-shamir and correlation intractability from strong kdm-secure encryption. In *EUROCRYPT (1)*, volume 10820 of *Lecture Notes in Computer Science*, pages 91–122. Springer, 2018.
- [CGH04] Ran Canetti, Oded Goldreich, and Shai Halevi. The random oracle methodology, revisited. *J. ACM*, 51(4):557–594, 2004.

- [CGJ⁺23] Arka Rai Choudhuri, Sanjam Garg, Abhishek Jain, Zhengzhong Jin, and Jiaheng Zhang. Correlation intractability and snargs from sub-exponential DDH. In *CRYPTO (4)*, volume 14084 of *Lecture Notes in Computer Science*, pages 635–668. Springer, 2023.
- [CH19] Geoffroy Couteau and Dennis Hofheinz. Designated-verifier pseudorandom generators, and their applications. In *EUROCRYPT (2)*, volume 11477 of *Lecture Notes in Computer Science*, pages 562–592. Springer, 2019.
- [CHK07] Ran Canetti, Shai Halevi, and Jonathan Katz. A forward-secure public-key encryption scheme. *J. Cryptol.*, 20(3):265–294, 2007.
- [Cob66] Alan Cobham. The recognition problem for the set of perfect squares. In *7th Annual Symposium on Switching and Automata Theory (swat 1966)*, pages 78–87. IEEE, 1966.
- [DJJ24] Quang Dao, Aayush Jain, and Zhengzhong Jin. Non-interactive zero-knowledge from LPN and MQ. In *CRYPTO (9)*, volume 14928 of *Lecture Notes in Computer Science*, pages 321–360. Springer, 2024.
- [DKW21] Pratish Datta, Ilan Komargodski, and Brent Waters. Decentralized multi-authority ABE for DNFs from LWE. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology – EUROCRYPT 2021, Part I*, volume 12696 of *Lecture Notes in Computer Science*, pages 177–209, Zagreb, Croatia, October 17–21, 2021. Springer, Heidelberg, Germany.
- [DRS04] Yevgeniy Dodis, Leonid Reyzin, and Adam Smith. Fuzzy extractors: How to generate strong keys from biometrics and other noisy data. In Christian Cachin and Jan Camenisch, editors, *Advances in Cryptology – EUROCRYPT 2004*, volume 3027 of *Lecture Notes in Computer Science*, pages 523–540, Interlaken, Switzerland, May 2–6, 2004. Springer, Heidelberg, Germany.
- [FLS90] Uriel Feige, Dror Lapidot, and Adi Shamir. Multiple non-interactive zero knowledge proofs based on a single random string (extended abstract). In *FOCS*, pages 308–317. IEEE Computer Society, 1990.
- [FS86] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *CRYPTO*, volume 263 of *Lecture Notes in Computer Science*, pages 186–194. Springer, 1986.
- [GHMO21] Sanjam Garg, Mohammad Hajiabadi, Giulio Malavolta, and Rafail Ostrovsky. How to build a trapdoor function from an encryption scheme. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2021, Part III*, volume 13092 of *Lecture Notes in Computer Science*, pages 220–249, Singapore, December 6–10, 2021. Springer, Heidelberg, Germany.
- [GIKM98] Yael Gertner, Yuval Ishai, Eyal Kushilevitz, and Tal Malkin. Protecting data privacy in private information retrieval schemes. In *30th Annual ACM Symposium on Theory of Computing*, pages 151–160, Dallas, TX, USA, May 23–26, 1998. ACM Press.
- [GK03] Shafi Goldwasser and Yael Tauman Kalai. On the (in)security of the fiat-shamir paradigm. In *FOCS*, pages 102–113. IEEE Computer Society, 2003.

- [GMR89] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof systems. *SIAM J. Comput.*, 18(1):186–208, 1989.
- [GMW87] Oded Goldreich, Silvio Micali, and Avi Wigderson. How to prove all NP-statements in zero-knowledge, and a methodology of cryptographic protocol design. In Andrew M. Odlyzko, editor, *Advances in Cryptology – CRYPTO’86*, volume 263 of *Lecture Notes in Computer Science*, pages 171–185, Santa Barbara, CA, USA, August 1987. Springer, Heidelberg, Germany.
- [Gol11] Oded Goldreich. Basing non-interactive zero-knowledge on (enhanced) trapdoor permutations: The state of the art. In *Studies in Complexity and Cryptography*, volume 6650 of *Lecture Notes in Computer Science*, pages 406–421. Springer, 2011.
- [GOS12] Jens Groth, Rafail Ostrovsky, and Amit Sahai. New techniques for noninteractive zero-knowledge. *J. ACM*, 59(3):11:1–11:35, 2012.
- [GPSW06] Vipul Goyal, Omkant Pandey, Amit Sahai, and Brent Waters. Attribute-based encryption for fine-grained access control of encrypted data. In Ari Juels, Rebecca N. Wright, and Sabrina De Capitani di Vimercati, editors, *ACM CCS 2006: 13th Conference on Computer and Communications Security*, pages 89–98, Alexandria, Virginia, USA, October 30 – November 3, 2006. ACM Press. Available as Cryptology ePrint Archive Report 2006/309.
- [GVW12] Sergey Gorbunov, Vinod Vaikuntanathan, and Hoeteck Wee. Functional encryption with bounded collusions via multi-party computation. In Reihaneh Safavi-Naini and Ran Canetti, editors, *Advances in Cryptology – CRYPTO 2012*, volume 7417 of *Lecture Notes in Computer Science*, pages 162–179, Santa Barbara, CA, USA, August 19–23, 2012. Springer, Heidelberg, Germany.
- [HKV24a] Mohammad Hajiabadi, Shahram Khazaei, and Behzad Vahdani. Randomness recoverable secret sharing schemes. *J. Cryptol.*, 37(4):34, 2024.
- [HKV24b] Mohammad Hajiabadi, Shahram Khazaei, and Behzad Vahdani. Randomness recoverable secret sharing schemes. *Journal of Cryptology*, 37(4):34, 2024.
- [HKW20] Susan Hohenberger, Venkata Koppula, and Brent Waters. Chosen ciphertext security from injective trapdoor functions. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology – CRYPTO 2020, Part I*, volume 12170 of *Lecture Notes in Computer Science*, pages 836–866, Santa Barbara, CA, USA, August 17–21, 2020. Springer, Heidelberg, Germany.
- [IKOS07] Yuval Ishai, Eyal Kushilevitz, Rafail Ostrovsky, and Amit Sahai. Zero-knowledge from secure multiparty computation. In David S. Johnson and Uriel Feige, editors, *39th Annual ACM Symposium on Theory of Computing*, pages 21–30, San Diego, CA, USA, June 11–13, 2007. ACM Press.
- [IMS12] Yuval Ishai, Mohammad Mahmoody, and Amit Sahai. On efficient zero-knowledge PCPs. In Ronald Cramer, editor, *TCC 2012: 9th Theory of Cryptography Conference*, volume 7194 of *Lecture Notes in Computer Science*, pages 151–168, Taormina, Sicily, Italy, March 19–21, 2012. Springer, Heidelberg, Germany.

- [KMT19] Fuyuki Kitagawa, Takahiro Matsuda, and Keisuke Tanaka. CCA security and trapdoor functions via key-dependent-message security. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019, Part III*, volume 11694 of *Lecture Notes in Computer Science*, pages 33–64, Santa Barbara, CA, USA, August 18–22, 2019. Springer, Heidelberg, Germany.
- [KNYY21] Shuichi Katsumata, Ryo Nishimaki, Shota Yamada, and Takashi Yamakawa. Compact designated verifier nizks from the CDH assumption without pairings. *J. Cryptol.*, 34(4):42, 2021.
- [KPT97] Joe Kilian, Erez Petrank, and Gábor Tardos. Probabilistically checkable proofs with zero knowledge. In *29th Annual ACM Symposium on Theory of Computing*, pages 496–505, El Paso, TX, USA, May 4–6, 1997. ACM Press.
- [KW93] Mauricio Karchmer and Avi Wigderson. On span programs. In *[1993] Proceedings of the Eighth Annual Structure in Complexity Theory Conference*, pages 102–111. IEEE, 1993.
- [KW19] Venkata Koppula and Brent Waters. Realizing chosen ciphertext security generically in attribute-based encryption and predicate encryption. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019, Part II*, volume 11693 of *Lecture Notes in Computer Science*, pages 671–700, Santa Barbara, CA, USA, August 18–22, 2019. Springer, Heidelberg, Germany.
- [LPWW20] Benoît Libert, Alain Passelègue, Hoeteck Wee, and David J. Wu. New constructions of statistical NIZKs: Dual-mode DV-NIZKs and more. In Anne Canteaut and Yuval Ishai, editors, *Advances in Cryptology – EUROCRYPT 2020, Part III*, volume 12107 of *Lecture Notes in Computer Science*, pages 410–441, Zagreb, Croatia, May 10–14, 2020. Springer, Heidelberg, Germany.
- [LQR⁺19] Alex Lombardi, Willy Quach, Ron D. Rothblum, Daniel Wichs, and David J. Wu. New constructions of reusable designated-verifier NIZKs. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019, Part III*, volume 11694 of *Lecture Notes in Computer Science*, pages 670–700, Santa Barbara, CA, USA, August 18–22, 2019. Springer, Heidelberg, Germany.
- [LW11] Allison Lewko and Brent Waters. Decentralizing attribute-based encryption. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 568–588. Springer, 2011.
- [NR04] Moni Naor and Omer Reingold. Number-theoretic constructions of efficient pseudo-random functions. *Journal of the ACM (JACM)*, 51(2):231–262, 2004.
- [OW93] Rafail Ostrovsky and Avi Wigderson. One-way functions are essential for non-trivial zero-knowledge. In *ISTCS*, pages 3–17. IEEE Computer Society, 1993.
- [PS19] Chris Peikert and Sina Shiehian. Noninteractive zero knowledge for NP from (plain) learning with errors. In *CRYPTO (1)*, volume 11692 of *Lecture Notes in Computer Science*, pages 89–114. Springer, 2019.
- [PSV06] Rafael Pass, Abhi Shelat, and Vinod Vaikuntanathan. Construction of a non-malleable encryption scheme from any semantically secure one. In *CRYPTO*, volume 4117 of *Lecture Notes in Computer Science*, pages 271–289. Springer, 2006.

- [PW08] Chris Peikert and Brent Waters. Lossy trapdoor functions and their applications. In Richard E. Ladner and Cynthia Dwork, editors, *40th Annual ACM Symposium on Theory of Computing*, pages 187–196, Victoria, BC, Canada, May 17–20, 2008. ACM Press.
- [PW11] Chris Peikert and Brent Waters. Lossy trapdoor functions and their applications. *SIAM J. Comput.*, 40(6):1803–1844, 2011.
- [QRW19] Willy Quach, Ron D. Rothblum, and Daniel Wichs. Reusable designated-verifier nizks for all NP from CDH. In *EUROCRYPT (2)*, volume 11477 of *Lecture Notes in Computer Science*, pages 593–621. Springer, 2019.
- [QWW21] Willy Quach, Brent Waters, and Daniel Wichs. Targeted lossy functions and applications. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology – CRYPTO 2021, Part IV*, volume 12828 of *Lecture Notes in Computer Science*, pages 424–453, Virtual Event, August 16–20, 2021. Springer, Heidelberg, Germany.
- [Rei08] Omer Reingold. Undirected connectivity in log-space. *Journal of the ACM (JACM)*, 55(4):1–24, 2008.
- [SCO⁺01] Alfredo De Santis, Giovanni Di Crescenzo, Rafail Ostrovsky, Giuseppe Persiano, and Amit Sahai. Robust non-interactive zero knowledge. In *CRYPTO*, volume 2139 of *Lecture Notes in Computer Science*, pages 566–598. Springer, 2001.
- [SMP87] Alfredo De Santis, Silvio Micali, and Giuseppe Persiano. Non-interactive zero-knowledge proof systems. In *CRYPTO*, volume 293 of *Lecture Notes in Computer Science*, pages 52–72. Springer, 1987.
- [SS10] Amit Sahai and Hakan Seyalioglu. Worry-free encryption: functional encryption with public keys. In Ehab Al-Shaer, Angelos D. Keromytis, and Vitaly Shmatikov, editors, *ACM CCS 2010: 17th Conference on Computer and Communications Security*, pages 463–472, Chicago, Illinois, USA, October 4–8, 2010. ACM Press.
- [SW05] Amit Sahai and Brent R. Waters. Fuzzy identity-based encryption. In Ronald Cramer, editor, *Advances in Cryptology – EUROCRYPT 2005*, volume 3494 of *Lecture Notes in Computer Science*, pages 457–473, Aarhus, Denmark, May 22–26, 2005. Springer, Heidelberg, Germany.
- [VV15] Vinod Vaikuntanathan and Prashant Nalini Vasudevan. Secret sharing and statistical zero knowledge. In Tetsu Iwata and Jung Hee Cheon, editors, *Advances in Cryptology – ASIACRYPT 2015, Part I*, volume 9452 of *Lecture Notes in Computer Science*, pages 656–680, Auckland, New Zealand, November 30 – December 3, 2015. Springer, Heidelberg, Germany.
- [Wat24] Brent Waters. A new approach for non-interactive zero-knowledge from learning with errors. In *STOC*, pages 399–410. ACM, 2024.
- [Wik24] Wikipedia. L (complexity) — Wikipedia, the free encyclopedia. [http://en.wikipedia.org/w/index.php?title=L%20\(complexity\)&oldid=1189612885](http://en.wikipedia.org/w/index.php?title=L%20(complexity)&oldid=1189612885), 2024. [Online; accessed 22-September-2024].

- [WZ24] Brent Waters and Mark Zhandry. Adaptive security in snargs via io and lossy functions. In *CRYPTO (10)*, volume 14929 of *Lecture Notes in Computer Science*, pages 72–104. Springer, 2024.
- [Yao] Andrew Chi-Chih Yao. How to generate and exchange secrets extended abstract. In *27th FOCS*, pages 162–167.
- [Yao82] Andrew C Yao. Protocols for secure computations. In *23rd annual symposium on foundations of computer science (sfcs 1982)*, pages 160–164. IEEE, 1982.

A Computational Monotone Secret Sharing Scheme with Randomness Recovery

A.1 Preliminaries

A.1.1 Garbling Scheme

Garbling scheme for circuits are a useful tool introduced by Yao [Yao82, Yao] and formalized by [BHR12].

A garbling scheme is a tuple of three algorithms (Garble.Ckt , Garble.Inp , Garble.Eval).

- $(GC, \text{sk}) \leftarrow \text{Garble.Ckt}(1^\lambda, C)$ takes as input security parameter 1^λ and the description of a circuit C (on say n inputs, with 1 output) and outputs (GC, sk) , where GC is the garbled circuit and sk is a key that may later be used to garble an input.
- $gx \leftarrow \text{Garble.Inp}(\text{sk}, x)$ takes as input a key sk (that is supposed to have been previously generated by Garble.Ckt) and an n -bit string x and outputs gx , which is the garbled input corresponding to x under sk .
- $\text{out} \leftarrow \text{Garble.Eval}(GC, gx)$ takes as input a garbled circuit GC and a garbled input gx and outputs out .

We require the following of any Garbling Scheme:

- **Correctness:** $\forall (GC, gx)$ such that $(GC, \text{sk}) \leftarrow \text{Garble.Ckt}(1^\lambda, C)$ and $gx \leftarrow \text{Garble.Inp}(x, \text{sk})$, then $\text{Garble.Eval}(GC, gx) = C(x)$.
- **Efficiency:** The running time of Garble.Ckt , Garble.Inp , Garble.Eval must be $\text{poly}(\lambda, |C|)$, $\text{poly}(\lambda, |x|)$, and $\text{poly}(\lambda, |x|, |C|)$.
- **Security:** For all $\lambda \in \mathbb{N}$, $\exists$ PPT simulator Sim such that $\forall (C, x)$, all distinguishers $\mathcal{D}$, there exists a negligible function $\text{negl}(\cdot)$ such that

$$\left| \Pr[\mathcal{D}(\text{Sim}(1^\lambda, C(x), 1^{|C|}, 1^{|x|})) = 1] - \Pr[\mathcal{D}(GC, gx) = 1 : (GC, \text{sk}) \leftarrow \text{Garble.Ckt}(1^\lambda, C), gx \leftarrow \text{Garble.Inp}(\text{sk}, x)] \right| \leq \text{negl}(\lambda).$$

Theorem A.1. [Yao82, Yao] Assuming one-way functions, there exists efficient garbling scheme for circuits.

A.1.2 Targeted Lossy Pseudorandom Generators

We introduce the notion of a targeted lossy PRG which is closely related to Targeted Lossy Functions introduced in [QWW21].

Definition A.2 (Targeted lossy pseudorandom generators). *A targeted lossy pseudorandom generator family with input length n , output length $m > n$ and lossiness parameter k is a tuple $\text{TLPRG} = (\text{TLPRG.Setup}, \text{TLPRG.AltSetup}, \text{TLPRG.Eval})$ such that*

- $\text{fk} \leftarrow \text{TLPRG.Setup}(1^\lambda)$: A randomized algorithm that takes as input the security parameter λ , and output function key fk .
- $\text{fk} \leftarrow \text{TLPRG.AltSetup}(1^\lambda, x^*)$: A randomized algorithm that takes as input the security parameter λ , a target point x^* , and output function key fk .
- $y \leftarrow \text{TLPRG.Eval}(\text{fk}, x)$: A deterministic function that takes as input the key fk , input x , and outputs y .

A targeted lossy PRG satisfies the following properties:

- **Pseudorandomness:** For all security parameter $\lambda \in \mathbb{N}$, input length $n = \text{poly}(\lambda)$, output length $m = \text{poly}(\lambda)$, and for all PPT adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that

$$\left| \Pr[\mathcal{A}(\text{fk}, y) = 1 \mid \text{fk} \leftarrow \text{TLPRG.Setup}(1^\lambda), x \xleftarrow{\$} \{0, 1\}^n, y \leftarrow \text{TLPRG.Eval}(\text{fk}, x)] - \Pr[\mathcal{A}(\text{fk}, y) = 1 \mid \text{fk} \leftarrow \text{TLPRG.Setup}(1^\lambda), y \xleftarrow{\$} \{0, 1\}^m] \right| \leq \text{negl}(\lambda).$$

- **Setup Indistinguishability:** For all security parameter $\lambda \in \mathbb{N}$, for all PPT adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that

$$\left| \Pr[\mathcal{A}(\text{fk}) = 1 \mid \text{fk} \leftarrow \text{TLPRG.Setup}(1^\lambda)] - \Pr[\mathcal{A}(\text{fk}) = 1 \mid \text{fk} \leftarrow \text{TLPRG.AltSetup}(1^\lambda)] \right| \leq \text{negl}(\lambda).$$

- **k -Lossiness:** For all $\lambda \in \mathbb{N}$, for all $x \in \{0, 1\}^n$, and $\text{fk} \leftarrow \text{TLPRG.AltSetup}(1^\lambda, x)$,

$$H_\infty(x \mid \text{fk}, \text{TLPRG.Eval}(\text{fk}, x)) \geq k.$$

We construct Targeted Lossy PRG in Section A.4 assuming one-way functions. At a very high level, the construction concatenates a pseudorandom generator with an appropriate pair-wise independent hash function.

Theorem A.3 (See Section A.4). *Assuming one-way functions, there exists Targeted Lossy Pseudorandom Generators with arbitrary polynomial lossiness.*

A.2 Linear Secret Sharing Scheme with Setup

In this section, we define and construct a linear secret sharing scheme for monotone functions. Our construction requires a setup step which outputs the secret along with all the shares. The secret sharing scheme has perfect reconstruction correctness our previous secret sharing scheme for non-monotone functions, however we guarantee privacy only when the secret and shares are generated by an alternate setup which is computationally indistinguishable from the real setup procedure.

Access Structure ([Bei96]): Let $P = \{P_1, \dots, P_n\}$ be a set of all parties. An access structure Γ is a collection of subset of parties, i.e., $\Gamma \subset \mathcal{P}(\{P_1, \dots, P_n\}) \setminus \{\emptyset\}$ (where for any set S , $\mathcal{P}(S)$ denotes the power set of S). Any set $A \subseteq P$ is called *authorized* iff $A \in \Gamma$.

Informally, a secret shared among parties $P = \{P_1, \dots, P_n\}$ can be reconstructed by a subset $A \subseteq P$ iff A is *authorized*. An access structure Γ is monotone if $A \in \Gamma$ and $A \subseteq B$ implies $B \in \Gamma$.

We now define the notion of a Linear Secret Sharing scheme (LSSS) where a matrix describes the linear mapping of the scheme. This matrix defines the access structure of the scheme.

Definition A.4 (Linear secret sharing schemes (adapted from [Bei96])). *A secret sharing scheme over $\mathbb{F}_p$ is said to be linear if*

1. *The shares of each party form a vector over $\mathbb{F}_p$.*
2. *There exists a matrix M with ℓ rows and d columns called the shared generating matrix.*
3. *Let P be the set of all parties. There exists an efficiently computable map $\rho : [\ell] \rightarrow P$ where $\rho(i)$ maps the i^{th} row in M to party $\rho(i)$.*
4. *Sharing a secret $s \in \mathbb{F}_p$ involves sampling $v_2, \dots, v_d$ at random over $\mathbb{F}_p$. Let $v := (s, v_2, \dots, v_d)$. Then, Mv forms the vector of ℓ shares where $a_i := (Mv)_i$ is given to party $\rho(i)$.*
5. *We use the convention that the target vector for any LSSS is $\mathbf{e}_1^\top$.*
6. *Say the LSSS is for an access function $f : \{0, 1\}^n \rightarrow \{0, 1\}$, and has secret generating matrix M . For an input $x \in \{0, 1\}^n$, if $f(x) = 1$, then $S = \{P_i : x_i = 1\}$ is an authorized set of parties. Let $I \subset [\ell]$ be defined as $I = \{i : \rho(i) \in S\}$. Every LSSS with target vector $\mathbf{e}_1^\top$ has the following linear reconstruction property: It must be the case that $\mathbf{e}_1^\top$ belongs to the span of M_I (M restricted to rows I). Thus, there exists efficiently computable values $\omega_i \in \mathbb{F}_p, i \in I$ such that $\sum_{i \in I} \omega_i M_i = \mathbf{e}_1^\top$. In other words, for every set of authorized parties S , $\sum_{i \in I} \omega_i a_i = s$.*
7. *Moreover, for any unauthorized set X , i.e., $f(x) = 0$ where $x := (x_1, \dots, x_\ell)$ and $x_i = 1$ iff $P_i \in X$. The target vector $\mathbf{e}_1$ is not in the span of $\{M_j\}_{j \in T}$ where $T = \{i : \rho(i) \in X\}$.*

Note. M is alternately also referred to as the policy matrix and we say that a set of parties satisfy the policy if $S \in \Gamma$.

We import the a “zero-out” lemma from [DKW21] that will be useful later in our construction. Informally, it says that any access policy M can be efficiently translated into another access policy W where some of the columns of the unauthorized set are zeroed-out, yet the distribution of shares generated remain identical.

Lemma A.5 (Claim 4.1, Lemma 4.2, in [DKW21]). *Let $M \in \mathbb{F}_p^{\ell \times d}$ be an access policy matrix. Let $X \subseteq [\ell]$ be a non-authorized subset of row indices of M . Let $c \in \mathbb{N}$ be the dimension of the subspace spanned by the rows indexed by X . Then, there exists a PPT algorithm $\text{Compatible}(M, X)$ which outputs $W \in \mathbb{F}_p^{\ell \times d}$ such that:*

- W is a valid access policy matrix.
- W is such that $W_{i,j} = 0$ for all $(i, j) \in X \times [d - c]$.
- The distribution of shares $\{a_j \in \mathbb{F}_p\}_{j \in [\ell]}$ while sharing a secret $s \in \mathbb{F}_q$ generated with matrix M is the same as the distribution of the shares $\{a'_j \in \mathbb{F}_p\}_{j \in [\ell]}$ while sharing the same secret s with matrix W .

Remark A.6. We will use a special case of the above lemma where we only require to construct a *compatible* W such that for a given set of unauthorized rows X , the first column of rows indexed by X in W is 0.

In this work, we define a special purpose Linear Secret Sharing scheme (LSSS) with randomness recovery properties which has two setup modes. Then, we show a black-box construction of such an LSSS using (1) pseudorandom generator, (2) garbled circuit, and (3) pairwise independent hash function. We formally define a Linear Secret Sharing Scheme with its properties and establish some conventions. We now describe the special purpose LSSS with randomness recovery which we eventually use as a building block in the construction of DV-NIZK.

Note on notation. In this section, we abuse notation by sometimes interpreting $(v_1, \dots, v_d)$ as a d -length column vector when it is obvious from context.

Definition A.7 (LSSS with Randomness Recovery). *A secret sharing scheme LSSS over $\mathbb{F}_p$ for a family of policies $\mathcal{M} = \{M \in \mathbb{F}_p^{\ell \times d}\}$ is a tuple $\text{LSSS} = (\text{LSSS.Setup}, \text{LSSS.AltSetup}, \text{LSSS.Reconstruct}, \text{LSSS.Regenerate})$ such that*

- $(s, a_1, \dots, a_\ell) \leftarrow \text{LSSS.Setup}(1^\lambda, M)$: A randomized setup algorithm on input the security parameter λ , share generating matrix $M \in \mathbb{F}_p^{\ell \times d}$ outputs, a secret $s \in \mathbb{F}_p$ and share values $a_1, \dots, a_\ell \in \mathbb{F}_p$.
- $(s, a_1, \dots, a_\ell) \leftarrow \text{LSSS.AltSetup}(1^\lambda, M, T)$: A randomized alternate setup algorithm on input the security parameter λ , share generating matrix $M \in \mathbb{F}_p^{\ell \times d}$, a set of rows $T \subset [\ell]$ and outputs a secret $s \in \mathbb{F}_p$ and values $a_1, \dots, a_\ell \in \mathbb{F}_p$.
- $s \leftarrow \text{LSSS.Reconstruct}(M, (z_1, \dots, z_T))$: A deterministic algorithm that takes as input the matrix $M \in \mathbb{F}_p^{\ell \times d}$, shares $(z_1, \dots, z_T)$, $T \subseteq [\ell]$, and outputs a secret $s \in \mathbb{F}_p$.
- $(a_1, \dots, a_\ell) \leftarrow \text{LSSS.Regenerate}(M, s)$: A deterministic algorithm which on input $M \in \mathbb{F}_p^{\ell \times d}$, and secret $s \in \mathbb{F}_p$, and outputs shares $(a_1, \dots, a_\ell) \in \mathbb{F}_p^\ell$.

An LSSS with randomness recovery satisfies the following properties:

Definition A.8 (Efficiency). *For any matrix $M \in \mathcal{M}$,*

$\text{LSSS.Setup}, \text{LSSS.AltSetup}, \text{LSSS.Reconstruct}$ and LSSS.Regenerate run in time $\text{poly}(\lambda, p, \ell, d)$.

The correctness of a secret sharing scheme for a family class $\mathcal{M}$ as above is captured by the following property.

Definition A.9 (Correctness). For any matrix $M \in \mathcal{M}$,

- *Reconstruction correctness*: for all $S : \{i : M_{i,\cdot}\}$ such that S spans $\mathbf{e}_1 \in \mathbb{F}_p$,

$$\Pr[\text{LSSS.Reconstruct}(M, (\{a_j\}_{j \in S})) = s] = 1,$$

where the probability is over the randomness used for sampling $(s, a_1, \dots, a_\ell) \leftarrow \text{LSSS.Setup}(M)$.

- *Regeneration correctness*:

$$\Pr[\text{LSSS.Regenerate}(M, s) = (a_1, \dots, a_\ell)] = 1,$$

where the probability is over the randomness used for sampling $(s, a_1, \dots, a_\ell) \leftarrow \text{LSSS.Setup}(M)$.

In other words, if the secret s was generated honestly by the share generation algorithm, then the secret s must regenerate exactly the set of shares $(a_1, \dots, a_\ell)$.

Remark A.10. The regeneration correctness property is akin to randomness recovery where one can imagine that the regeneration algorithm first recovers the partial randomness of LSSS.Setup which suffices to regenerate all the shares. Observe that this is not a complete randomness recovery as LSSS.Setup might also use other randomness to generate the secret s during its execution which is not recovered.

Definition A.11 (Computational Privacy). Let λ be the security parameter For any policy matrix M and any subset $T \subset [\ell]$ such that $\{M_{i,\cdot} : i \in T\}$ does not span $\mathbf{e}_1$,

- *Setup Indistinguishability*: for all PPT adversaries $\mathcal{A}$, there exists a negligible function negl such that

$$\left| \Pr[\mathcal{A}(s, a_1, \dots, a_\ell) = 1 \mid (s, a_1, \dots, a_\ell) \leftarrow \text{LSSS.Setup}(1^\lambda, M)] - \Pr[\mathcal{A}(s, a_1, \dots, a_\ell) = 1 \mid (s, a_1, \dots, a_\ell) \leftarrow \text{LSSS.AltSetup}(1^\lambda, M, T)] \right| \leq \text{negl}(\lambda),$$

- *Privacy*. there exists $k = \text{poly}(\lambda)$ such that

$$H_\infty(s \mid \{a_j\}_{j \in T}) \geq k,$$

where $(s, a_1, \dots, a_\ell) \leftarrow \text{LSSS.AltSetup}(1^\lambda, M, T)$.

Lastly, we formalize the soundness property of the **Reconstruct** procedure. In short, it says that every (potentially maliciously generated) set of shares are either *invalid*, or correspond to a set of sharing that satisfies a certain property with respect to the policy matrix (meaning they are valid).

Definition A.12 (Soundness). For all $M \in \mathcal{M}$,

- *Soundness of Reconstruction*. for all (maliciously generated) $(a_1, \dots, a_\ell) \in \mathbb{F}_p^\ell$, if $\nexists (v_1, \dots, v_d) \in \mathbb{F}_p^d \setminus \{0^d\} : M \cdot (v_1, v_2, \dots, v_d) = (a_1, \dots, a_\ell)$, then

$$\text{LSSS.Reconstruct}(M, (a_1, \dots, a_\ell)) = \perp.$$

Moreover, $\forall$ (maliciously generated) $(a_1, \dots, a_\ell) \in \mathbb{F}_p^\ell$ such that $\exists (v_1, \dots, v_d) \in \mathbb{F}_p^d \setminus \{0^d\} : M \cdot (v_1, v_2, \dots, v_d) = (a_1, \dots, a_\ell)$, for all sets $S = \{i : M_{i,\cdot}\}$ that spans $\mathbf{e}_1$, $\text{LSSS.Reconstruct}(M, \{a_i\}_{i \in S})$ outputs the same value that is not $\perp$.

- *Soundness of Regeneration.* for all $s \in \mathbb{F}_p$,
 - either $\text{LSSS.Regenerate}(M, s) = \perp$; or
 - $\exists (v_1, \dots, v_d) \in \mathbb{F}_p^d \setminus \{0^d\} : M \cdot (v_1, v_2, \dots, v_d) = \text{LSSS.Regenerate}(M, s)$

Remark A.13. This property is a slight variation of the regeneration soundness property of SS-RR. For LSSS, as an artifact of the abstraction, it is more convenient to consider the property that the reconstruction procedure outputs shares in the column space of M .

A.3 Construction of LSSS with Randomness Recovery

For this construction, we make black-box use of a Targeted Lossy Pseudorandom Generator as stated in definition A.2. Finally, we will construct these PRGs from one way functions.

Let us begin by defining some functions which we will use later in the construction.

- $\text{Strip} : \mathbb{F}_p^{\ell \times d} \rightarrow \mathbb{F}_p^{\ell \times (d-1)}$, $\text{Strip}(M) = M^{(-1)}$ where $M^{(-1)}$ denotes the matrix M without the first column.
- Function $f_{M, \text{fk}} : \{0, 1\}^n \rightarrow \mathbb{F}_p^\ell$ hardcoded with M and a PRG key,

$f_{M, \text{fk}}(r)$:

1. $\mathbf{v} := (v_2, \dots, v_d) = \text{TLPRG.Eval}(\text{fk}, r)$
2. $M' \leftarrow \text{Strip}(M)$
3. $\forall j \in [\ell], a'_j = M_j \cdot \mathbf{v}$.
4. Output $(a'_1, \dots, a'_\ell)$

- $\text{LSSS.Setup}(1^\lambda, M) :$
 1. $M' = \text{Strip}(M)$
 2. $\mathbf{v} := (v_2, \dots, v_d) \xleftarrow{\$} \mathbb{F}_p^d$
 3. $\forall j \in [\ell], a'_j = M'_j \cdot \mathbf{v}$
 4. $\text{fk} \leftarrow \text{PRG.Setup}(1^\lambda)$
 5. $(GC, (y_1, \dots, y_n)) \leftarrow \text{Garble.Sim}(1^\lambda, 1^{|f_{M, \text{fk}}|}, 1^n, (a'_1, \dots, a'_\ell))$
 6. $s := (GC, y_1, \dots, y_n)$ ⁹
 7. $\forall j \in [\ell], a_j \leftarrow a'_j + M_{j1} \cdot s$
 8. Output $s, (a_1, \dots, a_\ell)$
- $\text{LSSS.AltSetup}(1^\lambda, M, T) :$
 1. $W \leftarrow \text{Compatible}(M, T)$
 2. $r \xleftarrow{\$} \{0, 1\}^n$
 3. $\text{fk} \leftarrow \text{PRG.AltSetup}(1^\lambda, r)$

⁹We have to ensure that s has a valid representation as an element of $\mathbb{F}_p$. As, s is a concatenation of a bunch of field elements, this can be ensured using the standard polynomial representation without loss of generality.

4. $(GC, \text{sk}) \leftarrow \text{Garble.Ckt}(f_{W, \text{fk}})$
 5. $y_1, \dots, y_n \leftarrow \text{Garble.Inp}(\text{sk}, r)$
 6. $(a'_1, \dots, a'_\ell) \leftarrow \text{Garble.Eval}(GC, (y_1, \dots, y_n))$
 7. $s := (GC, y_1, \dots, y_n)$
 8. $\forall j \in [\ell], a_j \leftarrow a'_j + W_{j1} \cdot s$
 9. Output $s, (a_1, \dots, a_\ell)$
- $\text{LSSS.Reconstruct}(M, (z_1, \dots, z_T)) :$
 1. Interpret z_i as a_j for appropriate choice of index $j \in [\ell]$. Let these indices form set $S \subseteq [\ell]$
 2. Compute $\{\omega_j \in \mathbb{F}_p\}_{j \in S}$ such that $\sum_{j \in S} \omega_j M_{j\cdot} = \mathbf{e}_1^\top$
 3. if such a non-0 $\{\omega_j\}_{j \in S}$ does not exist, then output $\perp$
 4. if $T = \ell$ and there exists $\mathbf{x} \in \mathbb{F}_q^d$ such that $M(\mathbf{x}) = (z_1, \dots, z_\ell)$, then continue, else output $\perp$
 5. Output $s := \sum_{j \in S} \omega_j a_j$
 - $\text{LSSS.Regenerate}(M, s) :$
 1. Parse s as $(GC, y_1, \dots, y_n)$
 2. $(a'_1, \dots, a'_\ell) \leftarrow \text{Garble.Eval}(GC, (y_1, \dots, y_n))$
 3. $\forall j \in [\ell], a_j \leftarrow a'_j + M_{j1} \cdot s$
 4. if there exists $\mathbf{x} \in \mathbb{F}_q^d$ such that $M(\mathbf{x}) = (a_1, \dots, a_\ell)$, then continue, else output $\perp$
 5. Else output $(a_1, \dots, a_\ell)$

Reconstruction correctness.

Lemma A.14. For any function $M \in \mathbb{F}_p^{\ell \times d}$, for all $S : \{i : M_{i\cdot}\}$ such that S spans $\mathbf{e}_1$ and secret $s \in \mathbb{F}_p$,

$$\Pr [\text{LSSS.Reconstruct}(F, \{a_j\}_{j \in S}) = s] = 1,$$

where the probability is over the randomness used for sampling $(s, a_1, \dots, a_\ell) \leftarrow \text{LSSS.Setup}(1^\lambda, M)$. Here LSSS.Reconstruct and LSSS.Setup are as presented in the above construction.

Proof. Let $\mathbf{v}' \in \mathbb{F}_p^d$ be the column vector made by concatenating s with $\mathbf{v}$, i.e., $\mathbf{v}'^\top = (s, v_2, \dots, v_d)$. Correctness of the scheme follows due to the following:

$$\begin{aligned} \text{LSSS.Reconstruct}(M, \{a_j\}_{j \in S}) &= \sum_{j \in S} \omega_j a_j = \sum_{j \in S} \omega_j (a'_j + M_{j1} s) \\ &= \sum_{j \in S} \omega_j (M'_{j\cdot} \mathbf{v} + M_{j1} s) = \sum_{j \in S} \omega_j M_{j\cdot} \mathbf{v}' = \mathbf{e}_1^\top \mathbf{v}' = s. \end{aligned}$$

□

Regeneration correctness. It is easy to see that our construction satisfies regeneration correctness due to correctness and simulation security of the underlying garbling scheme.

Lemma A.15. *Assuming perfect correctness and simulation security of garbling scheme, for all matrices $M \in \mathcal{M}$ and for all subset $T \in [\ell]$ such that $\{M_{j,\cdot} : j \in T\}$ spans $\mathbf{e}_1$,*

$$\Pr[\text{LSSS.Regenerate}(M, (a_j)_{j \in T}) = (a_1, \dots, a_\ell)] = 1,$$

where the probability is over the randomness used for sampling $(s, a_1, \dots, a_\ell) \leftarrow \text{LSSS.Setup}(M)$

Proof. Observe that any set of rows of M that spans $\mathbf{e}_1$ will always generate the same secret s . This is because $M \cdot (s, v_2, \dots, v_d) = (a_1, \dots, a_\ell)$ and thus for all sets S containing rows of M , $\sum_{j \in S} \omega_j a_j = \mathbf{e}_1^\top (s, v_2, \dots, v_d) = s$. By construction $(a'_1, \dots, a'_\ell)$, and thus $(a_1, \dots, a_\ell)$ are generated deterministically, hence for all inputs $(z_1, \dots, z_T)$ honestly generated using Setup , the output of LSSS.Regenerate is identical. Moreover, for honestly generated shares by LSSS.Setup , Step 9 in the regeneration procedure clearly does not output $\perp$. Thus, it suffices to show that the output is indeed a correctly distributed, i.e., generated by LSSS.Setup .

Recall that s , i.e., $(y_1, \dots, y_n)$ is honestly generated, and therefore, the perfect privacy of garbled circuit also implies that $\text{Garble.Eval}(\text{Garble.Sim}(1^\lambda, 1^{|f_{M, \text{fk}}|}, 1^n, (a'_1, \dots, a'_\ell))) = (a'_1, \dots, a'_\ell)$. Therefore it must be that

$$\Pr[\text{LSSS.Regenerate}(M, (a_j)_{j \in T}) = (a_1, \dots, a_\ell)] = 1,$$

where the probability is over the randomness used for sampling $(s, a_1, \dots, a_\ell) \leftarrow \text{LSSS.Setup}(M)$. $\square$

Setup Indistinguishability. Let λ be the security parameter. For any policy matrix M and any subset $T \subset [\ell]$ such that $\{M_{i,\cdot} : i \in T\}$ does not span $\mathbf{e}_1$, for all PPT adversaries $\mathcal{A}$, there exists a negligible function negl such that

$$\left| \Pr[\mathcal{A}(s, a_1, \dots, a_\ell) = 1 \mid (s, a_1, \dots, a_\ell) \leftarrow \text{LSSS.Setup}(1^\lambda, M)] - \Pr[\mathcal{A}(s, a_1, \dots, a_\ell) = 1 \mid (s, a_1, \dots, a_\ell) \leftarrow \text{LSSS.AltSetup}(1^\lambda, M, T)] \right| \leq \text{negl}(\lambda),$$

Proof. It suffices to show that for all unauthorized set of rows T , LSSS.Setup and LSSS.AltSetup are computationally indistinguishable. We do so using a sequence of hybrids.

• **Hybrid 0:** This is identical to LSSS.Setup .

1. $M' = \text{Strip}(M)$
2. $\mathbf{v} := (v_2, \dots, v_d) \xleftarrow{\$} \mathbb{F}_p^d$
3. $\forall j \in [\ell], a'_j = M'_{j,\cdot} \mathbf{v}$
4. $\text{fk} \leftarrow \text{PRG.Setup}(1^\lambda)$
5. $s := (GC, (y_1, \dots, y_n)) \leftarrow \text{Garble.Sim}(1^\lambda, 1^{|f_{M, \text{fk}}|}, 1^n, (a'_1, \dots, a'_\ell))$
6. $\forall j \in [\ell], a_j \leftarrow a'_j + M_{j1}s$

- **Hybrid 1:** This is identical to LSSS.Setup except we rewrite how the final shares a_j get generated.

1. $M' = \text{Strip}(M)$
2. $\mathbf{v} := (v_2, \dots, v_d) \xleftarrow{\$} \mathbb{F}_p^d$
3. $\forall j \in [\ell], a'_j = M'_j \cdot \mathbf{v}$
4. $\text{fk} \leftarrow \text{PRG.Setup}(1^\lambda)$
5. $s := (GC, (y_1, \dots, y_n)) \leftarrow \text{Garble.Sim}(1^\lambda, 1^{|f_{M, \text{fk}}|}, 1^n, (a'_1, \dots, a'_\ell))$
6. $\mathbf{a} := (a_1, \dots, a_\ell)^\top \leftarrow M \cdot (s, \mathbf{v})^\top$

- **Hybrid 2:** The final shares are generated using the *compatible matrix* W instead of M .

1. $W \leftarrow \text{Compatible}(M)$
2. $M' = \text{Strip}(M)$
3. $\mathbf{v} := (v_2, \dots, v_d) \xleftarrow{\$} \mathbb{F}_p^d$
4. $\forall j \in [\ell], a'_j = M'_j \cdot \mathbf{v}$
5. $\text{fk} \leftarrow \text{PRG.Setup}(1^\lambda)$
6. $s := (GC, (y_1, \dots, y_n)) \leftarrow \text{Garble.Sim}(1^\lambda, 1^{|f_{M, \text{fk}}|}, 1^n, (a'_1, \dots, a'_\ell))$
7. $\mathbf{a} := (a_1, \dots, a_\ell)^\top \leftarrow W \cdot (s, \mathbf{v})^\top$

- **Hybrid 3:** The secret s is changed from being a function of M' to being a function of $W' = \text{Strip}(W)$.

1. $W \leftarrow \text{Compatible}(M)$
2. $W' = \text{Strip}(W)$
3. $\mathbf{v} := (v_2, \dots, v_d) \xleftarrow{\$} \mathbb{F}_p^d$
4. $\forall j \in [\ell], a'_j = W'_j \cdot \mathbf{v}$
5. $\text{fk} \leftarrow \text{PRG.Setup}(1^\lambda)$
6. $s := (GC, (y_1, \dots, y_n)) \leftarrow \text{Garble.Sim}(1^\lambda, 1^{|f_{W, \text{fk}}|}, 1^n, (a'_1, \dots, a'_\ell))$
7. $\mathbf{a} := (a_1, \dots, a_\ell)^\top \leftarrow W \cdot (s, \mathbf{v})^\top$

- **Hybrid 4:** We replace the truly random $v_2, \dots, v_d$ with PRG evaluations.

1. $W \leftarrow \text{Compatible}(M)$
2. $W' = \text{Strip}(W)$
3. $\text{fk} \leftarrow \text{PRG.Setup}(1^\lambda)$
4. $r \xleftarrow{\$} \{0, 1\}^n$
5. $\mathbf{v} := (v_2, \dots, v_d) \leftarrow \text{TLPRG.Eval}(\text{fk}, r)$
6. $\forall j \in [\ell], a'_j = W'_j \cdot \mathbf{v}$
7. $s := (GC, (y_1, \dots, y_n)) \leftarrow \text{Garble.Sim}(1^\lambda, 1^{|f_{W, \text{fk}}|}, 1^n, (a'_1, \dots, a'_\ell))$
8. $\mathbf{a} := (a_1, \dots, a_\ell)^\top \leftarrow W \cdot (s, \mathbf{v})^\top$

- **Hybrid 5:** In this hybrid, we generate the secret by running the Garbled Circuit evaluation rather than the simulator.

1. $W \leftarrow \text{Compatible}(M)$
2. $W' = \text{Strip}(W)$
3. $\text{fk} \leftarrow \text{PRG.Setup}(1^\lambda)$
4. $r \xleftarrow{\$} \{0, 1\}^n$
5. $\mathbf{v} := (v_2, \dots, v_d) \leftarrow \text{TLPRG.Eval}(\text{fk}, r)$
6. $(GC, \text{sk}) \leftarrow \text{Garble.Ckt}(f_{W, \text{fk}})$
7. $y_1, \dots, y_n \leftarrow \text{Garble.Inp}(\text{sk}, r)$
8. $(a'_1, \dots, a'_\ell) \leftarrow \text{Garble.Eval}(GC, (y_1, \dots, y_n))$
9. $s := (GC, y_1, \dots, y_n)$
10. $\mathbf{a} := (a_1, \dots, a_\ell)^\top \leftarrow W \cdot (s, \mathbf{v})^\top$

- **Hybrid 6:** Finally, replace setup for the PRG to the alternate mode. This is now identical to LSSS.AltSetup

1. $W \leftarrow \text{Compatible}(M)$
2. $W' = \text{Strip}(W)$
3. $r \xleftarrow{\$} \{0, 1\}^n$
4. $\text{fk} \leftarrow \text{PRG.AltSetup}(1^\lambda, r)$
5. $\mathbf{v} := (v_2, \dots, v_d) \leftarrow \text{TLPRG.Eval}(\text{fk}, r)$
6. $(GC, \text{sk}) \leftarrow \text{Garble.Ckt}(f_{W, g})$
7. $y_1, \dots, y_n \leftarrow \text{Garble.Inp}(\text{sk}, r)$
8. $(a'_1, \dots, a'_\ell) \leftarrow \text{Garble.Eval}(GC, (y_1, \dots, y_n))$
9. $s := (GC, y_1, \dots, y_n)$
10. $\mathbf{a} := (a_1, \dots, a_\ell)^\top \leftarrow W \cdot (s, \mathbf{v})^\top$

Lemma A.16. *Hybrid 0 is identical to Hybrid 1.*

The last step in Hybrid 1 is simply rewriting Hybrid 0 with a different notation.

Lemma A.17. *The output distributions of Hybrid 1 and Hybrid 2 are identical.*

The only difference in these two hybrids is in the generation of shares $\mathbf{a}$. Observe that in both hybrids, the same secret s is shared. Thus, Lemma A.5 implies that the output distribution of Hybrid 2 must be identical to that of Hybrid 1.

Lemma A.18. *The output distributions of Hybrid 2 and Hybrid 3 are identical.*

Proof. For any secret s , we can interpret $\{a'_j\}$ as the shares generated by sharing s using the matrix M' where M' is identical to M , except its first column is set of be 0. Let $\mathbf{D}_{M', s}$ denote the distribution of the shares generated using matrix M to share s .

Thus, to prove this lemma, it suffices to show that for any s , $\mathbf{D}_{M', s}$ is same as the distribution $\mathbf{D}_{W', s}$. We do this by using a sequence of intermediate distributions:

- For any s , the distribution $D_{M',s}$ must be identical to $D_{M,0}$. This is because M' is identical to M except its first column has been zeroed out.
- $D_{M,0}$ is identical to $D_{W,0}$. This follows from lemma A.5.
- Finally, by using the same argument as the first bullet point, we conclude that $D_{W,0}$ is identical to $D_{W',s}$.

This completes the proof. □

Lemma A.19. *For all security parameter $\lambda \in \mathbb{N}$, assuming one way functions, for all PPT adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that*

$$\left| \Pr[\mathcal{A}(s, (a_1, \dots, a_\ell)) = 1 \mid (s, (a_1, \dots, a_\ell)) \leftarrow \text{Hybrid 3}] \right. \\ \left. - \Pr[\mathcal{A}(s, (a_1, \dots, a_\ell)) = 1 \mid (s, (a_1, \dots, a_\ell)) \leftarrow \text{Hybrid 4}] \right|.$$

Proof. Observe that in Hybrid 4, the seed r is not used anywhere other than in the generation of $\mathbf{v}$, thus the indistinguishability of these two hybrids follow from the pseudorandomness of TLPRG. □

Lemma A.20. *Assuming perfect simulation security of Garble, the output distributions of Hybrid 4 and Hybrid 5 are identical.*

Proof. The only difference between these hybrids is the way we generate the secret s . Note that $\mathbf{a}$ is generated from s using a deterministic process. Therefore, it suffices to show that the distribution of s in both the hybrids are identical. In Hybrid 4, s is generated using the garbles circuit simulator `Garble.Sim` where it is generated using the honest garbled circuit and garbled input generators `Garble.Ckt` and `Garble.Inp`. Thus, simulation security of the garbled circuit proves the lemma. □

Lemma A.21. *For all security parameter $\lambda \in \mathbb{N}$, assuming one way functions, for all PPT adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that*

$$\left| \Pr[\mathcal{A}(s, (a_1, \dots, a_\ell)) = 1 \mid (s, (a_1, \dots, a_\ell)) \leftarrow \text{Hybrid 5}] \right. \\ \left. - \Pr[\mathcal{A}(s, (a_1, \dots, a_\ell)) = 1 \mid (s, (a_1, \dots, a_\ell)) \leftarrow \text{Hybrid 6}] \right|.$$

Proof. The proof of this lemma follows in a straightforward manner from the setup indistinguishability property of TLPRG. □

□

Privacy.

Lemma A.22. *Let λ be the security parameter. For any policy matrix M and any subset $T \subset [\ell]$ such that $\{M_{i,\cdot} : i \in T\}$ does not span $\mathbf{e}_1$, there exists $k = \text{poly}(\lambda)$ such that*

$$H_\infty(s \mid \{a_j\}_{j \in T}) \geq k,$$

where $(s, a_1, \dots, a_\ell) \leftarrow \text{LSSS.AltSetup}(1^\lambda, M, T)$.

Proof. This property follows from the k -lossiness of TLPRG in the alternate setup mode.

Note that W has the property, $W_{j1} = 0$ for all $j \in T$, where T is the set of all unauthorized rows. This implies that $\forall j \in T, a_j = a'_j$. k -lossiness of TLPRG in the alternate setup mode implies that $H_\infty(r \mid \mathbf{v} = \text{TLPRG.Eval}(\text{fk}, r)) \geq k$ where $\text{fk} \leftarrow \text{TLPRG.AltSetup}(1^\lambda, r)$. Thus the fact that $\{a'_j\}_{j \in [\ell]}$ are generated as a deterministic process from $\mathbf{v}$,

$$H_\infty(r \mid \{a'_j\}_{j \in [\ell]}) \geq k \implies H_\infty(r \mid \{a_j\}_{j \in T}) \geq k.$$

□

Reconstruction Soundness. For all $M \in \mathcal{M}$, if $\nexists (v_1, \dots, v_d) \in \mathbb{F}_p^d \setminus \{0^d\} : M \cdot (v_1, v_2, \dots, v_d) = (a_1, \dots, a_\ell)$, then

$$\text{LSSS.Reconstruct}(F, (a_1, \dots, a_\ell)) = \perp.$$

Moreover, $\forall$ (maliciously generated) $(a_1, \dots, a_\ell) \in \mathbb{F}_p^\ell$ such that $\exists (v_1, \dots, v_d) \in \mathbb{F}_p^d \setminus \{0^d\} : M \cdot (v_1, v_2, \dots, v_d) = (a_1, \dots, a_\ell)$, for all sets $S = \{i : M_{i,\cdot}\}$ that spans $\mathbf{e}_1$, $\text{LSSS.Reconstruct}(M, \{a_i\}_{i \in S})$ outputs the same value.

Proof. The first point follows directly by observation from the construction of **Reconstruct**.

For the second one, observe that if $\exists (v_1, \dots, v_d) \in \mathbb{F}_p^d \setminus \{0^d\} : M \cdot (v_1, v_2, \dots, v_d) = (a_1, \dots, a_\ell)$, then for all $S : \{i : M_{i,\cdot}\}$ such that $M_{i,\cdot}$ spans $\mathbf{e}_1$ we have

$$\sum_{j \in S} \omega_j a_j = \sum_{j \in S} \omega_j M_{j,\cdot} (v_1, \dots, v_d) = \mathbf{e}_1^\top (v_1, \dots, v_d) = v_1.$$

□

Regeneration Soundness.

Lemma A.23. *For all $M \in \mathcal{M}$, for all $s \in \mathbb{F}_p$, either $\text{LSSS.Regenerate}(M, s) = \perp$; or $\exists (v_1, \dots, v_d) \in \mathbb{F}_p^d \setminus \{0^d\} : M \cdot (v_1, v_2, \dots, v_d) = \text{LSSS.Regenerate}(M, s)$*

Proof. This again follows directly from the construction. □

We have constructed an efficient computational linear secret sharing scheme above. To categorize the class of functions that can admit an efficient LSSS, we use the following results.

Monotone Boolean Formula as LSSS matrix. [LW11] Any monotone boolean function $f : \{0, 1\}^n \rightarrow \{0, 1\}$ can be interpreted as an access structure over a set of n parties $P = (P_1, \dots, P_n)$, where a set $S \subset P$ is said to be authorized, $S \in \mathcal{P}(P)$ iff there exists x such that $f(x) = 1$ and $S = \{P_i : x_i = 1\}$. We can represent any monotone boolean formula as an access tree, where the interior nodes are AND and OR gates, and the leaf nodes correspond to inputs. The number of rows in the corresponding LSSS matrix will be same as the number of leaf nodes in the access tree.

The class of functions that admit an efficient LSSS therefore coincides with a subclass of the class of functions that can be solved by polynomial sized monotone boolean circuits. In particular we know the following lemmata:

Lemma A.24 (See Corollary 4 in [KW93]). *Every function in the complexity class monotone-Polynomial sized Span Programs (mPSP) has an efficient LSSS.*

Lemma A.25 (See Theorem 5 in [KW93]). *The complexity class monotone-Symmetric LOGSPACE (mSL) is contained in mPSP.*

Lemma A.26 ([Rei08]). *Monotone-LOGSPACE = mSL.*

Combining the above lemmata we get,

Lemma A.27. *Monotone-NC1 $\subseteq$ Monotone-LOGSPACE and therefore every function in monotone-NC1 has an efficient LSSS.*

Definition A.28 (Partite Functions [ABI⁺23]). *For every $x \in \{0, 1\}^n$, let a define an encoding y in $2n$ bits such that $y_i = 1$ and $y_{i+1} = 0$ if $x_i = 1$, and $y_i = 0$ and $y_{i+1} = 1$ if $x_i = 0$. For every boolean function $f : \{0, 1\}^n \rightarrow \{0, 1\}$, we define its partite function as any monotone function $g : \{0, 1\}^{2n} \rightarrow \{0, 1\}$ such that for all $y \in \{0, 1\}^{2n}$ that encodes some $x \in \{0, 1\}^n$, $f(x) = g(y)$, and*

Lemma A.29. *If $f : \{0, 1\}^n \rightarrow \{0, 1\}$ is in NC1, then there exists a function $g : \{0, 1\}^{2n} \rightarrow \{0, 1\}$ in monotone-NC1 which is a partite function of f .*

Proof. Let the circuit C computing f have n leaf nodes (inputs), fan-in 2, depth d and width w . Without loss of generality assume that f only has AND, OR, and NOT gates. We will show that one can convert this circuit into a monotone one for computing g with $2n$ leaf nodes, depth at most d and width at most $2w$. Define a new circuit C' via the following (inefficient) iterative transformation:

- If C has a NOT gate in depth $k > 1$ whose input wire emerges from an AND gate at depth $k - 1$, i.e., a partial circuit of the form NOT(a AND b), then replace it with the partial circuit (NOT(a) OR NOT(b)). Thus, replace the NOT gate in depth k with an OR gate and replace the AND gate in depth $k - 1$ with 2 NOT gates. The functionality remains unchanged due to De Morgan's law. This transformation does not alter the depth of the tree while increasing its width at most by a factor of 2. Similarly if there is a partial circuit of the form NOT(a OR b), replace it with (NOT(a) AND NOT(b))
- If C has a NOT gate in depth 1, then replace the NOT gate from a variable x with a wire from variable $\bar{x}$. This can neither increase the width or depth of the circuit while keeping the functionality same. That is for every y that is an encoding of x , $C'(y) = C(x)$.

C' is the circuit that evaluates the partite function g correctly on inputs $y \in \{0, 1\}^{2n}$ that are valid encodings of any $x \in \{0, 1\}^n$. $\square$

Thus we can say that there is an efficient LSSS for every function in NC1. Specifically,

Lemma A.30. *Every function in NC1 defined over n variables has an efficient LSSS with policy matrix consisting of $2n$ rows.*

Combining all these results we get the final theorem,

Theorem A.31 (Efficient randomness-recoverable secret sharing for NC1). *Assuming one-way functions, there exists an efficient computational linear secret sharing scheme for any function computable in NC1.*

A.4 Construction of Targeted Lossy PRG

Let g be a PRG from expanding from n bits to $3n + 1$ bits¹⁰. Every element of $\mathbb{F}_{d+1}$ can be represented as a degree d polynomial with coefficients over $\mathbb{F}_2$. For any $x \in \mathbb{F}_{d+1}$, $\text{chop}(x)$ represents the first d bits of x . For any field $\mathbb{F}_{d+1}$, $\mathbf{1}_{\mathbb{F}}$ represents the field element which has 1 as the constant, and 0 as all other d many coefficients. In other words, it is the field elements where the leftmost d bits are 0 and the least significant bit is 1. Note that for all $x \in \mathbb{F}_{d+1}$, $\text{chop}(x) = \text{chop}(x + \mathbf{1}_{\mathbb{F}})$.

- $\text{TLPRG.Setup}(1^\lambda) : \text{fk} := h \xleftarrow{\$} \mathbb{F}_{2^{3kn+k}}$
- $\text{TLPRG.AltSetup}(1^\lambda, x^*) :$
 1. Parse $x^* \in \{0, 1\}^{kn}$ as $x_1^* \parallel \dots \parallel x_k^*$ where $x_i^* \in \{0, 1\}^n$
 2. $\forall i \in [k]$, sample $x'_i \in \{0, 1\}^n \neq x_i^*$
 3. $\forall i \in [k]$, set $h_i = \mathbf{1}_{\mathbb{F}} \cdot (g(x_i^*) - g(x'_i))^{-1}$
 4. $\text{fk} := h_1 \parallel \dots \parallel h_k$
- $\text{TLPRG.Eval}(\text{fk}, x) :$
 1. Parse x as $x_1 \parallel \dots \parallel x_k$
 2. Parse fk as $h_1 \parallel \dots \parallel h_k$
 3. $\forall i \in [k]$, $c_i \leftarrow \text{chop}(h_i \cdot g(x_i))$
 4. Output $y := c_1 \parallel \dots \parallel c_k$

The proof that our construction satisfies the necessary properties follow mostly from the analysis of Targeted Lossy Functions in [QWW21]. We present somewhat detailed proofs here for completeness sake.

Pseudorandomness.

Lemma A.32. *Assuming pseudorandomness of g , for all security parameter $\lambda \in \mathbb{N}$, $k, n = \text{poly}(\lambda)$ and all PPT adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that*

$$\left| \Pr \left[\mathcal{A}(\text{TLPRG.Eval}(\text{fk}, x)) = 1 \mid x \xleftarrow{\$} \{0, 1\}^n, \text{fk} \leftarrow \text{TLPRG.Setup}(1^\lambda) \right] - \Pr \left[\mathcal{A}(y) = 1 \mid y \xleftarrow{\$} \{0, 1\}^{3kn} \right] \right| \leq \text{negl}(\lambda).$$

¹⁰The choice of $3n + 1$ as the output length is arbitrary.

Proof. We use an intermediate hybrid where TLPRG.Eval computes $c_i = \text{chop}(h_i \cdot r)$ where $r \xleftarrow{\$} \{0, 1\}^{3n+1}$. This is computationally indistinguishable from the real TLPRG.Eval algorithm if g is a PRG.

Say there exists an adversary $\mathcal{A}$ that can distinguish between the above two cases, then this can be converted to another adversary $\mathcal{A}'$ which can break the pseudorandomness of g .

Upon input some $y := (y_1, \dots, y_k)$ where $y_i \in \{0, 1\}^{3n+1}$, $\mathcal{A}'$ first computes fk by running TLPRG.Setup and then outputs $\text{chop}(h_1 \cdot y_1) \parallel \dots \parallel \text{chop}(h_k \cdot y_k)$. If y_i was output of a PRG g , then $\mathcal{A}$ is interacting with the real TLPRG.Eval algorithm, otherwise, it is interacting with the modified hybrid. $\mathcal{A}'$ returns whatever $\mathcal{A}$ returns. Clearly, the distinguishing probability of $\mathcal{A}'$ is same as that of $\mathcal{A}$.

Finally, it is easy to see that $\text{chop}(h_1 \cdot r_1) \parallel \dots \parallel \text{chop}(h_k \cdot r_k)$ has the same distribution as $r'_1 \parallel \dots \parallel r'_k$ where $r'_i \xleftarrow{\$} \{0, 1\}^{3n}$. This is because, given h_i , and for some $r'' \in \{0, 1\}^{3n+1}$

$$\Pr[h_i \cdot r_i = r''_i] = \Pr[r_i = r''_i \cdot h_i^{-1}] = \frac{1}{2^{3n+1}}.$$

Thus, for all $i \in [k]$, $\Pr[\text{chop}(h_i \cdot r_i) = r'_i] = \frac{1}{2^{3n}}$. □

Setup indistinguishability. We use an intermediate hybrid to show indistinguishability of TLPRG.Setup and TLPRG.AltSetup .

$\text{TLPRG.AltSetup}(1^\lambda, x^*) :$

1. Parse $x^* \in \{0, 1\}^{kn}$ as $x_1^* \parallel \dots \parallel x_k^*$ where $x_i^* \in \{0, 1\}^n$
2. $\forall i \in [k]$, sample $x'_i \in \{0, 1\}^n \neq x_i^*$
3. $\forall i \in [k]$, sample $u_i \in \{0, 1\}^{3n+1}$
4. if $\exists i \in [k]$ such that $u_i = g(x_i^*)$, then $\text{fk} := 0$
5. $\forall i \in [k]$, set $h_i = \mathbf{1}_{\mathbb{F}} \cdot (g(x_i^*) - u_i)^{-1}$
6. $\text{fk} := h_1 \parallel \dots \parallel h_k$

Observe that the probability that any $u_i = g(x_i^*)$ is at most $\frac{k}{2^{3n+1}}$. If this does not happen, then TLPRG.AltSetup is computationally indistinguishable from TLPRG.Setup due to the pseudorandomness of g .

Finally, since u_i are generated uniformly at random, the distribution of h_i is identical in both TLPRG.AltSetup and TLPRG.Setup .

k -Lossiness. Lossiness is also straightforward. Observe that when $k = 1$, clearly for every $x_1 \in \{0, 1\}^n$, there exists $x_2 \in \{0, 1\}^n$ such that $h_1(x_1) = h(x_2)$ whenever $h_1 \leftarrow \text{TLPRG.AltSetup}(1^\lambda, x_1)$. Therefore,

$$H_\infty(x_1 \mid \text{fk}, y) \geq H_\infty(x_1 \mid \{x_1, x_2\}) = 1.$$

Now, our TLPRG construction is nothing but k -many independent repetitions of this procedure, therefore, we can say that

$$H_\infty(x_1 \parallel \dots \parallel x_k \mid \text{fk}, y_1 \parallel \dots \parallel y_k) \geq k.$$

We refer reader to the proof of Theorem 4.1 in [QWW21] for a more detailed proof.

B ABSFE from Secret Sharing for Monotone Functions

In this section, we present the construction as in similar to Section 5, except we replace SS-RR with LSSS. Most of the construction remains identical.

- $\text{ABSFE.Setup}(1^\lambda)$:
 1. Sample $H \xleftarrow{\$} \mathcal{H}$
 2. Output $\text{pp} := H$
- $\text{ABSFE.Keygen}(\text{pp}, k \in \{0, 1\}^\ell)$:
 1. $S \leftarrow \rho(k)$
 2. $\forall i \in S, (ev_i, td_i) \leftarrow \text{LTDF.Setup}(1^\lambda, 1^\delta)$
 3. $\forall i \notin S, ev_i \leftarrow \text{LTDF.AltSetup}(1^\lambda, 1^\delta)$
 4. $\text{pk} \leftarrow \{ev_i\}_{i \in [\ell]}$
 5. $\text{sk} := (k, \{td_i\}_{i \in S})$
 6. $\text{msk} := \text{sk}$
 7. Output (pk, sk)
- $\text{ABSFE.Enc}(\text{pp}, \text{pk}, x \in \{0, 1\}^\lambda, m \in \{0, 1\}^\gamma)$:
 1. $(s, (a_1, \dots, a_\ell)) \leftarrow \text{LSSS.Setup}(f_x)$
 2. $\forall j \in \ell, \text{ct}_j \leftarrow \text{LTDF.Eval}(ev_j, a_j)$
 3. $\text{ct}_0 \leftarrow m \oplus H(s)$
 4. Output $\text{ct} := (\text{ct}_0, \{\text{ct}_j\}_{j \in [\ell]})$
- $\text{ABSFE.Dec}(\text{pp}, \text{sk}, x, \text{ct})$:
 1. If $f(x, k) = 0$, then output $\perp$
 2. for all $j \in S$, let $a'_j \leftarrow \text{LTDF.Invert}(td_j, \text{ct}_j)$
 3. $s' \leftarrow \text{LSSS.Reconstruct}(f_x, \{a'_j\}_{j \in S})$
 4. $(\tilde{a}_1, \dots, \tilde{a}_\ell) \leftarrow \text{LSSS.Regenerate}(f_x, s')$
 5. If $\exists j \in [\ell]$, such that $\text{LTDF.Eval}(ev_j, \tilde{a}_j) \neq \text{ct}_j$, then output $\perp$
 6. Else, output $\text{ct}_0 \oplus H(s')$

Strong Key-Hiding

Lemma B.1. *Assume the existence of length parameterized lossy trapdoor functions (see Definition 3.2). For all $\lambda \in \mathbb{N}$, there exists a PPT simulator $\text{Sim} := (\text{Sim}_1, \text{Sim}_2)$ such that for all keys $k \in \{0, 1\}^\ell$ and for all PPT adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that*

$$\left| \Pr \left[\mathcal{A}^{\mathcal{O}_1(\text{pp}, \text{sk}, \cdot, \cdot)}(\text{pp}, \text{pk}) = 1 \mid \text{pp} \leftarrow \text{ABSFE.Setup}(1^\lambda), (\text{pk}, \text{sk}) \leftarrow \text{ABSFE.Keygen}(\text{pp}, k) \right] - \Pr \left[\mathcal{A}^{\mathcal{O}_2(\text{st}, \cdot, \cdot)}(\text{pp}, \text{pk}) = 1 \mid (\text{st}, \text{pp}, \text{pk}) \leftarrow \text{Sim}_1(1^\lambda) \right] \right| \leq \text{negl}(\lambda),$$

where $\mathcal{O}_1$ is the real decryption oracle, i.e., $\text{ABSFE.Dec}(\text{pp}, \text{sk}, x, \text{ct})$, and $\mathcal{O}_2$ is the ideal decryption oracle which uses the simulator $\text{Sim}_2^{f(\cdot, k)}(\text{st}, x, \text{ct})$.

Proof. It suffices to construct a simulator such that the two oracles $\mathcal{O}_1$ and $\mathcal{O}_2$ are computationally indistinguishable. Clearly, the existence of a distinguisher for this can be transformed into an adversary that can win the key-hiding game with non-negligible probability.

First, let us describe the simulators $\text{Sim}_1(1^\lambda)$ and $\text{Sim}_2^{F(\cdot, k)}(\text{st}, x, \text{ct})$:

- $\text{Sim}_1(1^\lambda)$:

1. $H \xleftarrow{\$} \mathcal{H}$
2. $\forall i \in [\ell], (ev_i, td_i) \leftarrow \text{LTDF.Setup}(1^\lambda, 1^\delta)$
3. $\text{pk} \leftarrow \{ev_i\}_{i \in [\ell]}$
4. $\text{st} \leftarrow (\text{pk}, \{td_i\}_{i \in [\ell]})$
5. Output $(\text{st}, H, \text{pk})$

- $\text{Sim}_2^{F(\cdot, k)}(\text{st}, x, \text{ct})$

1. Parse ct as $(\text{ct}_0, \{\text{ct}_j\}_{j \in [\ell]})$
2. Query x to oracle $F(\cdot, k)$. If it returns 0, then output $\perp$, else continue
3. $\forall j \in [\ell], a'_j \leftarrow \text{LTDF.Invert}(td_j, \text{ct}_j)$
4. $s' \leftarrow \text{LSSS.Reconstruct}(f_x, \{a'_j\}_{j \in [\ell]})$
5. $(\tilde{a}_1, \dots, \tilde{a}_\ell) \leftarrow \text{LSSS.Regenerate}(f_x, s')$
6. If $\exists j \in [\ell]$, such that $\text{LTDF.Eval}(ev_j, \tilde{a}_j) \neq \text{ct}_j$, then output $\perp$
7. Else, output $\text{ct}_0 \oplus H(s')$

We will now go through a sequence of hybrids starting from the real world decryption oracle. Note that in all the hybrids here, the challenger always has access to the master secret key.

- **Hybrid 0:** This is the real world key generation and decryption oracle which responds to ciphertext queries by using the secret key sk . We write out the entire hybrid for ease of exposition. The setup algorithm remains unchanged, hence we skip writing it out.

- $\text{ABSFE.Keygen}(\text{pp}, k \in \{0, 1\}^\ell)$:

1. $S \leftarrow \rho(k)$.
2. $\forall i \in S, (ev_i, td_i) \leftarrow \text{LTDF.Setup}(1^\lambda, 1^\delta)$
3. $\forall i \notin S, (ev_i, td_i) \leftarrow \text{LTDF.AltSetup}(1^\lambda, 1^\delta)$
4. $\text{pk} \leftarrow \{ev_i\}_{i \in [\ell]}$
5. $\text{sk} := (k, \{td_i\}_{i \in S})$
6. $\text{msk} := \text{sk}$
7. Output (pk, sk)

- $\text{ABSFE.Dec}(\text{pp}, \text{sk}, x, \text{ct})$:

1. If $f(x, k) = 0$, then output $\perp$
2. for all $j \in S$, let $a'_j \leftarrow \text{LTDF.Invert}(td_j, \text{ct}_j)$
3. $s' \leftarrow \text{LSSS.Reconstruct}(f_x, \{a'_j\}_{j \in S})$
4. $(\tilde{a}_1, \dots, \tilde{a}_\ell) \leftarrow \text{LSSS.Regenerate}(f_x, s')$
5. If $\exists j \in [\ell]$, such that $\text{LTDF.Eval}(ev_j, \tilde{a}_j) \neq \text{ct}_j$, then output $\perp$

6. Else, output $\text{ct}_0 \oplus H(s')$

• **Hybrid 1:**

– $\text{ABSFE.Keygen}(\text{pp}, k \in \{0, 1\}^\ell)$:

1. $S \leftarrow \rho(k)$.
2. $\forall i \in [\ell], (ev_i, td_i) \leftarrow \text{LTDF.Setup}(1^\lambda, 1^\delta)$
3. $\text{pk} \leftarrow \{ev_i\}_{i \in [\ell]}$
4. $\text{sk} := (k, \{td_i\}_{i \in S})$
5. $\text{msk} := \text{sk}$
6. Output (pk, sk)

– $\text{ABSFE.Dec}(\text{pp}, \text{sk}, x, \text{ct})$:

1. If $f(x, k) = 0$, then output $\perp$
2. for all $j \in S$, let $a'_j \leftarrow \text{LTDF.Invert}(td_j, \text{ct}_j)$
3. $s' \leftarrow \text{LSSS.Reconstruct}(f_x, \{a'_j\}_{j \in S})$
4. $(\tilde{a}_1, \dots, \tilde{a}_\ell) \leftarrow \text{LSSS.Regenerate}(f_x, s')$
5. If $\exists j \in [\ell]$, such that $\text{LTDF.Eval}(ev_j, \tilde{a}_j) \neq \text{ct}_j$, then output $\perp$
6. Else, output $\text{ct}_0 \oplus H(s')$

• **Hybrid 2:**

1. If $f(x, k) = 0$, then output $\perp$
2. for all $j \in [\ell]$, let $a'_j \leftarrow \text{LTDF.Invert}(td_j, \text{ct}_j)$
3. $s' \leftarrow \text{LSSS.Reconstruct}(f_x, \{a'_j\}_{j \in S})$
4. $(\tilde{a}_1, \dots, \tilde{a}_\ell) \leftarrow \text{LSSS.Regenerate}(f_x, s')$
5. If $\exists j \in [\ell]$, such that $\text{LTDF.Eval}(ev_j, \tilde{a}_j) \neq \text{ct}_j$, then output $\perp$
6. If $\exists j \in [\ell]$ such that $a'_j \neq \tilde{a}_j$, then output $\perp$
7. Else, output $\text{ct}_0 \oplus H(s')$

- **Hybrid 3:** In this hybrid, we regenerate the shares using all the shares which were inverted using the master secret key. The reason for doing this step is to check whether $\{a'_{j,b}\}_{j \in [\ell], b \in \{0,1\}}$ forms a valid shares for some secret.

1. If $f(x, k) = 0$, then output $\perp$
2. for all $j \in [\ell]$, let $a'_j \leftarrow \text{LTDF.Invert}(td_j, \text{ct}_j)$
3. $s' \leftarrow \text{LSSS.Reconstruct}(f_x, \{a'_j\}_{j \in [\ell]})$
4. $(\tilde{a}_1, \dots, \tilde{a}_\ell) \leftarrow \text{LSSS.Regenerate}(f_x, s')$
5. If $\exists j \in [\ell]$, such that $\text{LTDF.Eval}(ev_j, \tilde{a}_j) \neq \text{ct}_j$, then output $\perp$
6. If $\exists j \in [\ell]$ such that $a'_j \neq \tilde{a}_j$, then output $\perp$
7. Else, output $\text{ct}_0 \oplus H(s')$

- **Hybrid 4:** Here we change the consistency check criterion.

1. If $f(x, k) = 0$, then output $\perp$

2. for all $j \in [\ell]$, let $a'_j \leftarrow \text{LTDF.Invert}(td_j, \text{ct}_j)$
3. $s' \leftarrow \text{LSSS.Reconstruct}(f_x, \{a'_j\}_{j \in [\ell]})$
4. $(\tilde{a}_1, \dots, \tilde{a}_\ell) \leftarrow \text{LSSS.Regenerate}(f_x, s')$
5. If $\exists j \in [\ell]$, such that $\text{LTDF.Eval}(ev_j, \tilde{a}_j) \neq \text{ct}_j$, then output $\perp$
6. ~~If $\exists j \in [\ell]$ such that $a'_j \neq \tilde{a}_j$, then output $\perp$~~
7. Else, output $\text{ct}_0 \oplus H(s')$

• **Hybrid 5:** This is identical to the Simulator

1. ~~Query x to oracle $F(\cdot, k)$. If it returns 0, then output $\perp$, else continue~~
2. for all $j \in [\ell]$, let $a'_j \leftarrow \text{LTDF.Invert}(td_j, \text{ct}_j)$
3. $s' \leftarrow \text{LSSS.Reconstruct}(f_x, \{a'_j\}_{j \in [\ell]})$
4. $(\tilde{a}_1, \dots, \tilde{a}_\ell) \leftarrow \text{LSSS.Regenerate}(f_x, s')$
5. If $\exists j \in [\ell]$, such that $\text{LTDF.Eval}(ev_j, \tilde{a}_j) \neq \text{ct}_j$, then output $\perp$
6. Else, output $\text{ct}_0 \oplus H(s')$

We go through a sequence of hybrids. We skip the details of the proof as it is identical to Section 5.

Lemma B.2. *Assuming setup indistinguishability of LTDF, for all $\lambda \in \mathbb{N}$, and all PPT distinguishers $\mathcal{D}$,*

$$\Pr[\mathcal{D}^{\text{Hybrid } 0}(1^\lambda) = 1] = \Pr[\mathcal{D}^{\text{Hybrid } 1}(1^\lambda) = 1].$$

Lemma B.3. *Assuming correctness of LTDF inversion in the injective mode, for all $\lambda \in \mathbb{N}$, and all PPT distinguishers $\mathcal{D}$,*

$$\Pr[\mathcal{D}^{\text{Hybrid } 1}(1^\lambda) = 1] = \Pr[\mathcal{D}^{\text{Hybrid } 2}(1^\lambda) = 1].$$

In fact we prove that Hybrid 1 and 2 are identical to each other.

Lemma B.4. *Assuming reconstruction and regeneration soundness of LSSS and correctness of LTDF inversion, Hybrids 2 and 3 are identical to each other.*

Proof. The two hybrids are identical if the secret reconstructed in both hybrids are identical. We split the analysis according to cases:

- $\nexists (v_1, \dots, v_d)$ such that $M(v_1, \dots, v_d) = (a'_1, \dots, a'_\ell)$: We know that Reconstruct in Hybrid 3 outputs $\perp$. Moreover by regeneration soundness either Regenerate in Hybrid 2 outputs $\perp$ in which case both hybrids are identical or it outputs something in the column space of M , and hence Hybrid 2 must abort in Step 6 by injectivity of LTDF.
- $\exists (v_1, \dots, v_d)$ such that $M(v_1, \dots, v_d) = (a'_1, \dots, a'_\ell)$, reconstruction soundness also ensures that the output of Reconstruct in both hybrids are identical.

□

Lemma B.5. *Hybrid 3 is identical to Hybrid 4.*

This just follows from the observation that Regenerate is deterministic and that in Hybrid 4 Step 6 cannot output $\perp$ unless Step 5 already does so.

Lemma B.6. *Hybrid 4 is identical to Hybrid 5.*

□

Weak Statistical Message Hiding.

Lemma B.7. *Assume the existence of length parametrized lossy trapdoor functions and a SS-RR perfect privacy. Fix $\lambda, \delta \in \mathbb{N}$, a PRF key $k \in \{0, 1\}^\lambda$. Then, for all admissible adversaries $\mathcal{A} := (\mathcal{A}_1, \mathcal{A}_2)$, there exists a negligible function $\text{negl}(\cdot)$ such that*

$$\Pr \left[b' \leftarrow \mathcal{A}_2^{\mathcal{O}(\text{pp}, \text{pk}, b, \cdot, \cdot)}(\text{pp}, \text{pk}, \text{sk}, \tau) \wedge b' = b \mid (k, \tau) \leftarrow \mathcal{A}_1(1^\lambda), \text{pp} \leftarrow \text{ABSFE.Setup}(1^\lambda), \right. \\ \left. (\text{pk}, \text{sk}) \leftarrow \text{ABSFE.Keygen}(\text{pp}, k), b \xleftarrow{\$} \{0, 1\} \right] = \text{negl}(\lambda),$$

where

- $\mathcal{O}(\text{pp}, \text{pk}, b, (x, m_0, m_1))$ is an oracle which upon queried an attribute x , messages $m_0, m_1 \in \mathcal{M}$ returns a ciphertext $\text{ct} \leftarrow \text{ABSFE.Enc}(\text{pp}, \text{pk}, x, m_b)$.
- $\mathcal{A}$ is admissible if it makes one ciphertext query (x, m_0, m_1) to $\mathcal{O}$, and $F(x, k) = 0$.

Proof. We show via a sequence of hybrids that there does not exist any PPT distinguisher $\mathcal{A}$ that can distinguish between $\text{ABSFE.Enc}(\text{pp}, \text{pk}, x, m_0)$ and $\text{ABSFE.Enc}(\text{pp}, \text{pk}, x, m_1)$ even when it is allowed to make a single-key-generation query for k as long as for all encryption queries of the form $(x, \cdot, \cdot)$ satisfy $F(x, k) = 0$. Since $\mathcal{A}$ is admissible, it receives a secret key sk corresponding to k which is associated with an *unauthorized* set of shares.

1. Hybrid 0:

- $\text{ABSFE.Setup}(1^\lambda)$:
 - Sample $H \xleftarrow{\$} \mathcal{H}$
 - Output $\text{pp} := H$
- $\text{ABSFE.Keygen}(\text{pp}, k \in \{0, 1\}^\ell)$:
 - $S \leftarrow \rho(k)$
 - $\forall i \in S, (ev_i, td_i) \leftarrow \text{LTDF.Setup}(1^\lambda, 1^\delta)$
 - $\forall i \notin S, ev_i \leftarrow \text{LTDF.AltSetup}(1^\lambda, 1^\delta)$
 - $\text{pk} \leftarrow \{ev_i\}_{i \in [\ell]}$
 - $\text{sk} := (k, \{td_i\}_{i \in S})$
 - $\text{msk} := \text{sk}$
 - Output (pk, sk)
- $\text{ABSFE.Enc}(\text{pp}, \text{pk}, x \in \{0, 1\}^\lambda, m \in \{0, 1\}^\gamma)$:
 - $(s, (a_1, \dots, a_\ell)) \leftarrow \text{LSSS.Setup}(f_x)$
 - $\forall j \in \ell, \text{ct}_j \leftarrow \text{LTDF.Eval}(ev_j, a_j)$
 - $\text{ct}_0 \leftarrow m \oplus H(s)$
 - Output $\text{ct} := (\text{ct}_0, \{\text{ct}_j\}_{j \in [\ell]})$

2. **Hybrid 1:** Here we swap the linear secret sharing setup to the alternate mode. Note that the adversary queries k to the reduction, and therefore it knows the key and can compute unauthorized set T .

- $\text{ABSFE.Setup}(1^\lambda)$:

- (a) Sample $H \xleftarrow{\$} \mathcal{H}$
- (b) Output $\text{pp} := H$
- **ABSFE.Keygen**($\text{pp}, k \in \{0, 1\}^\ell$):
 - (a) $S \leftarrow \rho(k)$
 - (b) $\forall i \in S, (ev_i, td_i) \leftarrow \text{LTDF.Setup}(1^\lambda, 1^\delta)$
 - (c) $\forall i \notin S, ev_i \leftarrow \text{LTDF.AltSetup}(1^\lambda, 1^\delta)$
 - (d) $\text{pk} \leftarrow \{ev_i\}_{i \in [\ell]}$
 - (e) $\text{sk} := (k, \{td_i\}_{i \in S})$
 - (f) $\text{msk} := \text{sk}$
 - (g) Output (pk, sk)
- **ABSFE.Enc**($\text{pp}, \text{pk}, x \in \{0, 1\}^\lambda, m \in \{0, 1\}^\gamma$):
 - (a) $T \leftarrow \rho(k)$
 - (b) $(s, (a_1, \dots, a_\ell)) \leftarrow \text{LSSS.AltSetup}(f_x, T)$
 - (c) $\forall j \in \ell, \text{ct}_j \leftarrow \text{LTDF.Eval}(ev_j, a_j)$
 - (d) $\text{ct}_0 \leftarrow m \oplus H(s)$
 - (e) Output $\text{ct} := (\text{ct}_0, \{\text{ct}_j\}_{j \in [\ell]})$

3. Hybrid 2:

- **ABSFE.Setup**(1^λ):
 - (a) Sample $H \xleftarrow{\$} \mathcal{H}$
 - (b) Output $\text{pp} := H$
- **ABSFE.Keygen**($\text{pp}, k \in \{0, 1\}^\ell$):
 - (a) $S \leftarrow \rho(k)$
 - (b) $\forall i \in S, (ev_i, td_i) \leftarrow \text{LTDF.Setup}(1^\lambda, 1^\delta)$
 - (c) $\forall i \notin S, ev_i \leftarrow \text{LTDF.AltSetup}(1^\lambda, 1^\delta)$
 - (d) $\text{pk} \leftarrow \{ev_i\}_{i \in [\ell]}$
 - (e) $\text{sk} := (k, \{td_i\}_{i \in S})$
 - (f) $\text{msk} := \text{sk}$
 - (g) Output (pk, sk)
- **ABSFE.Enc**($\text{pp}, \text{pk}, x \in \{0, 1\}^\lambda, m \in \{0, 1\}^\gamma$):
 - (a) $(s, (a_1, \dots, a_\ell)) \leftarrow \text{LSSS.AltSetup}(f_x)$
 - (b) $\forall j \in \ell, \text{ct}_j \leftarrow \text{LTDF.Eval}(ev_j, a_j)$
 - (c) $r \xleftarrow{\$} \{0, 1\}^\gamma$
 - (d) $\text{ct}_0 \leftarrow r$
 - (e) Output $\text{ct} := (\text{ct}_0, \{\text{ct}_j\}_{j \in [\ell]})$

Lemma B.8. Assuming setup indistinguishability of LSSS, for all PPT adversaries $\mathcal{D}$, there exists a negligible function $\text{negl}(\cdot)$ such that,

$$\Pr[\mathcal{D}^{\text{Hybrid } 0}(1^\lambda) = 1] - \Pr[\mathcal{D}^{\text{Hybrid } 1}(1^\lambda) = 1] \leq \text{negl}(\lambda)$$

Proof. This proof works via a reduction from a PPT adversary that wants to distinguish between secret and shares generated by LSSS.Setup vs LSSS.AltSetup . We skip details of the reduction as the proof is similar to that of Lemma 5.3. $\square$

Lemma B.9. *Fix $\lambda \in \mathbb{N}$ and $\delta, k, \ell, \gamma, \gamma_1 \in \text{poly}(\lambda)$. Assume γ_1 -lossiness of length parameterized LTDF in the lossy mode ($k < \delta$), and k -lossiness of LSSS in the alternate mode, for all distinguishers $\mathcal{D}$, there exists a negligible function $\text{negl}(\cdot)$ such that*

$$\left| \Pr[\mathcal{D}(\text{pp}, \text{pk}, \text{sk}, \text{ct}) = 1 : (\text{pp}, \text{pk}, \text{sk}, \text{ct}) \leftarrow \text{Hybrid } 1] - \Pr[\mathcal{D}(\text{pp}, \text{pk}, \text{sk}, \text{ct}) = 1 : (\text{pp}, \text{pk}, \text{sk}, \text{ct}) \leftarrow \text{Hybrid } 2] \right| \leq \text{negl}(\lambda),$$

whenever $\gamma < k - 2\ell\gamma_1 - 2\log\left(\frac{1}{\text{negl}(\lambda)}\right)$.

This proof is identical to the message-hiding proof, except the conditional entropy of the input to the hash function is k instead of δ , for some $k < \delta$. $\square$